Library User Manual

AXL library family
AXA

AJINEXTEK AXT

Product Information

Full information about other AJINEXTEK products
is available by visiting our Web Site at:
Home Page : www.ajinextek.com
E-Mail ::support@ajinextek.com

Useful Contact Information

Customer Support Seoul
Tel : 82-031-436-2180~2 Fax: 82-031-436-2183

Customer Support Cheonan
Tel : 82-041-555-9771~2 Fax: 82-041-555-9773

Customer Support Deagu
Tel : 82-053-593-3700~2 Fax: 82-053-593-3703

AJINEXTEK's sales team is always available to assist you in making your decision the final choice of boards or systems is solely and wholly theresponsibility of the buyer. AJINEXTEK's entire liability in respect of the board or systems is as set out in AJINEXTEK's standard terms and conditions of sale

ⓒ Copyright 2010. AJINEXTEK co.ltd. All rights reserved.

Content

CONTENT 3

HOW TO USE AIO LIBRARY 7

Introduction of AIO Library 7

Basic rules of AIO APIs 8

Description of AIO API group 9

Initialization and Exiting 10

Initialization of Library 10

Exiting Library 12

Analog Input 15

Input module set and check 15

A/D conversion of input module 21

Event/Interrupt 31

Setting Event Mask 31

Setting Use of Event 32

Setting the Process Method of Event 32

Setting Interrupt Mask 40

Analog Output 42

Setting output module 42

Analog output 43

AIO COMMAND MANUAL INFORMATION 48

Header file 48

Terms of API 49

API names in this manual 49

Argument names in this manual 49

AIO COMMAND QUICK LIST 51

API group for trigger mode **51**
Board and module information **51**
Input module information search **52**
Set and check input module interrupt and channel event **52**
Set and check input module parameter **52**
A/D conversion of input module **53**
Checking buffer status of input module **53**
Output module information search **54**
Setting and checking output module **54**
Define Sentence **55**

AIO COMMAND FUNCTION LIST **57**

Board and Module Information **57**

AxaInfoIsAIOModule 58
AxaInfoGetModuleNo 60
AxaInfoGetModuleCount 62
AxaInfoGetInputCount 64
AxaInfoGetOutputCount 66
AxaInfoGetChannelNoOfModuleNo 68
AxaInfoGetModule 70

Input Module Information Search **72**

AxaiInfoGetModuleNoOfChannelNo 73
AxaiInfoGetChannelCount 75

Input Module Interrupt/Set and Check Channel Event **77**

AxaiEventSetChannel 78
AxaiEventSetChannelEnable 81
AxaiEventGetChannelEnable 83
AxaiEventSetMultiChannelEnable 85
AxaiEventSetChannelMask 87
AxaiEventGetChannelMask 89
AxaiEventSetMultiChannelMask 91

AxaiEventRead 93
AxaiInterruptSetModuleMask 95
AxaiInterruptGetModuleMask 97

Setting and Checking Input Module Parameter 99

AxaiSetRange 100
AxaiGetRange 102
AxaiSetMultiRange 104
AxaiHwSetBufferOverflowMode 107
AxaiHwGetBufferOverflowMode 109
AxaiHwSetMultiBufferOverflowMode 111
AxaiHwSetLimit 113
AxaiHwGetLimit 115
AxaiHwSetMultiLimit 117
AxaiSetTriggerMode 119
AxaiGetTriggerMode 121
AxaiHwSetSampleFreq 123
AxaiHwGetSampleFreq 125
AxaiHwSetSamplePeriod 127
AxaiHwGetSamplePeriod 129
AxaiSetModuleOffsetValue 131
AxaiGetModuleOffsetValue 133

A/D Conversion of Input Module 135

AxaiSwReadVoltage 136
AxaiSwReadDigit 138
AxaiSwReadMultiVoltage 140
AxaiSwReadMultiDigit 142
AxaiHwStartMultiChannel 144
AxaiHwStartMultiFilter 146
AxaiHwStopMultiChannel 148
AxaiHwSetMultiAccess 150
AxaiHwStartMultiAccess 153
AxaiHwReadSampleVoltage 155

AxaiHwReadSampleDigit 157

Checking the Buffer Status of Input Module 159

AxaiHwIsBufferEmpty 160

AxaiHwIsBufferUpper 162

AxaiHwIsBufferLower 164

AxaiHwReadDataLength 166

Output Module Information Search 168

AxaoInfoGetModuleNoOfChannelNo 169

AxaoInfoGetChannelCount 171

Setting and Checking Output Module 173

AxaoSetRange 174

AxaoGetRange 176

AxaoSetMultiRange 178

AxaoWriteVoltage 181

AxaoWriteMultiVoltage 183

AxaoReadVoltage 185

AxaoReadMultiVoltage 187

Error Code Table Check 189

How to use AIO library

Introduction of AIO Library

AIO (Analog Input Output) is a control module to take in or to make an output of Analog signal. AIO module can be divided into two groups compared to PCI-A116HR single unit board: SIO-AI16HB is an AI module that has 16 analog input channels; SIO-AO2HB is an AO module that has 2 analog output channels. If more channels are required, extension of channel is possible through a single unit board and module.

AXL Library provides ample amount of function APIs for Analog input and output modules. Library APIs for AIO module are consisted of 63 APIs and classified as following at large.

- Initialization
- Module information check
- Setting and information check of output module
- Setting and information check of input module
- Event setting of input module
- Setting interrupt mask of input module
- A/D conversion
- Buffer state check on input module

In the User Guide all descriptions are based on PCI-AI16HR (SIO-AI16HB/SIO-AO2HB) single unit board that has 16bit resolution among a few types of analog input/output module. The range of input voltage is based on SIO-AI4RB standard.

AXA products that are supported in the library as follows.

Name	Explanation	Whether interrupt support	Note
AXT_SIO_AI4RB	Input 4Ch, 12bit	Support	
AXT_SIO_AO4RB	Output 4Ch, 12bit	Not support	
AXT_SIO_AI16H	Input 4Ch, 16bit	Support	
AXT_SIO_AO8H	Output 4Ch, 16bit	Not support	
AXT_SIO_AI16HB	Input 16Ch, 16bit	Support	
AXT_SIO_AO2HB	Output 2Ch, 16bit	Not support	
AXT_SIO_RAI8RB	Input 8Ch, 16bit	Not support	RTEX
AXT_SIO_RAO4RB	Output 4Ch, 16bit	Not support	RTEX
AXT_SIO_RAI4MLII	Input 4Ch, 16bit	Not support	Mechatrolink II
AXT_SIO_RAO4MLII	Output 2Ch, 16bit	Not support	Mechatrolink II

Basic rules of AIO APIs

As a basic rule for all AXL APIs, AXA which falls in AIO API group returns the result of API execution. The retured value shows that if a API performed well or not for certain reason.

Do as follow if you want to check the return value of API.

```
DWORD Code = AxaInfoIsAIOModule(&dwStatus);

if(Code == AXT_RT_SUCCESS)
  printf ("it has been run properly.”);
else
  printf ("AxaInfoIsAIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
```

Error Code	Description
[0000] AXT_RT_SUCCESS	API function executed successfully
[1001] AXT_RT_OPEN_ERROR	Library is not open
[1002] AXT_RT_OPEN_ALREADY	Library is already open
[1053] AXT_RT_NOT_OPEN	Failed to initialize the library
[1054] AXT_RT_NOT_SUPPORT_VERSION	Hardware not supported
[1101] AXT_RT_INVALID_BOARD_NO	Invalid board number
[1102] AXT_RT_INVALID_MODULE_POS	Invalid module position
[1103] AXT_RT_INVALID_LEVEL	Invalid level
[1151] AXT_RT_ERROR_VERSION_READ	Failed to read library version
[2001] AXT_RT_AIO_OPEN_ERROR	Failed to open AIO module
[2051] AXT_RT_AIO_NOT_MODULE	No AIO module
[2052] AXT_RT_AIO_NOT_EVENT	Failed to read AIO event
[2101] AXT_RT_AIO_INVALID_MODULE_NO	Invalid AIO module
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO	Invalid AIO channel number
[2106] AXT_RT_AIO_INVALID_USE	Can not use AIO
[2107] AXT_RT_AIO_INVALID_TRIGGER_MODE	Invalid trigger mode

Also, APIs take inputs of necessary arguments through parameters. For output, APIs take API pointer and store processed result to deliver it to user. As seen below, to execute AxaExFunction, the values for inPara1 and inPara2 are required and if the result values coming from outPara1 and outPara2 are as follow.

```
double inPara1, inPara2;
double outPara1, outPara2;
AxaExFunction(inPara1, inPara2, &outPara1, &outPara2);
```

Description of AIO API group

AIO API is classified into API prefix groups as follows.

Prefix Category	Explanation
Axai	Category related to Analog input
AxaiInfo	API for board and module information check
AxaiInfo	API for input module information search
AxaiEvent	API for event setting and check of input channel
AxaiInterrupt	API for interrupt setting and check of input module
AxaiSw	API related to Software Timer (Normal Trigger Mode) method
AxaiHw	API related to Hardware Timer(Timer Trigger + External Trigger Mode) method
Axao	Category related to Analog output
AxaoInfo	API for output module check

Initialization and Exiting

Initialization of AXL library is done by executing AxlOpen API. If AxlOpen API is executed, not only the initialization of library internally, but also are installed boards and the modules on the boards initialized. Therefore, user must check if any AIO module exists before using AIO library.

Initialization of Library

● Initialization of Library

AxlOpen: AxlOpen is an API that initializes library. When user uses interrupt, IRQ number for processing interrupt must be registered. Especially, for base board of PCI type, IRQ number is neglected for library initialization was IRQ number is automatically created.

This API returns BOOL value, not error code so it can be used as below.

```
// Initialize library.
// 7 means IRQ. In PCI, IRQ is automatically set.
if (AxlOpen(7) == AXT_RT_SUCCESS)
  printf("Library has been initialized.");
```

● Checking library initialization

AxlIsOpened: AxlIsOpened is an API that checks the initialization status of library. This API also returns BOOL value, not error code. If library is initialized, it returns TRUE. If library is not initialized, it returns FALSE.

```
// Check the initialization status of library.
if (AxlIsOpened())
  printf("Library has been initialized.");
else
  printf("Library has not been initialized.");
```

● Checking existence of AIO module

AxaInfoIsAIOModule: AxaInfoIsAIOModule is an API to check the existence of actual AIO module. It is used to check that there is AIO module before using APIs related AIO. To check the existence of module, do as follows.

```
// AIO Check if there is AIO module.
DWORD dwStatus;
AxaInfoIsAIOModule(&dwStatus);
if(dwStatus == STATUS_EXIST)
  printf("There is AIO module.");
else
  printf ("There is no AIO module.");
```

● Checking the number of input/ouput module and channel

AxaInfoGetModuleCount: AxaInfoGetInputCount is an API that checks the number of input channeles in designated AIO module.

```
// Check the number of AIO module.
long    lCount;
AxaInfoGetModuleCount(&lCount);
printf("The number of AIO module is %d.", lCount);
```

AxaInfoGetInputCount: AxaInfoGetOutputCount is an API that checks the number of output channels in designated AIO module.

```
// Check the number of input channel in 0th module.
long    lCount;
AxaInfoGetInputCount(0, &lCount);
printf("The number of input channel of 0th AIO module is %d.", lCount);
```

AxaInfoGetOutputCount: AxaiInfoGetChannelCount is an API that checks the number of entire input channel.

```
// Check the number of output channel in 0th module.
long    lCount;
AxaInfoGetOutputCount(0, &lCount);
printf("The number of output channel of 0th AIO module is %d.", lCount);
```

AxaiInfoGetChannelCount: AxaiInfoGetChannelCount is an API that checks the number of entire input channel.

```
// Return the number of entire registered analog input channels
long lAdcChannelCounts;
Code = AxaiInfoGetChannelCount(&lAdcChannelCounts);
printf("The number of entire registered analog input channels is %d.", lAdcChannelCounts);
```

AxaoInfoGetChannelCount: AxaoInfoGetChannelCount is an API that checks the number of entire output channels.

```
// Return the number of entire registered analog output channels
long lDacChannelCounts;
Code = AxaoInfoGetChannelCount(&lDacChannelCounts);
printf("The number of entire registered analog output channels is %d.", lAdcChannelCounts);
```

AxaiInfoGetModuleNoOfChannelNo: AxaiInfoGetModuleNoOfChannelNo is an API that checks the module number of the designated input channel.

```
// Return the module number of designated input channel
long lModuleNo1, lModuleNo2;
Code = AxaiInfoGetModuleNoOfChannelNo(0, &lModuleNo1);
Code = AxaiInfoGetModuleNoOfChannelNo(20, &lModuleNo2);
printf("Module number of the 0th input channel: %d\n", lModuleNo1);
printf("Module number of the 20th input channel: %d\n", lModuleNo2);
```

AxaoInfoGetModuleNoOfChannelNo: AxaoInfoGetModuleNoOfChannelNo is an API that checks the module number of designated input channel.

```
// Return the module number of designated input channel
long lModuleNo1, lModuleNo2;
Code = AxaoInfoGetModuleNoOfChannelNo(0, &lModuleNo1);
Code = AxaoInfoGetModuleNoOfChannelNo(2, &lModuleNo2);
printf("Module number of the 0th output channel: %d\n", lModuleNo1);
printf("Module number of the 2nd output channel: %d\n", lModuleNo2);
```

● The meaning of module and channel number

With PCI-AI16HR as standard, PCI-AI16HR has 16 channel input module(SIO-AI16H) and 2 channel input module(SIO-AO2HB). For the case that 2 PCI-AI16HR boards are mounted on the system, allotted index results for module and each channel are as follows.

For the case of mouting 2 PCI-AI16HR boards, the allotting result for module and channel number

Module Number	Input/Output	Channel Index
0	Input(SIO-AI16HB)	0~15
1	Output(SIO-AO2HB)	0~1
2	Input(SIO-AI16HB)	16~31
3	Output(SIO-AO2HB)	2~3

Channels 0~15 and 16~31 whose modules are different can take different methods to collect data for each type of module by setting different hardware of Sample rate, Trigger Mode, and interrupt mask. For example, as seen above, channels 0~16 whose module number is 0 collect data through a method of Software Trigger Mode, channels 16~31 whose module number is 2 collect data through a method of External Trigger Mode.

Exiting Library

AxlClose: AxlClose is an API that ends library. After using library, library must be shut down to return the allotted memory. Return TRUE if library is shut down after the Successful execution of API. If not, return FALSE.

```
// Exit library.
if (AxlClose())
  printf("Library has been exited.");
```

```
// Ex1_AXA_InitAndClose.cpp : Defines the entry point for the console application.
// Exit library after initializing and checking input/output information.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"
```

```
void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is automatically set in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library has been initialized.\n");

      // Check the existence of AIO module
      DWORD dwStatus;
      Code = AxaInfoIsAIOModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            printf("There is AIO module.\n");
            // Check the number of AIO module
            long lModuleCounts;
            Code = AxaInfoGetModuleCount(&lModuleCounts);
            if (Code == AXT_RT_SUCCESS)
               printf("The number of AIO module: %d\n",lModuleCounts);
            else
               printf("AxaInfoGetModuleCounts() : ERROR code 0x%x\n",Code);

            // Check the number of input/output channel
            long lInputCounts;
            long lOutputCounts;
            for(int ModuleNo=0;ModuleNo<lModuleCounts;ModuleNo++)
            {
               Code = AxaInfoGetInputCount(ModuleNo,&lInputCounts);
               DWORD Code2 = AxaInfoGetOutputCount(ModuleNo,&lOutputCounts);

               if (Code == AXT_RT_SUCCESS || Code2 == AXT_RT_SUCCESS)
                  printf ("%dth module : %d input channels, %d output channels \n",
                        ModuleNo, lInputCounts, lOutputCounts);
               if (Code != AXT_RT_SUCCESS)
                  printf ("AxaInfoGetInputCounts() : ERROR   code 0x%x\n",Code);
               if (Code2 != AXT_RT_SUCCESS)
                  printf ("AxaInfoGetOutputCounts() : ERROR   code 0x%x\n",Code2);
            }

            // Return the number of input channel of entire registered input channels
            long lAdcChannelCounts;
            Code = AxaiInfoGetChannelCount(&lAdcChannelCounts);
            printf("The number of the entire registered input channel is %d.\n",
               lAdcChannelCounts);
```

```
            // Return the number of input channel of the entire registered output channels
            long lDacChannelCounts;
            Code = AxaoInfoGetChannelCount(&lDacChannelCounts);
            printf("The number of the entire registered output channel is %d.\n",
               lAdcChannelCounts);
         }
         else
            printf ("AxaInfoIsAIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
      }
      else
         printf ("AxaInfoIsAIOModule() : ERROR ( Return FALSE ) code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("Library has been exitted.\n");
   else
      printf("Library has not been exited properly.\n");
}
```

< Output result for the case of 2 PCI-AI16HR boards mounted >

Library has been initialized.

There is AIO module.

The number of AIO module: 4

0th module: 16 input channels, 0 output channel

1st module: 0 input channel, 2 output channels

2nd module: 16 input channels, 0 output channel

Analog Input

Analog input module is a module that transforms input analog signal of voltage or current type to ditigal data after sampling of set frequency or interval.

Input module set and check

1) Setting the range of input voltage

```
Description of the range of input voltage below is based on SIO-A14RB as standard. SIO-A14RB
 whose resolution is 16 bit has the input voltage range as -10~10V.
```

The setting condition for using input module is the range of input voltage. To set the allowed input voltage range of designated input channel, AxaiSetRange API is utilized. The setting of proper standard volage is important as it has an impact on performance improvement.

Allowed input voltage is set for each module. If user inputs input voltage other than that, a value containg the input voltage is automatically set. Minimum input voltage means the lowest voltage among the allowed voltages for analog input modules. The value that user has set is set as a closet value containing the minimum allowed voltage that hardware provides. For SIO-A14R, the minimum standard voltages are -10V, -5V, and 0V, so if the user inputs -8V, it is automatically set as -10V. Maximum input voltage means the the biggest voltage among the allowed voltages for analog input. The value that user has set is set as a closet value containing the maximum allowed voltage that hardware provides. For SIO-A14R, the maxinum standard voltages are 10V and 5V, so if the user inputs 8V, it is automatically set as 10V.

As channel 0 and channel 1 are linked, channel 2 and 3 are linked, the range of input voltage for them is set as the same. That is, if one channel's range is set, all the other channels' ranges are automatically set. For example, as channel 0 and 1 are linked, the range of volage set for input channel 0 is applied for channel 0.

Module	Min. Voltage	Max. Voltage	Min. Measurement Unit
SIO-AO4R	-10V	10V	4.88mV
	-5V	5V	2.44mV
	0V	10V	2.44mV
	0V	5V	1.22mV
SIO-AO2HB(PCI-AI16HR)	-10V	10V	0.305mV

AxaiSetRange: To set the range of input voltage, AxaiSetRange API is used. To set channel 0's input voltage range from -8V to 8V, do as follows.

```
// Set the range of output voltage for channel 0 from -8V to 8V
DWORD dwReturn = AxaiSetRange(0,-8,8);
// For SIO_AI16HB module (input module of single unit board PCI-AI16HR), as the voltage
// range is fixed, it is returned to AXT_RT_AIO_INVALID_USE.
```

If set as above, -10V and 10V are automatically selected and set. Under this setting condition, 9V is taken as input, 9V is taken as a proper input. It is because that although the setting was -8V~8V, the actual setting value is -10V~10V.

AxaiSetMultiRange: There is AxaiSetMultiRange API that sets voltage ranges for a few input channels at the same time, as opposed to the case seen above that one range was set for one channel. If you want to set the voltage range of 0~3 input channel to -10V~10V, follow as below.

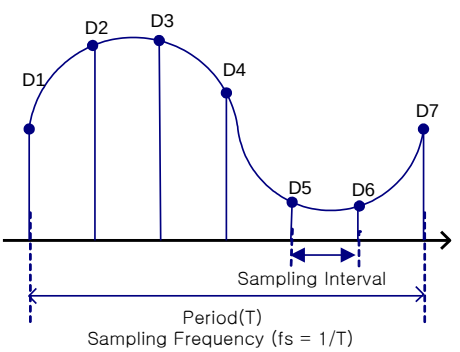
```
// Set the input voltage ranges of several channels
long AdcChannelNo[4] = {0,1,2,3};
DWORD dwReturn = AxaiSetMultiRange(4,AdcChannelNo,dMinVolt,dMaxVolt);
// For SIO_AI16HB module, as the voltage range is fixed, it is returned to
// AXT_RT_AIO_INVALID_USE.
```

AxaiGetRange: The standard voltage that is not supported by hardware is set, the setting value and the actually set value can be different. If you wanto to know which value has been actually set, AxaiGetRange API can be used. If you want to know the set value for channel 0, do as follows.

```
// Check the output voltage range of 0th output channel.
double dVmin;
double dVmax;
AxaiGetRange(0, &dVmin, &dVmax);
```

2) Setting the sampling mode

If the range of input voltage has been set, sampling method must be set. Sampling is a process that transforms analog value into corresponding digital value by extracting input analog voltage by fixed interval, sampling interval noted as Δt, All transformed data are stored in FIFO memory within module.



AxaiSetTriggerMode: The process to determine how to do the sampling is the selection of trigger mode. Trigger modes are divided into Normal Trigger, Timer Trigger, External Trigger. The difference of trigger mode lies in the moment that sampling is made. The well-known sampling method done by specific sampling frequency or sampling interval is Timer Trigger. Normal Trigger mode is a sampling method done at the moment user determines such as specific time or condition using software. External Trigger mode is a sampling method by specific external signals. For SIO-AI16HB module, as internal ADC is just one, only one trigger mode can be applied to one module. The setting value for each module is as follows.

Define	Value	Trigger Mode	Function
NORMAL_MODE	1	Normal Trigger	Trigger method that performs A/D conversion at the moment user wants
TIMER_MODE,	2	Timer Trigger	Trigger method that performs A/D conversion by using H/W internal click
EXTERNAL_MODE	3	External Trigger	Trigger method that performs A/D conversion by using external input terminal's click

For example, the voltage change coming from strain gauge attached to a sample in a strain test, continuous data at constant interval can be acquired by using Timer Trigger mode. However, for a program that performs a work by measuring the current state at a specific moment, all data are not needed to be measured but sampling can be done at a moment that user wants by using Normal Trigger.

As each trigger mode has different characteristic as above, APIs for setting each trigger mode are given.

Trigger Mode	APIs to Use
Normal Trigger Mode	AxaiSwReadVoltage, AxaiSwReadDigit AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit
Timer Trigger Mode	AxaiHwStartMultiChannel, AxaiHwStartMultiFilter AxaiHwStopMultiChannel AxaiHwSetMultiAccess, AxaiHwStartMultiAccess
External Trigger Mode	AxaiHwStartMultiChannel, AxaiHwStartMultiFilter AxaiHwStopMultiChannel

If Normal Trigger mode is going to be used, do as follows.

```
// Set Timer Trigger mode of 0th input module.
AxaiSetTriggerMode(0,NORMAL_MODE);
```

If Normal Trigger Mode is used, given APIs such as AxaiSwReadVoltage for instance, can be used for reading value. But for Timer Trigger mode using internal timer, additional sampling frequency must be set. For continuous singal sampling not Immediate Mode, Interrupt Mask setting and a method to receive buffer size and the moment that the upper and lower limit of buffer is reached in double buffering to prevent data loss msut be set by related APIs.

There are two methods to set sampling interval. First one is setting the sampling interval itself. The second one is to set sampling frequency.

Sampliing frequency which is a reverse of sampling period means how many samplings are done per a second. Of course, the higher the frequency is, the smaller the interval is done to give finer measurement and vice versa. In theory, the sampling frequency should be more than 2 times that of analog singal to be measured. If not, correct measurement is greatly hindered. However, in fact, it is better to use the frequency as high as 5 to 10 times that of analog signal. AxaiHwSetSamplePeriod is an API that sets the sampling interval in the unit of time. AxaiHwSetSampleFreq is an API that sets the sampling frequency in the unit of frequency. As two APIs are to set sampling frequency, one of them can be used. For SIO-AI16HB, all 16 internal channels use a common ADC(Analog Digital Converter), all 16 channels's sampling is done at one sampling frequency or period.

For example, for SIO-AI16HB, if sampling for one channel is done, sampling can be done at maximum 100Khz. If sampling for all 16 channels are done, the sampling frequency that is to be set by AxaiHwSetSampleFreq should not run over (100K/16 = 6250) Hz. However, if a value over the maximum sampling limit set by hardware is typed in, the sampling frequency is automatically figured again to be lower than the upper limit. For SIO-AI4RB whose hardware is different from SIO-AI16HB, as it uses 4 channel ADC, the selected number of channel does not impose limit on the frequency. However, the sampling frequency for SIO-AI4RB could not run over 100Khz, the ADC sampling limit.

AxaiHwSetSamplePeriod: If the input analog data for module 0 should be sampled at 100ms intervnal, set as follows. The unit here is uSec.

```
// Set the sampling period of module 0 as 100ms.
// 100 uSec
AxaiHwSetSamplePeriod(0,100);
```

AxaiHwGetSamplePeriod: If you want to verify the sampling period set in specific module, AxaiHwGetSamplePeriod API can be used as seen below. The unit here is uSec.

```
// Check the sampling period of module 0.
double lSamplePeriod
AxaiHwGetSamplePeriod(AI_0,&lSamplePeriod);
```

AxaiHwSetSampleFreq: If you want to have a sampling of input analog data for module 0 at 10 Hz sampling frequency, do as follows. The unit is Hz.

```
// Set the sampling frequency of module 0 as 10 Hz.
AxaiHwSetSampleFreq(0,10);
```

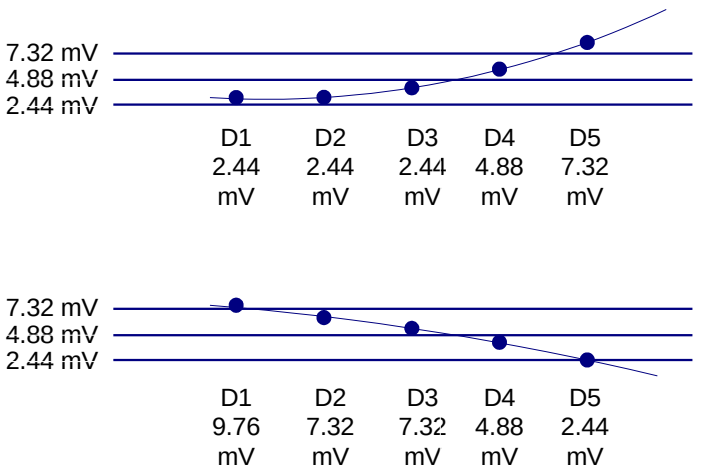
AxaiHwGetSampleFreq: If you want to check the current sampling frequency set for a certain module, use AxaiHwGetSampleFreq API as shown below. The unit is Hz.

```
double dSampleFreq;
AxaiHwGetSampleFreq(0,&dSampleFreq);
```

3) Setting buffer for input channel

For SIO-AI16HB module, to have a sampling with 16 bit resolution, there are 216=65536 pieces of values to be stored between the maximum and minimum voltage value.

Therefore, for SIO-AI16HB module whose voltage range is set between -10V and 10V, the measurable minimum unit is about 0.3 mV, which is 20V/65536. For SIO-AI4RB that uses 12 bit resolution for adjusting input voltage range, if the voltage range is set as 0V~10V, the measurable minimum unit is 2.44mV, which is 10V/1024. If the input is given above the measurable minimum value, the previous value becomes the result value as seen below.



The sampling data made out of this process are stored in FIFO memory. Though each module has different FIFO size, the sampling data can be obtained without losing data by using FUNCTION that AIO library provides.

AxaiHwSetBufferOverflowMode: When it comes to performing continuous signal inspection, as FIFO size inside module has a limit in terms of hardware, double buffering is done to prevent data loss. Then, if the butter could not receive data at the proper moment due to the lack of buffer size set for double buffering, overflow happens. AxaiHwSetBufferOverflowMode is an API to deal with the OverFlow, When the allotted memory buffer is full of data and a new datum is coming in, the buffer has to either get rid of the first data to store the new datum or neglect the new datum.

When memory buffer allotted for each channel is full of data, AxaiHwSetBufferOverflowMode API gets rid of the old data to accept the new datum if NEW_DATA_KEEP is given as an input. If CURR_DATA_KEEP is given as an input, it neglects the new datum to retain the old data. .

For the case that the buffer of input channel 0 is full of data, do as follows if you want to accept the new datum after getting rid of the old data.

```
// Set to accept the new datum when buffer of input channel 0 is full of data
AxaiHwSetBufferOverflowMode(0,NEW_DATA_KEEP);
```

AxaiHwSetMultiBufferOverflowMode: As opposed to the above described case that an API sets one channel at a time, AxaiHwSetMultiBufferOverflowMode is an API that sets a few channels at the same time.

If you want to set to get rid of new data and maintain olad data when buffer of input channel 0~3 is full, do as follows.

```
// Set to accept the new datum when buffer of input channel 0~3 is full
long AdcChannelNo[4] = {0,1,2,3};
AxaiHwSetMultiBufferOverflowMode(4, AdcChannelNo,CURR_DATA_KEEP);
```

AxaiHwSetLimit: For continous signal check that is to tranfrom data, data transfer is done from FIFO memory to PC memory buffer that user has set through the internal interrupt. The current state of memory buffer can be obtained from event, window message, and collect API. For this case, the maximum and

minimum value for memory buffer from which event comes is set by AxaiHwSetLimit API. If the number of data transferred from FIFO to memory buffer is over the upper limit set by the user, the user receives the event to get the proper moment to read the buffer. If you want to set the upper limit of input channel 0 as 1000 and the lower limit as 10, do as follows.

However, the upper value should not be bigger than the actual memory buffer size that is to be set in AxaiHwStartMultiChannel , which is an API that will be used for obtaining continuous data.

```
// Set the uppoer limit as 1000 and lower limit as 10 of input channel 0
AxaiHwSetLimit(0,10,1000);
```

AxaiHwGetLimit: If you want to check the upper and lower limit of a specific input channel, do as follows using AxaiHwGetLimit API.

```
// Check the upper and lower limit of input channel 0
long lLowLimitGet, lUpLimitGet;
AxaiHwGetLimit(AdcChannelNo,&lLowLimitGet,&lUpLimitGet);
```

AxaiHwSetMultiLimit: As opposed to the above described case that an API sets one channel at a time, AxaiHwSetMultiLimit is an API that sets several channels at once.

If you want to set the upper and lower limit of input channel 0~3 as 1000 and 10, respectively, do as follows.

```
// Set the upper and lower limit for buffers for several channels
long AdcChannelNo[4] = {0,1,2,3};
Code = AxaiHwSetMultiLimit(4,AdcChannelNo,10,1000);
```

AxaiInterruptSetModuleMask: While performing continuous signal check, AxaiInterruptSetModuleMask API can be used to determine the moment to copy from FIFO of hardware module to memory buffer by adjusting the occurrence moment of the internal interrupt. SIO-AI4RB is not provided.

The types of interrupt mask are as follows.

(Trigger mode is included in TIMER_MODE and EXTERNAL_MODE.)

FIFO_HALF_FULL : When FIFO memory inside module is filled with data more than its half capacity, internal interrupt occurs to send a copy of data to memory buffer. (For high speed sampling)

SCAN_END : For all set channels, each time ADC conversion is made, interrupt occurs to send a copy of data to memory buffer. (For low speed sampling-recommended to use when trigger signal is between 1~100 Hz)

The default setting value is FIFO_HALF_FULL.

```
// Decide hardware interrupt mask for module.

Code = AxaiInterruptSetModuleMask(0, SCAN_END);
// Use SCAN_END for low speed sampling (1~100 Hz) under External or Timer Trigger Mode.
```

If the interrupt mask is set as SCAN_END while high speed sampling, overhead becomes noticeable as each ADC conversion induces the interrupt. Contrarily, if the interrupt mask is set as FIFO_HALF_FULL while low speed sampling, the time delay for data becomes noticeable as the data copy is done as long as the data reach the half of FIFO size.

For example, when a sampling is done for a channel at 10 Hz, if FIFO_HALF_FULL is set, the collected data can be read after the time delay of about 51 seconds. Because 512 data must be collected for the copy to memory buffer as the FiFO size of SIO_AI16HB is 1024. However, if SCAN_END is set, 0.1 second which is the time for a single sampling required for the first collected datum to be read in memory buffer. Also, if SCAN_END is set for having a sampling for a channel at 100Khz frequency, at every 10 usec, interrupt occurs and makes data copy to cause a huge degree of overhead. For this case, set FIFO_HALF_FULL then at every 5msec, 512 data is copied at once so the load from overhead does not occur.

A/D conversion of input module

1) A/D conversion in Normal Trigger Mode

You can sampling the state of the moment you want by using software in Normal Normal Trigger Mode. Sampling of input analog signal and its A/D are executed by calling up AxaiSwReadVoltage or AxaiSwReadDigit API at a specific moment or under a certain circumtstance then A/D coversion of it takes place.

AxaiSwReadVoltage: AxaiSwReadVoltage API converts the current analog input value of the designated input channel to voltage value after A/D conversion. If you want to check the analog input value of channel 0 as voltage value, do as follows.

```
// Check the analog input value of channel 0 as voltage value.
double dVolt;
AxaiSwReadVoltage(0,&dpVolt);
```

AxaiSwReadDigit: AxaiSwReadDigit API converts the analog input value of the input channel to Digit value after A/D conversion. As SIO-AI16HB has 16 bit resolution, digit value ranges from 0 to 65535. If you want to check the analog input value of channel 0 as digit value, do as follows.

```
// Check the analog input value of channel 0 as digit value.
DWORD dpDigit;
AxaiSwReadDigit(0,&dpDigit);
short retShort = (short) dpDigit;     // Converstion to signed 2 Byte
double dVolt = (((double)retShort ) / 32768.0) * 10;
// As SIO-AI16HB digit value is two's complement, the conversion following the above
// procedure will give voltage value.
```

```
// Ex2_AXA_NormalTrigger.cpp : Defines the entry point for the console application.
// Intialize library and exit after checking input/output information.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
```

```
// Initialize library.
// 7 means IRQ. IRQ is automatically set in PCI.
DWORD Code = AxlOpen(7);
if (Code == AXT_RT_SUCCESS)
{
   printf("Library has been initialized.\n");

   // Check the existence of AIO module
   DWORD dwStatus;
   Code = AxaInfoIsAIOModule(&dwStatus);
   if(Code == AXT_RT_SUCCESS)
   {
      if(dwStatus == STATUS_EXIST)
      {
         // Set the range of input voltage for a few channels as -10V ~ 10V
         long AdcChannelNo[4] = {0,1,2,3};
         AxaiSetMultiRange(4,AdcChannelNo,-10,10);

         // Set the trigger mode of module 0 as NORMAL
         AxaiSetTriggerMode(0,NORMAL_MODE);

         // Take analog input as going round the infinite loop.
         printf("[INFORMATION]******************************** \n");

         // Check the set range of input voltage.
         double dVmin, dVmax;
         for(int nChannalNo=0; nChannalNo<4; nChannalNo++)
         {
            AxaiGetRange(0, &dVmin, &dVmax);
            printf("Voltage range of input module: Channel %d(%.1f~%.1f)
               \n" ,nChannalNo,dVmin,dVmax);
         }

         printf("[ESC] : Exit \n");
         printf("[1] : Sampling in Normal Trigger Mode (voltage value )\n [1] : Normal
            Trigger Mode \n");
         printf("[2] : Sampling in Normal Trigger Mode (Digit)\n [2] : Normal Trigger
            Mode (Digit)\n
         printf("****************************************** \n");

         BOOL fExit = FALSE;
         while (!fExit)          // Infinite loop
         {
            if (kbhit())         // Push any keys
            {
               int ch = getch();
               DWORD dpDigit;
               double dpVolt;
               int nChannalNo;
```

```
                switch (ch)
                {
                   case 27:       // Esc key
                      fExit = TRUE;
                   break;

                   case '1':
                      // Check the current input value as voltage value
                      for(nChannalNo=0; nChannalNo<4; nChannalNo++)
                      {
                         AxaiSwReadVoltage(nChannalNo,&dpVolt);
                         printf("The current input voltage of channel %d : %f V
                            ",nChannalNo,dpVolt);
                      }
                   break;

                   case '2':
                      // Check the current input value as digit value
                      for(nChannalNo=0; nChannalNo<4; nChannalNo++)
                      {
                         AxaiSwReadDigit(nChannalNo,&dpDigit);
                         printf("\n The current input Digit of channel %d : %f V
                            ",nChannalNo,dpDigit);
                      }
                   break;
                }
             }
          }
       }
       else
          printf ("AxaInfoIsAIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
    }
    else
       printf ("AxaInfoIsAIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
 }
 else
    printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

 // Library Exit.
 if (AxlClose())
    printf("Library has been exitted.\n");
 else
    printf("Library has not been exitted properly.\n");
}
```

2) A/D conversion of Timer Trigger Mode

Timer Trigger Mode is an API that continuously performs sampling of input analog singal for a specific channel at the sampling frequency or at the sampling interval set by the user. As noted above, it can be used for measuring continuous signal produced by analog sensors.

AxaiHwStartMultiChannel: AxaiHwStartMultiChannel API is used to perform the sampling for several input channels through Timer Trigger Mode/External Trigger Mode. AxaiHwStartMultiChannel API is operated by receiving 3 arguments of the number of input channel, the array of channel number and the size of the allotted buffer. If channel 0 and 1 are to be used, the number of channel would be 2 and the array of channel number would be 0,1. Although user changes the size of the allotted buffer, it is better to get the buffer size big enough considering the time delay between taking in sampling rate and buffer upper event and removing contents in the buffer. For example, for SIO-AI16HB, if the sampling for one channel is done at 100Khz, every time 512 data are collected the data is transferred from FIFO inside module to PC buffer memory. Thus, 512 data are collected in 5msec and if the upper value is set as 2048, each event can be received at every 20msec. Therefore, for setting buffer size, big enough space must be allotted considering the fact that the data loss during the time data are processed in application must be prevented. As seen in the above example, for high speed sampling, if the time for processing the maximum load is expected tob e 40msec, the minimum buffer size should be bigger than 8 times (4096~)as half size of FIFO. If N channels are upon collecting data at the same time, as the internal for event for each channel becomes N times bigger under the same upper value, the upper limit needs to be set accordingly. Of course, the upper value could not run over the buffer size. The setting of the upper limit can be done by AxaiHwSetMultiLimit or AxaiHwSetLimit API before calling up AxaiHwStartMultiChannel API.

If you only want to use channel 0 and 1, and allot 1024 buffers for each channel, do as follows.

```
// Start A/D conversion allotting 1024 buffers for channel 0 and 1.
long AdcChannelNo[2] = {0,1};
AxaiInterruptSetModuleMask(0, FIFO_HALF_FULL);
// Set the lower and upper limit for taking in the state event of buffer.
AxaiHwSetMultiLimit(2, AdcChannelNo, 0, 256);
// Set the buffer for each channel
AxaiHwStartMultiChannel (2, AdcChannelNo, 1024);
// For SIO-AI16HB with the code above, if FIFO is bigger than 512 it moves to buffer
// As 2 channels are under inspection, if the upper limit for each buffer is set to 256,
// each time of FIFO_HALF_FULL and interrupt reaches the upper limit of the buffer so
// AIO_EVENT_DATA_UPPER occurs
// For the code above, data is collected for each time of internal interrupt
// It is not required for high speed sampling as it causes system load
// For high speed sampling, set the upper limt of buffer (512 per channel) as N times as
// that of normal one. Here, N is set from 2 to a few hundred proportional to sampling
// frequency
```

AxaiHwStopMultiChannel: A/D conversion of each module to which the channels checking the continuous signal is exitted by AxaiHwStartMultiChannel. If the parameter IModuleNo is -1, A/D conversions for all modules which are checking continuous singal and mounted on the system are exitted. Use API as follows.

```
// Exit A/D conversion.
AxaiHwStopMultiChannel (-1);
// Exit A/D conversions for all modules
```

AxaiHwReadSampleVoltage: When A/D conversion is started, the continuous sampling result done at set

sampling interval is stored in PC memory buffer in module FIFO through interrup. AxaiHwReadSampleVoltage API checks the stored sampling result by voltage value. Also, AxaiHwReadSampleDigit API checks the result as digit value. These APIs read the number of data stored to pick the number of data wanted out of the stored ones. As the actual sampling is done continuously very fast according to sampling rate or signal rate of external trigger, the data stored in buffer must be read on by using these APIs if data is checked to exist. Once data is read, buffer clears away. The proper method to investigate the existence of data in buffer is to set the upper limit of buffer through AxaiHwSetMultiLimit and AxaiEventSetChannel APIs. If buffer runs over the upper limit, an event is received to read the number of data stacked in the buffer for each channel at the proper moment. If there is data in buffer, investigation is not required.

If you want to check the analog input value of channel 0 as voltage value, do as follows.

```
// Data stored in input channel, the channel 0, is read as voltage values by the designated
// block.
double dpVolt[1024];
long lCount = 1024;
AxaiHwReadSampleVoltage(0, &lCount, dpVolt);
// The number of data that have been actually read is returned to lCount.
```

AxaiHwReadSampleDigit: AxaiHwReadSampleDigit API converts the analog input value of the designated input channel and returns as digit value. As SIO-AI16HB module has 16 bit resolution, the value would fall between 0 and 65535. If you want to check the analog input value of channel 0 as digit value, do as follows.

```
// Each datum stored in input buffer of input channel 0 is read as digit value.
DWORD lpDigit[1024];
Long lCount = 1024
AxaiHwReadSampleDigit(0,&lCount, lpDigit);  // The number of data that have been actually
                                            // read is returned to lCount
short retShort = (short) lpDigit[0];        // Transform to signed 2 byte type
double dVoltFirst = (((double)retShort ) / 32768.0) * 10;
// As digit value is two's complement, voltage value close to actual value will be given if
// the transformation is done.
```

AxaiHwReadDataLength: AxaiHwReadDataLength API is used to check the number of data in a buffer. The API returns the number of data in the designated memory buffer. As memory buffer is allotted in a mode using Timer Trigger/External Trigger, the API is meaningless in a mode using Normal Trigger. If you want to check the number of data in the buffer of channel 0, do as follows.

```
// Check the number of data in the memory buffer of input channel 0.
long lpDataLength;
AxaiHwReadDataLength(AdcChannelNo,&lpDataLength);
```

```
// Ex3_AXA_TimerTrigger.cpp : Defines the entry point for the console application.
// A/D conversion is done by using Timer Trigger Mode.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
```

```
#include "stdio.h"
#include "time.h"

void __stdcall CallBack_ReadChannelData(long lActiveNo, DWORD uFlag)
{
  // Call back API to take back event when the designated buffer upper limit is reached
  // In this example, call back API is used for window message and event processing method.
  if(uFlag == AIO_EVENT_DATA_UPPER) // For an event that the upper limit is reached
  {
     GraphData graphData;
     double dVolt[2048];
     long lDataLength = 0;
     long lCount = 0;

     DWORD dwReturn = AxaiHwReadDataLength(lActiveNo, &lDataLength);
     if(dwReturn!=0)
     {
        return;
     }
     if(lDataLength > 2048)
        lCount = 2048;
     else
        lCount = lDataLength;
     dwReturn = AxaiHwReadSampleVoltage(lActiveNo, &lCount, dVolt);
     if(dwReturn!=0)
     {
        return;
     }
     /* Collected data as many as Icount is processed here.
        If there is a chance that the code of this part would perform a work that takes the
time delay 512 times that of the sampling period of data collecting, it is highly
recommended to use window message method or event process method, not call back method to
prevent data loss.  */
  }
}

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is automatically set in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library has been initialized.\n");

     // Check the existence of AIO module
     DWORD dwStatus;
     Code = AxaInfoIsAIOModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
```

```
{
   if(dwStatus == STATUS_EXIST)
   {
      long AdcChannelNo[4] = {0,1,2,3};

      // Exit if already inputting.
      AxaiHwStopMultiChannel(-1);
      AxaiInterruptSetModuleMask(0, FIFO_HALF_FULL);
      for(int I = 0; I < 4; i++)
      {
         AxaiEventSetChannel(i, NULL, NULL, (AXT_EVENT_PROC)CallBack_ReadChannelData,
            NULL);                                  // Set call back API to receive
                                                    // event
         AxaiEventSetChannelMask(i, DATA_MANY);     // when the upper limit is reached,
                                                    // call back API is called up
         AxaiEventSetChannelEnable(i, ENABLE);      // Event calling up enabled
      }

      // Set the range of input voltage of several channels as -10V ~10V.
      AxaiSetMultiRange(4,AdcChannelNo,-10,10);

      // Set upper and lower limit of several buffers.
      AxaiHwSetMultiLimit(4,AdcChannelNo,0,1000);

      // Set trigger mode of module 0 as Timer Trigger.
      AxaiSetTriggerMode(0,TIMER_MODE);

      // Set sampling frequency of a specific module as 1000Hz.
      AxaiHwSetSampleFreq(0,1000);

      // Set to keep the data when buffer of several input channels are full.
      AxaiHwSetMultiBufferOverflowMode(4,AdcChannelNo,NEW_DATA_KEEP);

      // Start A/D conversion by allotting 2000 buffers for each input channel.
      long lBuffSize = 2000;
      AxaiHwStartMultiChannel(4,AdcChannelNo,lBuffSize);
      printf("Starting Timer Trigger Mode \n");
      // Take in analog input by going round infinite loop.
      printf("[INFORMATION]************************************ \n");

      printf("[ESC] : Exit \n");
      printf("[1] : Sampling in Timer Trigger Mod (voltage value)\n");
      printf("[2] : Samping in Timer Trigger Mode(Digit)\n");
      printf("************************************************** \n");

      BOOL fExit = FALSE;
      int ReadMode = 0;
      while (!fExit)          // Infinite loop
      {
```

```
            if (kbhit())         // Push any keys
            {
               int ch = getch();
               switch (ch)
               {
                  case 27:       // Esc key
                     fExit = TRUE;
                     // End of Timer Trigger Mode
                     AxaiHwStopMultiChannel(-1);
                     printf("End of Timer Trigger Mode \n");
                     break;
                  default:
                     break;
               }
            }
            Sleep(100);
         } // END of while
      }
      else
         printf ("AxaInfoIsAIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
   }
   else
      printf ("AxaInfoIsAIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
   printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

// Exit library.
if (AxlClose())
   printf("\n Library has been exitted.\n");
else
   printf("\n Library has not been exitted properly.\n");
}
```

3) A/D conversion in Immediate mode

Immediate mode is used for a case that a certain number of data are taken in as input from the moment A/D conversion started. It is a method to read the data that have been A/D converted after storing them in FIFO in hardware.

AxaiHwSetMultiAccess: Set the value to use Immdiate mode for several input channels that user designated. WordSize is the number of data for each channel. SIO-AI16HB has internal hardware FIFO of which word is less than 1024. Thus, when user uses only one channel, 1024 data can be stored, When user uses several channels, 1024/ (the number of channels) can be the maximum data that can be set. If the user sets a number bigger than this, the data to be stored are automatically limited. After running the API, open *lpWord Size to check the automatically limited value. Before using the API, Timer Trigger Mode must be set by using AxaiSetTriggerMode API.

```
// Set the value to use Immdiagte mode for the designated input channel.
long AdcChannelNo[4] = {0,1,2,3};
long WordSize[4] = {100,100,100,100};
AxaiHwSetMultiAccess(4, AdcChannelNo, WordSize);
```

AxaiHwStartMultiAccess: AxaiHwStartMultiAccess API produces the number of data internally set for each channel. If this API is executed, the value after A/D conversion is automatically stored in the array. *dpBuffer[] stores the data value after A/D conversion. It is an array that stores 2 dimensional array pointer. Firstly, determine the channel for use and the number of data for A/D conversion of each channel. For example, if you use 4 channels and the number of data for A/D conversion of each channel is 10, declare double dBuffer[4][10] and double *pdBuffer[4]. The pointer of the declared dBuffer is allotted to pdBuffer. Do as follows to receive 100 data for channel 0~3 each.

```
// Return the value after the A/D conversion for the designated number.
double pdCBuffer[4][100];
double *pdBuffer[4];
for(int i=0; i<4; i++)
  pdBuffer[i] = pdCBuffer[i];
AxaiHwStartMultiAccess(pdBuffer);
```

```
// Ex4_AXA_ImmediateMode.cpp : Defines the entry point for the console application.
// Initialize library, check input/output information, and exit.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
  // Initialize library. 7 means IRQ. IRQ is automatically set in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library has been initialized.\n");

     // Check the existence of AIO module
     DWORD dwStatus;
     Code = AxaInfoIsAIOModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           // Set the range of input voltage of several channels as -10V~10V.
           long AdcChannelNo[4] = {0,1,2,3};
           AxaiSetMultiRange(4,AdcChannelNo,-10,10);
```

```
            // Set the trigger mode of module 0 as Timer Trigger.
            AxaiSetTriggerMode(0,TIMER_MODE);

            // Set the sampling frequency of a specific module as 10Hz.
            AxaiHwSetSampleFreq(0,100);

            // Set the value to use Immdiate mode forr the designated input channel.
            long lWordSize[4] = {100,100,100,100};
            AxaiHwSetMultiAccess(4, AdcChannelNo, lWordSize);

            // An array to store the value after A/D conversion for the designated number.
            double pdCBuffer[4][100];
            double *pdBuffer[4];
            for(int i=0; i<4; i++)
               pdBuffer[i] = pdCBuffer[i];

            printf("\n Inputting data.\n");
            AxaiHwStartMultiAccess(pdBuffer);

            // Print out data
            for (i = 0; i < 100; i++)
               printf("T=%d : Ch0 [%3f], Ch1 [%3f], Ch2 [%3f], Ch3 [%3f]\n", i,
                  pdBuffer[0][i], pdBuffer[1][i], pdBuffer[2][i], pdBuffer[3][i]);
         }
         else
            printf ("AxaInfoIsAIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
      }
      else
         printf ("AxaInfoIsAIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("Library has been exitted.\n");
   else
      printf("Library has not been exited properly.\n");
}
```

Event/Interrupt

When executing continuous signal check for all input channels(exclusive of immediate collecting API in Axaihw API group) AXA provides APIs to set an event for each case that if the allotted buffer for input channel is filled, empty, over upper limit, or below lower limit. To use an event, set Event Mask for the relevant channel in the first place and determine by which method the event is going to be processed. As AXA supports call back API method, window message method, and event method as methods to process event, choose one of them that fit the development environment of the user. AxaiEventSetChannelEnable API is used to determine whether or not to use event. Each input channel can receive event, but the recommended and efficient method is as followS: Although several input channels can collect data at the same time, give one representative channel within the module and if the event for the channel is received, process other channels.

In addition, the moment that the copy from hardware module's FIFO to memory buffer can be set by using AxaiInterruptSetModuleMask API. The moment that internal interrupt happens can be set by selecting FIFO_HALF_FULL, for high speed sampling, or SCAN_END, for low speed data collecting sampling below hundreds Hz.

SIO-AI4RB module does not support AxaiInterruptSetModuleMask API.

Setting Event Mask

Event mask is a value to set in which situation event will be created. The values in the list below will create the corresponding events. There are 4 types of event mask as below.

Define	Value	Type of Event Mask
DATA_EMPTY	0x01	Event occurred when no data is in buffer.
DATA_MANY	0x02	Event occurred when data in bufffer is bigger than upper set value.
DATA_SMALL	0x04	Event occurred when data in buffer is smaller than lower set value.
DATA_FULL	0x08	Event occurred when buffer is full.

AxaiEventSetChannelMask: AxaiEventSetChannelMask API is used to set event mask for a specific input channel. If you want to set to occur an event when the allotted buffer for input channel 0 is over the set upper limit or below the set lower limit.

```
// Set to occur an event when the value for input channel 0 is over the upper limit or below
// the lower limit.
AxaiEventSetChannelMask (0, DATA_MANY | DATA_SMALL);
```

AxaiEventGetChannelMask: AxaiEventGetChannelMask API is used to check the event mask set for a specific input channel. If you want to check the event type set for input channel 0, do as follows.

```
// Check the event type of input channel 0.
DWORD dwMask;
AxaiEventGetChannelMask (0, &dwMask);
```

AxaiEventSetMultiChannelMask: To set a specific event mask for several channels at the same time, AxaiEventSetMultiChannelMask API is used. Input number of channels used for lSize and input array value of channel numbers used for lAdcChannelNo. For instance, long lAdcChannelNo[4] = {0, 2, 3, 4} means that channels 0, 2, 3, and 4 are set. If you want to set that the buffers allotted for channels 0~3 are over the set upper limit or below the set lower limit, do as follows.

```
// Set to occur an event when the buffers for channels 0~3 are over the set upper limit or
// below the set lower limit.
long lSize = 4;
long lAdcChannelNo[4] = {0, 1, 2, 3};
AxaiEventSetMultiChannelMask(4, lAdcChannelNo, DATA_MANY | DATA_SMALL);
```

Setting Use of Event

Use of event is a default set as enable. Therefore, if event mask is set and the upper and lower value of the necessary buffer are set as the setting of the mask, an event is created as set in event mask from the moment that AxaiHwStartMultiChannel/AxaiHwStartMultiFilter API runs and continuous signal check is done. Set as disable if you do not want to use it. When it is set as enable again, the buffer resets and data collecting restarts from that moment on.

AxaiEventSetChannelEnable: AxaiEventSet ChannelEnable is an API that sets the use of event. If you want to set the event for channel 0 as enable, do as follows.

```
// Set the event for channel 0 as enable.
AxaiEventSetChannelEnable (0,ENABLE);
```

Setting the Process Method of Event

As methods to process events, AXA provides 3 methods: call back API method, window message method, and event method. Call back API method has an advantage that the fastest event receiving can be done by calling up call back API right at the moment event occurs, but also has an disadvantage to have a risk of data loss when a work that imposes a hugh system load like routine processing data within call back API. Thus, for SIO-AI16HB, the time delay because of the abovementioned system load is bigger than 512 times as sampling period, and data loss can be made by the traffic of the data transfer from FIFO inside module to buffer. For calculation, if sampling frequency is 100Khz, data loss can be made for the case whose system load is any bigger than 5msec. Therefore, for a work that imposes load on the system, window message method or event method is recommended as they make this part work independently. In high speed sampling, as window que dispatch routine of window message method is a single thread, application operation can be influenced by message stacking when message is not properly processed. However, as event method uses a specific thread that checks on the occurrence of event and processes the data collected within the thread, it is highly recommended as MultiProcessor system makes uses of resource in the most efficient way.

Event process method can be set by AxaiEventSetChannel as the way below.

● Call back API method

Set as below to use call back API method.

```
// Set the event process method of channel 0 as call back API method.
AxaiEventSetChannel(0,NULL,NULL,OnAIOEventCallback,NULL);
```

OnAIOEventCallback is the pointer of call back API that will process event. Call back API is a global API and can be set as below. If an event occurrs, call back API to process the event is automacially called up.

```
// Call back API to process event
void OnAIOEventCallback(long lChannelNo, DWORD dwStatus)
{
  // lChannelNo : Channel number of event occurrence (0, 1, 2, 3)
  // dwStatus : Event occurrence Flag
  long lpDataLength;
  DWORD dwDigit;

  switch(dwStatus)
  {
     case AIO_EVENT_DATA_UPPER:
        // For the case that it is over upper limt value
        printf("\nEVENT : Data of input channel %d are over the upper limit
           value.\n",lChannelNo);
     break;

     case AIO_EVENT_DATA_LOWER:
        // For the case that it is below lower value
        printf("\nEVENT : Data of input channel %d are below the lower
           limit.\n",lChannelNo);
     break;

     case AIO_EVENT_DATA_FULL:
        // For the case that it is filled with data
        printf("\nEVENT : Input channel %d is filled with data.\n",lChannelNo);
     break;

     case AIO_EVENT_DATA_EMPTY:
        // For the case that it is empty of data
        printf("\nEVENT : Input channel %d is empty of data.\n",lChannelNo);
     break;
  }
}
```

● Windows message method

Set as below to use window message method. C programming does not provide window message methods so use VC++ to explain this part. Input window handle and window message to use window message method. If you do not want to use window message method or want to default value, input 0.

```
// Set the event process method of channel 0 as window message method.
AxaiEventSetChannel(CHANNEL_NO,this->m_hWnd,WM_AIO_EVENT,NULL,NULL);
```

The overview of the method is seen below. OnAIOEventMessage API processes event.

```
#define WM_AIO_EVENT 1011    // Define message to bring event message
#define CHANNEL_NO 0         // Input channel number

…(omitted)…

// Connect message and API
BEGIN_MESSAGE_MAP(CExDIOEventDlg, CDialog)
   //{{AFX_MSG_MAP(CExDIOEventDlg)

ON_MESSAGE(WM_AIO_EVENT,OnAIOEventMessage)

   //}}AFX_MSG_MAP
END_MESSAGE_MAP()

…(omitted)…

void CExAIOEventDlg::OnBtnStartEventMessage()
{
   CString Message;

   // Setting event process method. An example of window message method
   DWORD Code = AxaiEventSetChannel(CHANNEL_NO,this->m_hWnd,WM_AIO_EVENT,NULL,NULL);
   if (Code == AXT_RT_SUCCESS)
      Message.Format("Set call back API method as event process method for module %d ...
         OK",CHANNEL_NO);
   else
      Message.Format("AxdSetlModuleEvent() : ERROR   code 0x%d",Code);

   m_LIST_Message.InsertItem(CHANNEL_NO,Message);

   // Set the event type to be used for a specific channel.
   AxaiEventSetChannelMask(CHANNEL_NO,DATA_EMPTY|DATA_MANY|DATA_SMALL|DATA_FULL);

   // Set use of event for a specific channel.
   AxaiEventSetChannelEnable(CHANNEL_NO,ENABLE);

   // A/D conversion begins in TimerTriggerMode.
   StartAnalogInput();
}

void CExAIOEventDlg::OnAIOEventMessage(long lChannelNo, DWORD dwStatus)
{
   CString IntMessage;
   long lpDataLength;
```

```
    DWORD dwDigit;

    switch(dwStatus)
    {
       case AIO_EVENT_DATA_UPPER:
          // For the case that it is over upper limit value
          IntMessage.Format("EVENT : Data of input channel %d are over the upper limit
             value.",lChannelNo);
          m_LIST_Message.InsertItem(0,IntMessage);
       break;
       case AIO_EVENT _DATA_LOWER:
          // For the case that it is below the lower limit value
          IntMessage.Format("EVENT : Data of input channel %d are below the lower limit
             value.",lChannelNo);
          m_LIST_Message.InsertItem(0,IntMessage);
       break;
       case AIO_EVENT_DATA_FULL:
          // For the case that it is full of data
          g_bSaveBuffer = FALSE;
          IntMessage.Format("EVENT : Input channel %d is filled with data.",lChannelNo);
          m_LIST_Message.InsertItem(0,IntMessage);

          // Exit A/D conversion of Timer Trigger.
          AxaiHwStopMultiChannel();
          IntMessage.Format("EVENT : Exit A/D conversion of input channel %d.",lChannelNo);
          m_LIST_Message.InsertItem(0,IntMessage);

          // Read data stored in buffer.
          g_bReadBuffer = TRUE;
       break;
       case AIO_EVENT_DATA_EMPTY:
          // For the case that it is empty of data
          g_bReadBuffer = FALSE;
          IntMessage.Format("EVENT : Inputh channel %d is empty of data.",lChannelNo);
          m_LIST_Message.InsertItem(0,IntMessage);
       break;
    }
}
```

● Event method

Set as below to use event method. Event method makes use of a specific thread that checks on the occurrence of event. As it processes the collected data within the thread, it is highly recommended for MultiProcessor system as the most efficient method. This part is also explained in VC++. To use event method, HANDLE type member variable, m_hEvent, should be declared first.

```
// Set event process method of channel 0 as event method.
AxaiEventSetChannel (0,NULL,NULL,NULL,&m_hEvent);
```

The overview of how to use is as below. ThreadProc API processes event. Keep in mind the fact that AxaiEventSetChannel API should be called up before executing thread.

Thread recognizes the event when the set event occurs. Then, thread calls up AxdReadEvent API to check the number of the channel that event occurs and event flag information to notify what event occurs in which channel.

```
// Variable needed for using event with event method.
HANDLE m_hThreadHandle;
BOOL m_bThread;
HANDLE m_hEventEvent;
```

```
void CExAIOEventDlg::OnBtnStartEventEvent()
{
   CString Message;

   // Set the process method for event.
   DWORD Code =   AxaiEventSetChannel(0,NULL,NULL,NULL,&m_hEvent); // Event method
   if (Code == AXT_RT_SUCCESS)
      Message.Format("Set the event process method of module %d as event method ... OK",
         CHANNEL_NO);
   else
      Message.Format("AxdSetlModuleEvent() : ERROR   code 0x%d",Code);

   m_LIST_Message.InsertItem(CHANNEL_NO,Message);

   // To use event method, declare HANDLE pEvent member variable and initialize it
   // and create thread.
   m_bThread = TRUE;
   CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)ThreadProc, this, NULL, NULL);

   // Set the event type of specific channel.
   AxaiEventSetChannelMask(CHANNEL_NO,DATA_EMPTY | DATA_MANY | DATA_SMALL | DATA_FULL);

   // Set use of event of a specific channel.
   AxaiEventSetChannelEnable(CHANNEL_NO,ENABLE);

   // Start A/D conversion in TimerTriggerMode.
   StartAnalogInput();
}

void ThreadProc(LPVOID lpData)
{
   CExAIOEventDlg *pDlg = (CExAIOEventDlg *)lpData;

   while(pDlg->m_bThread)
   {
      if (WaitForSingleObject(pDlg->m_hEventEvent, INFINITE) == WAIT_OBJECT_0)
```

```
    {
        // lChannelNo : Input channel number of event occurence ( 0, 1, 2, 3)
        // dwStatus : Event occurence Flag
        CString IntMessage;
        long lpDataLength;
        DWORD dwDigit;

        long lChannelNo;
        DWORD dwStatus;
        DWORD Code = AxaiEventRead(&lChannelNo, &dwStatus);

        switch(dwStatus)
        {
            case AIO_EVENT_DATA_UPPER:       // When data is over the upper limit
                IntMessage.Format("EVENT : EVENT : Data of input channel %d are over the
                    upper limit.",lChannelNo);
                pDlg->m_LIST_Message.InsertItem(0,IntMessage);
            break;
            case AIO_EVENT_DATA_LOWER:       // When data is below the lower limit
                IntMessage.Format("EVENT : Data of input channel %d are below the lower
                    limit.", ChannelNo);
                pDlg->m_LIST_Message.InsertItem(0,IntMessage);
            break;
            case AIO_EVENT_DATA_FULL:        // When input channel is filled with data
                g_bSaveBuffer = FALSE;
                IntMessage.Format("EVENT : Input channel %d is filled with data.",
                    lChannelNo);
                pDlg->m_LIST_Message.InsertItem(0,IntMessage);

                // Exit A/D conversion of Timer Trigger.
                AxaiHwStopMultiChannel();
                IntMessage.Format("EVENT : A/D conversion of input channel %d is shut
                    down.",lChannelNo);
                pDlg->m_LIST_Message.InsertItem(0,IntMessage);

                // Read the data stored in buffer.
                g_bReadBuffer = TRUE;
            break;
            case AIO_EVENT_DATA_EMPTY:       // When input channel is empty of data
                g_bReadBuffer = FALSE;
                IntMessage.Format("EVENT : Input channel %d is empty of data.", lChannelNo);
                pDlg->m_LIST_Message.InsertItem(0,IntMessage);
            break;
        }
    }
  }
}
```

```
// Ex5_AXA_Event.cpp :
// An example of call back method.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"
#include "time.h"

void __stdcall CallBack_ReadChannelData(long lActiveNo, DWORD uFlag)
{
  // Call back API that receives event when the upper limit of the designated buffer
  // is reached
  long AdcChannelNo[4] = {0,1,2,3};
  if(uFlag == AIO_EVENT_DATA_UPPER) // For an event that upper limit value is reached
  {
     GraphData graphData;
     double dVolt[4][2048];
     long lDataLength = 0;
     long lCount = 2048;
     int i;
     for(i = 0; i<4; i++)
     {
        DWORD dwReturn = AxaiHwReadDataLength(AdcChannelNo[i], &lDataLength);
        if(dwReturn!=0)
        {
           return ;
        }
        long lTemp;
        if(lDataLength > 2048)
           lTemp = 2048;
        else
           lTemp = lDataLength;
        if(lCount > lTemp)
           lCount = lTemp;
     }
     for(i = 0; i<4; i++)
     {
        dwReturn = AxaiHwReadSampleVoltage(AdcChannelNo[i], &lCount, dVolt[i]);
        if(dwReturn!=0)
        {
           return ;
        }
     }
     /* Process the collected data for each channel as much as lCount.
        When performing a work that the time delay would be more than 512 times as the
sampling period, window message method or event process method, not call back method must be
used. */
  }
```

```
}

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is automatically set in PCI.
  DWORD Code = AxlOpen(7);
  if (Code == AXT_RT_SUCCESS)
  {
     printf("Library has been initialized.\n");

     // Check the existence of AIO module.
     DWORD dwStatus;
     Code = AxaInfoIsAIOModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           long AdcChannelNo[4] = {0,1,2,3};

           // If is already receiving input.
           AxaiHwStopMultiChannel(-1);
           AxaiInterruptSetModuleMask(0, FIFO_HALF_FULL);
           AxaiEventSetChannel(AdcChannelNo[0], NULL, NULL, (AXT_EVENT_PROC)
              CallBack_ReadChannelData, NULL);
           //Designate call back API that receives event.
           AxaiEventSetChannelMask(AdcChannelNo[0], DATA_MANY);
           // When the upper limit value is reached.
           AxaiEventSetChannelEnable(AdcChannelNo[0], ENABLE);
           // Set the range of input voltage for several channels as -10V~10V.
           AxaiSetMultiRange(4,AdcChannelNo,-10,10);

           // Set the upper limit and lower limit for buffers of several channels.
           AxaiHwSetMultiLimit(4,AdcChannelNo,0,1000);

           // Set trigger mode of module 0 as Timer Trigger.
           AxaiSetTriggerMode(0,TIMER_MODE);

           // Set the sampling frequency of a specific module as 1000Hz.
           AxaiHwSetSampleFreq(0,1000);

           // When buffers for several input channels are full, set to retain the new data.
           AxaiHwSetMultiBufferOverflowMode(4,AdcChannelNo,NEW_DATA_KEEP);

           // A/D conversion starts by allotting 2000 buffers for all input channels.
           long lBuffSize = 2000;
           AxaiHwStartMultiChannel(4,AdcChannelNo,lBuffSize);
           printf("Start Timer Trigger Mode\n");
```

```
            // Take in analog input by going round infinite loop.
            printf("[INFORMATION]************************************ \n");
            printf("[ESC] : Exit \n");
            printf("************************************************** \n");

            BOOL fExit = FALSE;
            int ReadMode = 0;
            while (!fExit)          // Infinite loop
            {
               if (kbhit())         // Push any keys
               {
                  int ch = getch();
                  switch (ch)
                  {
                     case 27:       // Esc key
                        fExit = TRUE;
                        // Stop of Timer Trigger Mode
                        AxaiHwStopMultiChannel(-1);
                        printf("Stop of Timer Trigger Mode\n");
                        break;
                     default:
                        break;
                  }
               }
               Sleep(100);
            } // end of while
         }
         else
            printf ("AxaInfoIsAIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
      }
      else
         printf ("AxaInfoIsAIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("\nLibrary has been exited.\n");
   else
      printf("\nLibrary has not been exited properly.\n");
}
```

Setting Interrupt Mask

When performing continuous signal check, the moment of copying data from FIFO inside hardware module to memory buffer can be set by AxaiInterruptSetModuleMask API with adjusting the moment that internal

interrupt happens. SIO-AI4RB is not provided.

Types of interrupt mask are as follow. The default setting value is FIFO_HALF_FULL.

Type	Explanation
FIFO_HALF_FULL	When FIFO inside module is more than half full, internal interrupt happens so the data copy to buffer is done. (Applied for high speed sampling.)
SCAN_END	For all set channels, every time ADC conversion takes place, interrupt happens so the data copy to memory buffer is done. (For low speed sampling-It is recommended when trigger signal is less than 100~200Hz.)

```
// Set interrupt maks of module hardware.

Code = AxaiInterruptSetModuleMask(0, SCAN_END);
// Use SCAN_END for low speed sampling in External or Timer Trigger Mode.
```

If interrupt mask is set as SCAN_END for high speed sampling, overhead becomes noticeable as each ADC conversion induces the interrupt. Contrary to this, if interrupt mask is set as FIFO_HALF_FULL for low speed sampling, time delay for data becomes noticeable as data copy is done as long as data reaches the half of FIFO size.

For example, when a sampling is done for a channel at 10 Hz, if FIFO_HALF_FULL is set, the collected data can be read after the time delay of about 51 seconds. Because 512 data must be collected for the copy to memory buffer as the FiFO size of SIO_AI16HB is 1024. However, if SCAN_END is set, 0.1 second which is time for a single sampling is required for the first collected datum to be read in memory buffer. Also, if SCAN_END is set for having a sampling for a channel at 100Khz frequency, at every 10 usec, the interrupt is created for making data copy to cause a huge degree of overhead. For this case, FIFO_HALF_FULL is set, at every 5msec, a copy of 512 data is done so the load from overhead does not occur.

Analog Output

Analog output module is processed in PC. It is a module to make an output of analog data in the form of continuous voltage type based on digital data, Output module sets the range of output voltage and makes a proper voltage according to the resolution of the module concerned.

Setting output module

User sets the output range of the designated output channel. As the accuracy of output voltage varies with the setting, it must be set properly. For example, when SIO-AO4R, the 12 bit analog output module, is used, if the reference voltage is set as 0V-5V, the user can control the system with a unit of 1.22mV=5V/4092 in theory. But if the reference voltage is set as -10V - 10V, the user can control the system with a unit of 4.8mV=20V/4096. Therefore the reference voltage must be determined adequately to control the system accurately.

The minimum output voltage means the lowest voltage among the reference voltages among analog output. That is, user cannot measure any voltage smaller than this. The value that user sets is determined as a value closest to a given reference value, including the lowest reference voltage that hardware provides. As SIO-AO4R provides -10V, -5V, and 0V as the lowest reference voltage, if user inputs -8V, -10V is automatically set.

Maximum output voltage means the highest voltage among reference voltages of analog output. The value that user sets is determined as a value closest to a given reference value, including the highest reference voltages that hardware provides. SIO-AO2HB, 2 channel Output module of single unit board PCI-AI16HR having 16bit resolution, has the fixed output voltage range of -10V~10V. The minimal measuring unit is about 20V / 65536=0.305mV.

Module	Min. Voltage	Max. Voltage	Min. Measurement Unit
SIO-AO4R	-10V	10V	4.88mV
	-5V	5V	2.44mV
	0V	10V	2.44mV
	0V	5V	1.22mV
SIO-AO2HB(PCI-AI16HR)	-10V	10V	0.305mV

AxaoSetRange: To set the reference voltage, use AxaoSetRange API. If you want to set the output voltage range of channel 0 as -8V~8V, do as follow. As 2 output channels for PCI-AI16HR board, SIO-AO2HB, have fixed output voltage range, AXT_RT_AIO_INVALID_USE error is returned when executing the API.

```
// Set the output voltage range of channel 0 as -8V~ 8V.
AxaoSetRange(0,-8,8);
```

AxaoGetRange: When it is set as above, -10V and 10V are automatically set. When voltages that hardware does not provide are set as reference, the value user sets and the actually set value can be different from each other. AxaoGetRange API can be used to check which of two values is actually set. If you want to know the set value for channel 0, do as follow.

```
// Check the output voltage range of output channel 0.
double dVmin;
double dVmax;
AxaoGetRange (0, &dVmin, &dVmax);
```

Analog output

AxaoWriteVoltage: After setting the range of output voltage, user can get an output of a specific voltage of the designated output channel. To get an output of a specific voltage, use AxaoWriteVoltage API. Then, input the voltage user wants and the output of voltage is made. Before a new value is set, the voltage is retained. This output voltage can not run over the voltage range that AxaoGetRange sets. If you want to get an output of 5V from channel 0, do as follows.

```
// Get an output of 5V to channel.
AxaoWriteVoltage (0,5);
```

AxaoReadVoltage: If you want to know the current output value, use AxaoReadVoltage API.

```
// Check the output value of channel 0.
Double dVolt;
AxaoReadVoltage(0, dVolt);
```

If user wants analog output of a specific wave form, use the abovementioned AxaoWriteVoltage API to get it. For example, if you want to get an output of a sin wave, do as follows.

```
// Create a sin wave form which period is 1 and size is 10.
double OutputValue;
double SampleCount = 1000;   // Set the sampling as 1000
double Magnitude = 10;       // Set the size as 10V
while (1)                    // Infinite loop
{
  // Get an output sin wave form which period is 1 and size is 10V.
  for(int i=0;i< SampleCount;i++)
  {
     OutputValue = sin((2*3.141592 / SampleCount)*(double) i ) * Magnitude;
     AxaoWriteVoltage(0,OutputValue);
     Sleep(1);      // 1msec Sleep * SampleCount(1000) = Wave form which period is 1 second
  }
}
/* In the above code, Sleep(1) is not exactly 1 msec in the real system. For an output of
the exact wave form, accurate sleep time must be realized for each system and OS. In window
system, QueryPerformanceFrequency/QueryPerformanceCounter, the Windows function, is used to
realize the relatively accurate delay time by measuring CPU frequency and clock number. */
```

```
// Ex6_AXA_AnalogOut.cpp : Defines the entry point for the console application.
// Set analog output module and send out output.

#include "stdafx.h"
```

```
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is automatically set in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library has been initialized.\n");

      // Check the existence of AIO module
      DWORD dwStatus;
      Code = AxaInfoIsAIOModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            // Initialize output module.
            // Set the output voltage range of channel 0 as -8V~ 8V.
            AxaoSetRange(0,-8,8);
            // Check the set output voltage range.
            // Check the output voltage range of output channel 0.
            double dVmin;
            double dVmax;
            AxaoGetRange(0, &dVmin, &dVmax);

            // Send out analog output by going round infinite loop.
            printf("[INFORMATION]*************** \n");
            printf("Voltage range of output module: %.1f ~ %.1f\n",dVmin,dVmax);
            printf("[ESC] : Exit \n");
            printf("[1] : -10V output\n");
            printf("[2] : -8V output\n");
            printf("[3] : -4V output\n");
            printf("[4] :  0V output\n");
            printf("[5] :  4V output\n");
            printf("[6] :  8V output\n");
            printf("[7] : 10V output\n");
            printf("*************************** \n");

            BOOL fExit = FALSE;
            while (!fExit)          // Infinite loop
            {
               if (kbhit())         // Push any keys
               {
                  int ch = getch();
                  switch (ch)
```

```
                {
                   case 27:       // Esc key
                      fExit = TRUE;
                   break;

                   // Channel 0 outputs the voltage as key
                   case '1':   AxaoWriteVoltage(0,-10);
                   break; //-10V output
                   case '2':   AxaoWriteVoltage(0,-8);
                   break; // -8V output
                   case '3':   AxaoWriteVoltage(0,-4);
                   break; // -4V output
                   case '4':   AxaoWriteVoltage(0,0);
                   break; // 0V output
                   case '5':   AxaoWriteVoltage(0,4);
                   break; // 4V output
                   case '6':   AxaoWriteVoltage(0,8);
                   break; // 8V output
                   case '7':   AxaoWriteVoltage(0,10);
                   break; // 10V output
                }
             }
          }
       }
       else
          printf ("AxaInfoIsAIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
    }
    else
       printf ("AxaInfoIsAIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
    printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

  // Exit library.
  if (AxlClose())
    printf("\nLibrary has been exited.\n");
  else
    printf("\nLibrary has not been exited properly.\n");
}
```

```
// Ex7_AXA_AnalogOutSin.cpp : Defines the entry point for the console application.
// Send out Sin wave form.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"
#include "math.h"
```

```
void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is automatically set in PCI.
   DWORD Code = AxlOpen(7);
   if (Code == AXT_RT_SUCCESS)
   {
      printf("Library has been initialized.\n");
      // Check the existence of AIO module.
      DWORD dwStatus;
      Code = AxaInfoIsAIOModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            // Initialize output module.
            // Set the output voltage range of channel 0 as -10V~10V.
            AxaoSetRange(0,-10,10);

            // Send out analog output going round infinite loop.
            double OutputValue;
            double SampleCount = 1000;    // Set sampling rate as 1000.
            double Magnitude = 10;        // Set the size as 10V.
            while(1)                      // Infinite loop.
            {
               // Output a sin wave form which period is 1 second and size is 10V.
               for(int i=0;i< SampleCount;i++)
               {
                  OutputValue = sin((2*3.141592/ SampleCount)*(double) i ) * Magnitude;
                  AxaoWriteVoltage(0,OutputValue);
                  Sleep(1);               // 1msec Sleep * SampleCount(1000) = 1second
               }
            }
         }
         else
            printf ("AxaInfoIsAIOModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",Code);
      }
      else
         printf ("AxaInfoIsAIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf ("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if (AxlClose())
      printf("\nLibrary has been exited.\n");
   else
      printf("\nLibrary has not been exited properly.\n");
```

```
}
```

AIO Command Manual Information

This manual is to run AIO board/module of Ajinextek in Windows 98, Windows NT, Windows 2000 or Windows XP OS system by Microsoft VC++6.0, Visual Basic, Borland C-Builder, and Delphi. Library APIs included are categorized as API and described.

Header file

C++

AXA.h

Visual Basic

AXA.bas

Delphi

AXA.pas

Terms of API

API names in this manual

API names used in this manual have prefixes to distinguish each action.

Prefix of library API

Prefix	Explanation
Axa	It means that it is only for Axa. All APIs starting with Axa are defined in Axa.h.
AxaInfo	API to check board and module information.
Axai	API related to input.
AxaiInfo	API that searches for input module information.
AxaiEvent	API that sets and checks input channel event.
AxaiInterrupt	API that sets and checks input module interrupt.
AxaiSw	API related to Software Timer (Normal Trigger Mode).
AxaiHw	API related to Hardware Timer(Timer Trigger + External Trigger Mode).
Axao	API related to output.
AxaoInfo	API that searches output module information.
Set	Set the register or variable of a chip.
Get	In a pair with Set, it checks the variable value set by Set API or reads the state of chip register.
Multi	Process a few channels at the same time.
Read	Read the state and value.
Write	Write the value.

Argument names in this manual

Common arguments used in this manual have the following meanings.

long lBoardNo : Automatic arrangement takes place in ascending order from the first initialized base board. Board number starts from 0.

long lModuleNo : When AIO module initialization takes place, automatic arrangement takes place in ascending order from the first board. Module number starts from 0.

long lChannelNo : Channel number can be divided into two groups: input channel number and output channel number.

Input channel number is automatically arranged in ascending order from the first input module of the first board.

Output channel number is automatically arranged in ascending order from the first output module of the first board.

Input and output channel number starts from 0.

The result of allotted module number and channel number for mounting 2 PCI-AI16HR boards

Module No.	Input/Output	Channel Index
0	Input(SIO-AI16HB)	0~15
1	Output(SIO-AO2HB)	0~1
2	Input(SIO-AI16HB)	16~31
3	Output(SIO-AO2HB)	2~3

AIO Command Quick List

API group for trigger mode

Set Trigger Mode (Refer to AxaiSetTriggerMode)

Enum	Value	Explanation
NORMAL_MODE	1	Software trigger method that A/D conversion is done at the moment user wants(general value)
TIMER_MODE	2	Trigger method that A/D conversion is done by using internal clock of H/W
EXTERNAL_MODE	3	Trigger method that A/D conversion is done by using clock of external input terminal

APIs related to Trigger Mode

Trigger Mode	Important Related APIs
Normal Trigger Mode	AxaiSwReadVoltage, AxaiSwReadMultiVoltage
Timer Trigger Mode	(API group: AxaiHw.., AxaiEvent..,AxaiInterrupt..) AxaiEventSetChannel AxaiHwSetSampleFreq, AxaiHwSetSamplePeriod AxaiHwStartMultiChannel , AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetLimit, AxaiHwReadDataLength AxaiHwReadSampleVoltage AxaiHwSetMultiAccess(Immediate Mode), AxaiHwStartMultiAccess(Immediate Mode) Etc..
External Trigger Mode	It is the same for Timer Trriger Mode. It has nothing to do with Immediate Mode API. AxaiHwSetSampleFreq / AxaiHwSetSamplePeriod.

Board and module information

Function	Description
AxaInfoIsAIOModule	Check the existence of AIO module.
AxaInfoGetModuleNo	Check the number of AIO module.
AxaInfoGetModuleCount	Check the counter of AIO module.
AxaInfoGetInputCount	Check the number of input channel of the designated module.
AxaInfoGetOutputCount	Check the number of output channel of the designated module.
AxaInfoGetChannelNoOfModuleNo	Check the first channel number of the designated module.
AxaInfoGetModule	Check base board, module location and module ID through the number of the designated module.

Input module information search

Function	Description
AxaiInfoGetModuleNoOfChannelNo	Check module number through the number of the designated input channel.
AxaiInfoGetChannelCount	Check the number of total channel of analog input module.

Set and check input module interrupt and channel event

Function	Description
AxaiEventSetChannel	Use window message, call back API or event method to give event message to the designated module.
AxaiEventSetChannelEnable	Set use of event for the designated input channel.
AxaiEventGetChannelEnable	Check use of event for the designated input channel.
AxaiEventSetMultiChannelEnable	Set use of event for several designated input channels.
AxaiEventSetChannelMask	Set the type of event for the designated input channel.
AxaiEventGetChannelMask	Check the type of event for the designated input channel.
AxaiEventSetMultiChannelMask	Set the types of event for several designated input channels.
AxaiEventRead	Check the location of event occurrence.
AxaiInterruptSetModuleMask	Set interrupt mask for the designated module.
AxaiInterruptGetModuleMask	Check the interrupt mask set for the designated module.

Set and check input module parameter

Function	Description
AxaiSetRange	Set the range of input voltage for the designated input channel.
AxaiGetRange	Check the range of input voltage for the designated input channel.
AxaiSetMultiRange	Set the range of allowed input voltage for several designated input channels.
AxaiHwSetBufferOverflowMode	Set the management method when the buffer of the designated input channel is full.
AxaiHwGetBufferOverflowMode	Check the management method when the buffer of the designated input channel is full.
AxaiHwSetMultiBufferOverflowMode	Set the management method when the buffers of several designated input channels are full.
AxaiHwSetLimit	Set the upper and lower limit for the buffer of the designated input channel.
AxaiHwGetLimit	Check the upper and lower limit for the buffer of the designated input channel.

AxaiHwSetMultiLimit	Set the upper and lower limits for the buffers of several designated input channels.
AxaiSetTriggerMode	Set trigger mode of the designated input module.
AxaiGetTriggerMode	Check tirgger mode of the designated input module.
AxaiHwSetSampleFreq	Set the sampling interval for the designated module in frequency unit.
AxaiHwGetSampleFreq	Check the sampling interval value for the designated module in frequency unit.
AxaiHwSetSamplePeriod	Set the sampling internval for the designated module in time unit.
AxaiHwGetSamplePeriod	Check the sampling internval value for the designated module in time unit..
AxaiSetModuleOffsetValue	Control the offset value of the designated input module.
AxaiGetModuleOffsetValue	Check the offset value of the designated input module.

A/D conversion of input module

Function	Description
AxaiSwReadVoltage	Return analog input value to the designated input channel as voltage value.
AxaiSwReadDigit	Return analog input value to the designated input channel as digit value.
AxaiSwReadMultiVoltage	Return analog input value to several designated input channels as voltage value.
AxaiSwReadMultiDigit	Return analog input value to several designated input channels as digit value.
AxaiHwStartMultiChannel	Start A/D conversion that uses H/W timer in several designated input channels.
AxaiHwStartMultiFilter	Start A/D conversion in the several designated input channels and return voltage vlause after filtering out the set number.
AxaiHwStopMultiChannel	Stop A/D conversion that uses H/W timer.
AxaiHwSetMultiAccess	Set the value to use Immediate mode in the several designated input channels.
AxaiHwStartMultiAccess	Return the data after A/D conversions of the designated number.
AxaiHwReadSampleVoltage	Read the value after A/D conversion as voltage value by using H/W timer in the designated input channel.
AxaiHwReadSampleDigit	Read the value after A/D conversion as digit value by using H/W timer in the designated input channel.

Checking buffer status of input module

Function	Description
AxaiHwIsBufferEmpty	Check if there is no data in the memory buffer of the designated input channel.
AxaiHwIsBufferUpper	Check if the data in the buffer of the designated input channel is are bigger than the set value.
AxaiHwIsBufferLower	Check if the data in the buffer of the designated input channel are smaller than the set value.

AxaiHwReadDataLength	Check the number of data stored in memory buffer of the designated input channel.

Output module information search

Function	Description
AxaoInfoGetModuleNoOfChannelNo	Check module number as the channel number of the designated output channel.
AxaoInfoGetChannelCount	Check the number of total channel number of the analog output module.

Setting and checking output module

Function	Description
AxaoSetRange	Set the output voltage range of the designanted output channel.
AxaoGetRange	Check the output voltage range of the designanted output channel.
AxaoSetMultiRange	Set the output voltage ranges of tseveral designanted output channels.
AxaoWriteVoltage	It outputs the Input voltage for the designated output channel.
AxaoWriteMultiVoltage	It outputs the Input voltages for several designated output channels.
AxaoReadVoltage	Check the output voltage vlaue for the designated output channel.
AxaoReadMultiVoltage	Check the output voltage vlauesfor several designated output channels.

Define Sentence

Statements which are used in AXA Analog Input/Output Library are in the following list.

Header File : #include "AXHS.h"

FALSE & TRUE

Definition	Value	Explanation
FALSE	00h	False
TRUE	01h	True

AXT_USE

Definition	Value	Explanation
DISABLE	00h	Not used
ENABLE	01h	Used

AXT_EXISTENCE

Definition	Value	Explanation
STATUS_NOTEXIST	00h	Module does not exist
STATUS_EXIST	01h	Module exists

AXT_AIO_TRIGGER_MODE

Definition	Value	Explanation
DISABLE_MODE	00h	Do not use the trigger
NORMAL_MODE	01h	Trigger by user event mode
TIMER_MODE	02h	Sampling time specified by the auto-trigger mode
EXTERNAL_MODE	03h	Trigger mode by external signal

AXT_AIO_FULL_MODE

Definition	Value	Explanation
NEW_DATA_KEEP	00h	Full updates on the data buffer with new data
CURR_DATA_KEEP	01h	Full data buffer maintained the existing data

AXT_AIO_EVENT_MASK

Definition	Value	Explanation
DATA_EMPTY	01h	When the data buffer is emptied

DATA_MANY	02h	When the number of data buffers is more than
DATA_SMALL	04h	When a specified number of data buffers is fewer
DATA_FULL	08h	When the data buffer is full

AXT_AIO_INTERRUPT_MASK

Definition	Value	Explanation
ADC_DONE	00h	When A/D conversion complete
SCAN_END	01h	Both the channel A/D conversion complete
FIFO_HALF_FULL	02h	When the half of H/W data buffer is more than
NO_SIGNAL	03h	No event

AIO Command Function List

Board and Module Information

Function	Description
AxaInfoIsAIOModule	Check the existence of AIO module.
AxaInfoGetModuleNo	Check the number of input channel of the designated module.
AxaInfoGetInputCount	Check the number of output channel of the designated module.
AxaInfoGetOutputCount	Check the number of AIO module.
AxaInfoGetModuleCount	Check the first channel number of the designated module.
AxaInfoGetChannelNoOfModuleNo	Check base board number, module location, and module ID through the desigated module number.
AxaInfoGetModule	Check the existence of AIO module.

AxaInfoIsAIOModule

Purpose

Check the existence of AIO module.

Format

C++

```
DWORD AxaInfoIsAIOModule(DWORD *upStatus);
```

Visual Basic

```
Function AxaInfoIsAIOModule(ByRef upStatus As Long) As Long
```

Delphi

```
function AxaInfoIsAIOModule(upStatus : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]upStatus	Check whether AIO module exist or not : AXT_EXISTENCE – [00h] No AIO module – [01h] AIO module exists

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
* See error code Table for more information on status error codes

Description

Initialize all AIO modules mounted on PC.

Example

C++

```
#include "Axl.h"       // Add library
DWORD dwStatus;

// Check the existence of AIO module.
AxaInfoIsAIOModule(&dwStatus);
if(dwStatus == STATUS_EXIST)
  AfxMessageBox("AIO module exists.");
else
  AfxMessageBox("AIO module does not exist.");
```

Visual Basic

```
' Add AXA.bas, AXL.bas, and AXHS.bas module to use library.
Dim lStatus As Long

' Check the existence of AIO module.
```

```
AxaInfoIsAIOModule lStatus
If lStatus = STATUS_EXIST Then
  MsgBox " AIO module exists.", vbOKCancel
Else
  MsgBox " AIO module does not exist.", vbOKCancel
End If
```

Delphi

```
Uses
Axl, AXHS, AXA   { Add library }
var
  dwStatus : DWord;

begin
  { Check the existence of AIO module. }
  AxaInfoIsAIOModule(@dwStatus);
  if(dwStatus = STATUS_EXIST) then
     Application.MessageBox ('AIO module exists.', 'Ajinextek', MB_OK);
  else
     Application.MessageBox ('AIO module does not exist.', 'Ajinextek', MB_OK);
end;
```

See Also

AxaInfoGetModuleNo, AxaInfoGetInputCount, , AxaInfoGetOutputCount, AxaInfoGetModuleCount, AxaInfoGetChannelNoOfModuleNo, AxaInfoGetModule

AxaInfoGetModuleNo

Purpose

Check the number of AIO module.

Format

C++

```
DWORD AxaInfoGetModuleNo(long lBoardNo, long lModulePos, long
*lpModuleNo);
```

Visual Basic

```
Function AxaInfoGetModuleNo(ByVal lBoardNo As Long, ByVal lModulePos
As Long, ByRef lpModuleNo As Long) As Long
```

Delphi

```
function AxInfoGetModuleNo(lBoardNo: LongInt; lModulePos: LongInt;
lpModuleNo: PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lBoardNo	Board Number
[>>]lModulePos	Module Position
[<<]lpModuleNo	AIO Module Number

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1101] AXT_RT_INVALID_BOARD_NO : Invalid board number
[1102] AXT_RT_INVALID_MODULE_POS : Invalid module position
* See error code Table for more information on status error codes

Description

Check the number of AIO module initialized.

Example

C++

```
// Check the number of AIO module.
Long lpModuleNo;
CString strData;

AxaInfoGetModuleNo (0, 0, &lpModuleNo);
strData.Format("The number of AIO module is %d.", lpModuleNo);
AfxMessageBox(strData);
```

Visual Basic

```
' Check the number of AIO module.
Dim lpModuleNo As Long
Dim  strData As String

AxaInfoGetModuleNo 0 0 lCount
strData    = "The number of AIO module is" + CStr(lpModuleNo) + "."

MsgBox strData
```

Delphi

```
{ Check the number of AIO module. }
var
  lpModuleNo: LongInt;
  strData : String;

begin
  AxaInfoGetModuleNo (0, 0, @ lpModuleNo);
  strData := 'The number of AIO module is' + IntToStr(lpModuleNo) + '.';

  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaInfoIsAIOModule, AxaInfoGetModuleCount, AxaInfoGetInputCount, AxaInfoGetOutputCount, AxaInfoGetChannelNoOfModuleNo, AxaInfoGetModule

AxaInfoGetModuleCount

Purpose

Check the Counter of AIO module.

Format

C++

```
DWORD AxaInfoGetModuleCount(long *lpModuleCount);
```

Visual Basic

```
Function AxaInfoGetModuleCount(ByRef lpModuleCount As Long) As Long
```

Delphi

```
function AxaInfoGetModuleCount(lpModuleCount : PLongInt) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]lpModuleCount	The number of AIO module

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2051] AXT_RT_AIO_NOT_MODULE : No AIO module
* See error code Table for more information on status error codes

Description

Check the number of AIO module initialized.

Example

C++

```
// Check the number of AIO module.
long lCount;
CString strData;

AxaInfoGetModuleCount(&lCount);
strData.Format("The number of AIO module is %d. ", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the number of AIO module.
Dim lCount As Long
Dim  strData As String

AxaInfoGetModuleCount lCount
```

```
strData   = "The number of AIO module is" + CStr(lCount) + "."

MsgBox strData
```

Delphi

```
{ Check the number of AIO module. }
var
   lCount : LongInt;
   strData : String;

begin
   AxaInfoGetModuleCount(@lCount);
   strData  := 'The number of AIO module is' + IntToStr(lCount) + '.';

   Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaInfoIsAIOModule, AxaInfoGetModuleNo, AxaInfoGetInputCount, AxaInfoGetOutputCount, AxaInfoGetChannelNoOfModuleNo, AxaInfoGetModule

AxaInfoGetInputCount

Purpose

Check the number of input channel of the designated module.

Format

C++

```
DWORD AxaInfoGetInputCount(long lModuleNo, long *lpCount);
```

Visual Basic

```
Function AxaInfoGetInputCount(ByVal lModuleNo As Long, ByRef lpCount
As Long) As Long
```

Delphi

```
function AxaInfoGetInputCount(lModuleNo : LongInt; lpCount :
PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No. (0 ~ N-1)
[<<]lpCount	The number of input channel

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_INVALID_MODULE_NO : Invalid AIO module
* See error code Table for more information on status error codes

Description

Check the number of input channel of the module designated by user.

SIO-AI16HB(input module among single unit board PCI-AI16HRs) has 16 input channels.

SIO-AI4RB has 4 input channels.

Example

C++

```
// Check the number of input channel of the 0th module.
long lCount;
CString   strData;

AxaInfoGetInputCount(0, &lCount);
strData.Format("The number of input channel of module 0 is %d . ", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the number of input channel of the 0th module.
Dim lCount As Long
Dim strData As String

AxaInfoGetInputCount 0, lCount
strData   = "The number of input channel of module 0 is" + CStr(lCount) + "."

MsgBox strData
```

Delphi

```
{ Check the number of input channel of the 0th module. }
var
  lCount : LongInt;
  strData : String;

begin
  AxaInfoGetInputCount(0, @lCount);
  strData := 'The number of input channel of module 0 is' + IntToStr(lCount) + '.';

  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaInfoIsAIOModule, AxaInfoGetModuleNo, AxaInfoGetModuleCount, AxaInfoGetOutputCount, AxaInfoGetChannelNoOfModuleNo, AxaInfoGetModule

AxaInfoGetOutputCount

Purpose

Check the number of output channel of the designated module.

Format

C++

```
DWORD AxaInfoGetOutputCount(long lModuleNo, long *lpCount);
```

Visual Basic

```
Function AxaInfoGetOutputCount(ByVal lModuleNo As Long, ByRef lpCount
As Long) As Long
```

Delphi

```
function AxaInfoGetOutputCount(lModuleNo : LongInt; lpCount :
PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No. (0 ~ N-1)
[<<]lpCount	The number of output channel

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_INVALID_MODULE_NO : Invalid AIO module
* See error code Table for more information on status error codes

Description

Check the number of output channel designated by user.

SIO-AI16HB(output module among single unit board PCI-AI16HRs) has 2 output channels.

SIO-AO4RB has 4 output channels.

Example

C++

```
// Check the number of output channel of the 0th module.
long lCount;
CString strData;

AxaInfoGetOutputCount(0, &lCount);
strData.Format("The number of the 0th module is %d . ", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the number of output channel of the 0th module.
Dim lCount As Long
Dim strData As String

AxaInfoGetOutputCount 0, lCount
strData = "The number of the 0th module is" + CStr(lCount) + "."

MsgBox strData
```

Delphi

```
{ Check the number of output channel of the 0th module. }
var
  lCount : LongInt;
  strData : String;

begin
  AxaInfoGetOutputCount(0, @lCount);
  strData  := 'The number of the 0th module is' + IntToStr(lCount) + '.';

  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaInfoIsAIOModule, AxaInfoGetModuleNo, AxaInfoGetModuleCount, AxaInfoGetInputCount, AxaInfoGetChannelNoOfModuleNo, AxaInfoGetModule

AxaInfoGetChannelNoOfModuleNo

Purpose

Check the first channel number of the designated module.

Format

C++

```
DWORD AxaInfoGetChannelNoOfModuleNo(long lModuleNo, long
*lpChannelNo);
```

Visual Basic

```
Function AxaInfoGetChannelNoOfModuleNo(ByVal lModuleNo As Long, ByRef
lpChannelNo As Long) As Long
```

Delphi

```
function AxaInfoGetChannelNoOfModuleNo(lModuleNo : LongInt;
lpChannelNo : PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Moduel No. (0 ~ N-1)
[<<]lpChannelNo	First channel No. of the relevant module

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_INVALID_MODULE_NO : Invalid AIO module
* See error code Table for more information on status error codes

Description

Check the first channel number of the module designated by user.

Module number can be obtained from both Analog Input and Analog Output.

The result of allotted module number and channel number for mounting 2 PCI-AI16HR boards.

Module No	Input/Output	Channel Index
0	Input (SIO-AI16HB)	0~15
1	Output (SIO-AO2HB)	0~1
2	Input (SIO-AI16HB)	16~31
3	Output (SIO-AO2HB)	2~3

Example

C++

```
// Check the first channel number of the 0th module.
long lChannelNo;
CString strData;

AxaInfoGetChannelNoOfModuleNo(0, &lChannelNo);
strData.Format("The first channel number of the 0th module is %d.", lChannelNo);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the first channel number of the 0th module.
Dim lChannelNo As Long
Dim strData As String

AxaInfoGetChannelNoOfModuleNo 0, lChannelNo
strData = "The first channel number of the 0th module is" + CStr(lChannelNo) + "."

MsgBox strData
```

Delphi

```
{ Check the first channel number of the 0th module. }
var
  lChannelNo : LongInt;
  strData : String;

begin
  AxaInfoGetChannelNoOfModuleNo(0, @lChannelNo);
  strData  := 'The first channel number of the 0th module is'  + IntToStr(lChannelNo) +
     '.';

  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaInfoIsAIOModule, AxaInfoGetModuleNo, AxaInfoGetModuleCount, AxaInfoGetInputCount, AxaInfoGetOutputCount, AxaInfoGetModule

AxaInfoGetModule

Purpose

Check base board number, module location, and module ID through the designated module number.

Format

C++

```
DWORD AxaInfoGetModule(long lModuleNo, long *lpBoardNo, long
*lpModulePos, DWORD *upModuleID);
```

Visual Basic

```
Function AxaInfoGetModule(ByVal lModuleNo As Long, ByRef lpBoardNo As
Long, ByRef lpModulePos As Long, ByRef upModuleID As Long) As Long
```

Delphi

```
function AxaInfoGetModule(lModuleNo : LongInt; lpBoardNo : PLongInt;
lpModulePos : PLongInt; upModuleID : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No. (0 ~ N-1)
[<<]lpBoardNo	Base board No. (0 ~ N-1)
[<<]lpModulePos	Module location (0 ~ 3)
[<<]upModuleID	Module ID

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_INVALID_MODULE_NO : Invalid AIO module
* See error code Table for more information on status error codes

Description

Check base board number, module location, and module ID through the module number that user designated.

The ID of AIO module is as follows.

Module ID	Value	Module
AXT_SIO_AI4RB	0xA1 (161)	SIO-AI4RB Input 4ch
AXT_SIO_AO4RB	0xA2 (162)	SIO-AO4RB Output 4ch
AXT_SIO_AI16HB	0xA5 (165)	Input 16ch module of single unit board PCI-AI16HR
AXT_SIO_AO2HB	0xA6 (166)	Output 2ch module of single unit board PCI-AI16HR

Example

C++

```
// Check base board, module number, and module ID of the 0th module.
long lBoardNo;
long lModulePos;
DWORD dwModuleID;
CString strData;

AxaInfoGetModule(0, &lBoardNo, &lModulePos, &dwModuleID);
strData.Format("BoardNo=%d, ModulePos=%d, dwModuleID=%02Xh", lBoardNo, lModulePos,
  dwModuleID);
AfxMessageBox(strData);
```

Visual Basic

```
' Check base board, module number, and module ID of the 0th module.
Dim lBoardNo As Long
Dim lModulePos As Long
Dim dwModuleID As DWord
Dim strData As String

AxaInfoGetModule 0, lBoardNo, lModulePos, dwModuleID
strData = "BoardNo=" + CStr(lBoardNo) + ", ModulePos=" + CStr(lModulePos) + ",
  ModuleID=" + CStr(dwModuleID)
MsgBox strData
```

Delphi

```
{ Check base board, module number, and module ID of the 0th module. }
var
  BoardNo : LongInt;
  lModulePos : LongInt;
  dwModuleID : DWord;
  strData : String;;

begin
  AxaInfoGetModule(0, @lBoardNo, @lModulePos, @dwModuleID);
  strData := 'BoardNo=' + IntToStr(lBoardNo) + ', ModulePos=' + IntToStr(lModulePos) +
     ', ModuleID=' + IntToStr(dwModuleID);
  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaInfoIsAIOModule, AxaInfoGetModuleNo, AxaInfoGetModuleCount, AxaInfoGetInputCount, AxaInfoGetOutputCount, AxaInfoGetChannelNoOfModuleNo

Input Module Information Search

Function	Description
AxaiInfoGetModuleNoOfChannelNo	Check the module number through the designated input channel number.
AxaiInfoGetChannelCount	Check the nubmer of total channel of analog input module.

AxaiInfoGetModuleNoOfChannelNo

Purpose

Check module number through the designated input channel number.

Format

C++

```
DWORD AxaiInfoGetModuleNoOfChannelNo(long lAdcChannelNo, long
*lpModuleNo);
```

Visual Basic

```
Function AxaiInfoGetModuleNoOfChannelNo(ByVal lAdcChannelNo As Long,
ByRef lpModuleNo As Long) As Long
```

Delphi

```
function AxaiInfoGetModuleNoOfChannelNo(lAdcChannelNo : LongInt;
lpModuleNo : PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel No. (0 ~ N-1)
[<<]lpModuleNo	Module No

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Check module number through the input channel number that user designated.

Channels having different module numbers can have distinct data collection methods by setting different hardware using setting APIs that have module numbers.

Example

C++

```
// Check module number of the 0th input channel.
long lModuleNo;
CString strData;

AxaiInfoGetModuleNoOfChannelNo(0, &lModuleNo);
strData.Format("The module number of the 0th input channel is %d .", lModuleNo);

AfxMessageBox(strData);
```

Visual Basic

```
' Check module number of the 0th input channel.
Dim lModuleNo As Long
Dim strData As String

AxaiInfoGetModuleNoOfChannelNo 0, lModuleNo
strData = "The module number of the 0th input channel is" + CStr(lCount) + "."

MsgBox strData
```

Delphi

```
{ Check module number of the 0th input channel. }
var
  lModuleNo : LongInt;
  strData : String;

begin
  AxaiInfoGetModuleNoOfChannelNo(0, @lModuleNo);
  strData  := 'The module number of the 0th input channel is' + IntToStr(lModuleNo) +
     '.';

  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiInfoGetChannelCount

AxaiInfoGetChannelCount

Purpose

Check the number of total channel of Analog input module.

Format

C++

```
DWORD AxaiInfoGetChannelCount(long *lpAdcChannelCount);
```

Visual Basic

```
Function AxaiInfoGetChannelCount(ByRef lpAdcChannelCount As Long) As
Long
```

Delphi

```
function AxaiInfoGetChannelCount(lpAdcChannelCount : PLongInt) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]lpAdcChannelCount	The number of total channel of input module

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
* See error code Table for more information on status error codes

Description

Check the number of total channel of the analog input modules registered in system.

Example

C++

```
// Check the number of total channel of analog input module.
long lCount;
CString strData;

AxaiInfoGetChannelCount(&lCount);
strData.Format("The number of total channel of analog input module is %d.", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the number of total channel of analog input module.
Dim lCount As Long
Dim strData As String

AxaiInfoGetChannelCount lCount
```

```
strData = "The number of total channel of analog input module is" + CStr(lCount) + "."

MsgBox strData
```

Delphi

```
{ Check the number of total channel of analog input module. }
var
   lCount : LongInt;
   strData : String;

begin
   AxaiInfoGetChannelCount(@lCount);
   strData  := 'The number of total channel of analog input module is' +
      IntToStr(lCount) +'.';

   Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiInfoGetModuleNoOfChannelNo

Input Module Interrupt/Set and Check Channel Event

Function	Description
AxaiEventSetChannel	Use window message, call back API or event method to bring event message to designated module.
AxaiEventSetChannelEnable	Set the use of event of the designated input channel.
AxaiEventGetChannelEnable	Set the use of event of the designated input channel.
AxaiEventSetMultiChannelEnable	Set the use of several designated input channels.
AxaiEventSetChannelMask	Set event type of the designated input channel.
AxaiEventGetChannelMask	Check event type of the designated input channel.
AxaiEventSetMultiChannelMask	Set event type of several designated input channels.
AxaiEventRead	Check the location of event occurrence.
AxaiInterruptSetModuleMask	Set interrupt mask of the designated module.
AxaiInterruptGetModuleMask	Check the set interrupt mask of the designated module.

AxaiEventSetChannel

Purpose

Use window message, call back API, or event method to bring event message to the designated channel. Use during continuous data collecting operation (refer to AxaiHwStartMultiChannel) using H/W Timer (Timer Trigger Mode, External Trigger Mode).

Format

C++

```
DWORD AxaiEventSetChannel(long lAdcChannelNo, HWND hWnd, DWORD
uMesssage, AXT_EVENT_PROC pProc, HANDLE *pEvent);
```

Visual Basic

```
Function AxaiEventSetChannel(ByVal lAdcChannelNo As Long, ByVal hWnd
As Long, ByVal uMesssage As Long, ByVal pProc As Long, ByRef pEvent
As Long) As Long
```

Delphi

```
function AxaiEventSetChannel(lAdcChannelNo : LongInt; hWnd : HWND;
uMesssage : DWord; pProc : AXT_EVENT_PROC; pEvent : PDWord) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel No.(0 ~ N-1)
[>>]hWnd	Window handle
[>>]uMesssage	Window message
[>>]pProc	Call back API
[<<]pEvent	Event handle

Defined call back API typedef void (__stdcall *AXT_EVENT_PROC)(long lActiveNo, DWORD uFlag);

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Event can be taken by the unit of channel.

Turn window handle to receive event message, and turn API pointer when using call back API. Turn event handle pointer when using event method.

Create event handle to be used in library and return. If signals are collected from several channels, event can be notified to each channel. However, setting event in one representative channel in module is recommendable since it also handles other channels when event is notified to that channel.

Refer to the table below.

Factor	Explanation
lAdcChannelNo	Input channel number to receive event.
hwnd	Window handle. Used when receiving window message. NULL if not in use.
uMessage	Window handle message. NULL if not in use.
Proc	Pointer of API to be called when event occurs. NULL if not in use.
pEvent	Event ID. Use in the case of event method. NULL if not in use.

Call back API has an advantage that event is notified the fastest since call back API is called immediately when event occurs. However, if there is a work that has system load like routine that handles data there is a risk of data loss. In other words, SIO-AI16HB(PCI-AI16HR input module of single board) for example, data loss occurs when time delay caused by system load exceeds 512 times of sampling cycle, accumulating data movement from FIFO in module to buffer. In numerical value, if sampling cycle is 100Khz, data loss can occur when system load is over 5msec. Therefore if there is there is load during handling data, using window message method or event method is recommendable as these operate separately. In high speed sampling, window message method is a Dispatch routine of window message queue and a single thread, so it affects on Application movement backlogging message if not handling in a short time adequately. However, event method is recommended especially in Multi Processor System and etc. to use resources most efficiently since it uses specific thread that observes event occurence and handles data collected in that thread.

In case of message method and call back method, first factor value becomes the channel that event occurs, and the second factor value is the flag of occurred event. When using event method, use AxaiEventRead to know the channel and flag of the occurred event.

Event flags that show when event occurs are as follows

Enum	Value	Explanation
AIO_EVENT_DATA_UPPER	1	Event occurs when data in buffer is over upper limit
AIO_EVENT_DATA_LOWER	2	Event occurs when data in buffer is below lower limit
AIO_EVENT_DATA_FULL	3	Event occurs when data in buffer is full
AIO_EVENT_DATA_EMPTY	4	Event occurs when there is no data in buffer

Example

C++

```
// Use window message to bring event message to 0th input channel
BEGIN_MESSAGE_MAP(CDIODlg, CDialog)
   //{{AFX_MSG_MAP
   // Declaration
   ON_MESSAGE(WM_AXL_EVENT, OnAdcDataEvent)
//}}AFX_MSG_MAP
END_MESSAGE_MAP()
```

```
// Call 'OnAdcDataEvent()' when event occurs
AxaiEventSetChannel(0, m_hWnd, WM_AXL_EVENT, NULL, NULL);

// When using event method
HANDLE g_hEvent;
AxaiEventSetChannel(0, NULL, NULL, NULL, &g_hEvent);
// Create handle in library
```

Visual Basic

```
' Use window message to bring event message to 0th input channel
' Declaration
Private Sub AxtMsg1_OnMessage1(ByVal wParam As Long, ByVal lParam As Long)
' Event message managing statement
End Sub

' Call 'AxtMsg1_OnMessage1()' when event occurs
AxtMsg1.Message1 = WM_AXL_EVENT

AxaiEventSetChannel 0, Me.hWnd, WM_AXL_EVENT, 0, 0
```

Delphi

```
{ Use window message to bring event message to 0th input channel }
procedure OnEventMessage(var Msg : TMessage); message WM_AXL_EVENT;

{ Declaration }
procedure TForm1.OnEventMessage(var Msg : TMessage);
begin
{ Event message managing statement }
end;

{ Call 'OnEventMessage()' when event occurs }
AxaiEventSetChannel(0, Form1.Handle, WM_AXL_EVENT, nil, nil);
```

See Also

AxaiEventSetChannelEnable, AxaiEventGetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventSetChannelMask, AxaiEventGetChannelMask, AxaiEventSetMultiChannelMask, AxaiEventRead, AxaiInterruptSetModuleMask, AxaiInterruptGetModuleMask

AxaiEventSetChannelEnable

Purpose

Set the use of event for designated input channel.

Format

C++

```
DWORD AxaiEventSetChannelEnable(long lAdcChannelNo, DWORD uUse);
```

Visual Basic

```
Function AxaiEventSetChannelEnable(ByVal lAdcChannelNo As Long, ByVal
uUse As Long) As Long
```

Delphi

```
function AxaiEventSetChannelEnable(lAdcChannelNo : LongInt; uUse :
DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel No. (0 ~ N-1)
[>>]uUse	Set whether to use event : AXT_USE - [00h] Release event use - [01h] Set event use

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the use of event for the input channel that user designates.

Example

C++

```
// Set to use event in 0th input channel.
AxaiEventSetChannelEnable(0, ENABLE);
```

Visual Basic

```
' Set to use event in 0th input channel.
AxaiEventSetChannelEnable 0, ENABLE
```

Delphi

```
{ Set to use event in 0th input channel. }
AxaiEventSetChannelEnable(0, ENABLE);
```

See Also

AxaiEventSetChannel, AxaiEventGetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventSetChannelMask, AxaiEventGetChannelMask, AxaiEventSetMultiChannelMask, AxaiEventRead, AxaiInterruptSetModuleMask, AxaiInterruptGetModuleMask

AxaiEventGetChannelEnable

Purpose

Verify the use of event of the designated input channel.

Format

C++

```
DWORD AxaiEventGetChannelEnable(long lAdcChannelNo, DWORD *upUse);
```

Visual Basic

```
Function AxaiEventGetChannelEnable(ByVal lAdcChannelNo As Long, ByRef
upUse As Long) As Long
```

Delphi

```
function AxaiEventGetChannelEnable(lAdcChannelNo : LongInt; upUse :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel No. (0 ~ N-1)
[<<]upUse	Set whether to use event : AXT_USE - [00h] Release event use - [01h] Set event use

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Verify the use of event of the channel designated by user.

Example

C++

```
// Verify the use of event of input channel 0.
DWORD dwUse;
AxaiEventGetChannelEnable(0, &dwUse);
if (dwUse)
{
  AfxMessageBox("Channel 0 can use event.");
}
else
{
  AfxMessageBox("Channel 0 cannot use event.");
}
```

Visual Basic

```
' Verify the use of event of input channel 0.
Dim lUse As Long
AxaiEventGetChannelEnable 0, lUse

If lUse = ENABLE Then
  MsgBox "Channel 0 can use event."
Else
  MsgBox "Channel 0 cannot use event."
End If
```

Delphi

```
{ Verify the use of event of input channel 0. }
var
  dwUse : DWord;

begin
  AxaiEventGetChannelEnable(0, @dwUse);
  if (dwUse = ENABLE) then
     Application.MessageBox ('Channel 0 can use event.', 'Ajinextek', MB_OK);
  else
     Application.MessageBox ('Channel 0 cannot use event.', 'Ajinextek', MB_OK);
end;
```

See Also

AxaiEventSetChannel, AxaiEventSetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventSetChannelMask, AxaiEventGetChannelMask, AxaiEventSetMultiChannelMask, AxaiEventRead, AxaiInterruptSetModuleMask, AxaiInterruptGetModuleMask

AxaiEventSetMultiChannelEnable

Purpose

Set the use of event for several designated input channels.

Format

C++

```
DWORD AxaiEventSetMultiChannelEnable(long lSize, long *lpAdcChannelNo,
DWORD uUse);
```

Visual Basic

```
Function AxaiEventSetMultiChannelEnable(ByVal lSize As Long, ByRef
lpAdcChannelNo As Long, ByVal uUse As Long) As Long
```

Delphi

```
function AxaiEventSetMultiChannelEnable(lSize : LongInt;
lpAdcChannelNo : PLongInt; uUse : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channel for use
[>>]lpAdcChannelNo	Arrary of channel numbers for use
[>>]uUse	Set whether to use event : AXT_USE – [00h] Release event use – [01h] Set event use

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the use of event for several input channels designated by user.

Input the number of channel for use for lSize.

*lpAdcChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

Example

C++

```
// Set so that several input channels can use event.
long lAdcChannelNo[4] = {0, 1, 2, 3};
```

```
AxaiEventSetMultiChannelEnable(4, lAdcChannelNo, ENABLE);
```

Visual Basic

```
' Set so that several input channels can use event.
Dim lAdcChannelNo(0 To 3) As Long

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiEventSetMultiChannelEnable 4, lAdcChannelNo(0), ENABLE
```

Delphi

```
{ Set so that several input channels can use event. }
var
  lAdcChannelNo : array [0..3] of LongInt;

begin
  lAdcChannelNo[0] := 0;
  lAdcChannelNo[1] := 1;
  lAdcChannelNo[2] := 2;
  lAdcChannelNo[3] := 3;

  AxaiEventSetMultiChannelEnable(4, @lAdcChannelNo, ENABLE);
end;
```

See Also

AxaiEventSetChannel, AxaiEventSetChannelEnable, AxaiEventGetChannelEnable, AxaiEventSetChannelMask, AxaiEventGetChannelMask, AxaiEventSetMultiChannelMask, AxaiEventRead, AxaiInterruptSetModuleMask, AxaiInterruptGetModuleMask

AxaiEventSetChannelMask

Purpose

Set event type for the designated input channel.

Format

C++

```
DWORD AxaiEventSetChannelMask(long lAdcChannelNo, DWORD uMask);
```

Visual Basic

```
Function AxaiEventSetChannelMask(ByVal lAdcChannelNo As Long, ByVal
uMask As Long) As Long
```

Delphi

```
function AxaiEventSetChannelMask(lAdcChannelNo : LongInt; uMask :
DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel No
[>>]uMask	Event type : AXT_AIO_EVENT_MASK – [01h] Event occurs when there is no data in buffer – [02h] Event occurs when the data in buffer is over the upper limit – [04h] Event occurs when the data in buffer is below the lower limit – [08h] Event occurs when the buffer if full of data

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set event type for the input channel designated by user.

The event set here is only received. The event types are for the following situations: when there is no data in buffer, when the data in buffer is over the set upper limit, when the data in buffer is below the lower limit, and when the buffer is full of data.

To know the event types, refer to the table of bit values.

Example

C++

```
// Set the event type to create an event when the data is over the upper limit for
// input channel 0 or below the lower limit of input channel 0.
AxaiEventSetChannelMask (0, DATA_MANY | DATA_SMALL);
```

Visual Basic

```
' Set the event type to create an event when the data is over the upper limit for
' input channel 0 or below the lower limit of input channel 0.
AxaiEventSetChannelMask 0, 6
```

Delphi

```
{ Set the event type to create an event when the data is over the upper limit for input
channel 0 or below the lower limit of input channel 0. }
AxaiEventSetChannelMask (0, 6);
```

See Also

AxaiEventSetChannel, AxaiEventSetChannelEnable, AxaiEventGetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventGetChannelMask, AxaiEventSetMultiChannelMask, AxaiEventRead, AxaiInterruptSetModuleMask, AxaiInterruptGetModuleMask

AxaiEventGetChannelMask

Purpose

Verify the event type for the designated input channel.

Format

C++

```
DWORD AxaiEventGetChannelMask(long lAdcChannelNo, DWORD *upMask);
```

Visual Basic

```
Function AxaiEventGetChannelMask(ByVal lAdcChannelNo As Long, ByRef
upMask As Long) As Long
```

Delphi

```
function AxaiEventGetChannelMask(lAdcChannelNo : LongInt; upMask :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel No
[<<]upMask	Event type : AXT_AIO_EVENT_MASK – [01h] Event occurs when there is no data in buffer – [02h] Event occurs when the data in buffer is over the upper limit – [04h] Event occurs when the data in buffer is below the lower limit – [08h] Event occurs when the buffer if full of data

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Verify the event type for the input channel designated by user.

Example

C++

```
// Verify the event type for input channel 0.
DWORD dwMask;

AxaiEventGetChannelMask(0, &dwMask);
```

Visual Basic

```
' Verify the event type for input channel 0.
Dim lMask As Long
```

```
AxaiEventGetChannelMask 0, dwMask
```

Delphi

```
{ Verify the event type for input channel 0. }
var
  dwMask : DWord;

begin
  AxaiEventGetChannelMask(0, @dwMask);
end;
```

See Also

AxaiEventSetChannel, AxaiEventSetChannelEnable, AxaiEventGetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventSetChannelMask, AxaiEventSetMultiChannelMask, AxaiEventRead, AxaiInterruptSetModuleMask, AxaiInterruptGetModuleMask

AxaiEventSetMultiChannelMask

Purpose

Set the event type for several designated input channels.

Format

C++

```
DWORD AxaiEventSetMultiChannelMask(long lSize, long *lpAdcChannelNo,
DWORD uMask);
```

Visual Basic

```
Function AxaiEventSetMultiChannelMask(ByVal lSize As Long, ByRef
lpAdcChannelNo As Long, ByVal uMask As Long) As Long
```

Delphi

```
function AxaiEventSetMultiChannelMask(lSize : LongInt;
lpAdcChannelNo : PLongInt; uMask : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channel for use
[>>]lpAdcChannelNo	Array of channel numbers for use
[>>]uMask	Event type : AXT_AIO_EVENT_MASK – [01h] Event occurs when there is no data in buffer – [02h] Event occurs when the data in buffer is over the upper limit – [04h] Event occurs when the data in buffer is below the lower limit – [08h] Event occurs when the buffer if full of data

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set event types for several input channels designated by user.

The event set here is only received. The event types are as follow: when there is no data in buffer, when the data in buffer is over the set upper limit, when the data in buffer is below the lower limit, and when the buffer is full of data.

Input the number of channel for use for lSize.

*lpAdcChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For ex0xample, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

Example

C++

```
// Set to create events for several input channels when the data is over the upper
// limit or below the lower limit.
long lAdcChannelNo[4] = {0, 1, 2, 3};

AxaiEventSetMultiChannelMask(4, lAdcChannelNo, DATA_MANY | DATA_SMALL);
```

Visual Basic

```
' Set to create events for several input channels when the data is over the upper
' limit or below the lower limit.
Dim lAdcChannelNo(0 To 3) As Long

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiEventSetMultiChannelMask 4, lAdcChannelNo(0), 6
```

Delphi

```
{ Set to create events for several input channels when the data is over the upper limit
or below the lower limit. }
var
  lAdcChannelNo : array [0..3] of LongInt;

begin
  lAdcChannelNo[0] := 0;
  lAdcChannelNo[1] := 1;
  lAdcChannelNo[2] := 2;
  lAdcChannelNo[3] := 3;

  AxaiEventSetMultiChannelMask(4, @lAdcChannelNo, 6);
end;
```

See Also

AxaiEventSetChannel, AxaiEventSetChannelEnable, AxaiEventGetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventSetChannelMask, AxaiEventGetChannelMask, AxaiEventRead, AxaiInterruptSetModuleMask, AxaiInterruptGetModuleMask

AxaiEventRead

Purpose

Verify the location that event occurs.

Format

C++

```
DWORD AxaiEventRead(long *lpAdcChannelNo, DWORD *upMode);
```

Visual Basic

```
Function AxaiEventRead(ByRef lpAdcChannelNo As Long, ByRef upMode As
Long) As Long
```

Delphi

```
function AxaiEventRead(lpAdcChannelNo : PLongInt; upMode : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]lpAdcChannelNo	Input channel No
[<<]upMode	Event flag : AXT_AIO_EVENT_MASK – [01h] Event occurs when the data in buffer is over the upper limit – [02h] Event occurs when the data in buffer is below the lower limit – [03h] Event occurs when buffer is full of data – [04h] Event occurs when buffer is empty of data

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2052] AXT_RT_AIO_NOT_EVENT : Failed to read AIO event
* See error code Table for more information on status error codes

Description

For the case that uses event as event method, Verify the location that event occurs.

Refer to the table below for event flag.

Example

C++

```
// Verify the location on the 0th channel that event occurs.
long lAdcChannelNo;
DWORD dwMode;

AxaiEventRead(&lAdcChannelNo, &dwMode);
```

Visual Basic

```
' Verify the location on the 0th channel that event occurs.
Dim lAdcChannelNo As long
Dim lMode As long

AxaiEventRead lAdcChannelNo, lMode
```

Delphi

```
{ Verify the location on the 0th channel that event occurs. }
var
  lAdcChannelNo : LongInt;
  dwMode : DWord;

begin
  AxaiEventRead(@lAdcChannelNo, @dwMode);
end;
```

See Also

AxaiEventSetChannel, AxaiEventSetChannelEnable, AxaiEventGetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventSetChannelMask, AxaiEventGetChannelMask, AxaiEventSetMultiChannelMask, AxaiInterruptSetModuleMask, AxaiInterruptGetModuleMask

AxaiInterruptSetModuleMask

Purpose

Set the interrupt mask of the designated module.

Format

C++

```
DWORD AxaiInterruptSetModuleMask(long lModuleNo, DWORD uMask);
```

Visual Basic

```
Function AxaiInterruptSetModuleMask(ByVal lModuleNo As Long, ByVal
uMask As Long) As Long
```

Delphi

```
function AxaiInterruptSetModuleMask(lModuleNo : LongInt; uMask :
DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module No
[>>]uMask	[02h] Interrupt maks to be designated (1 ~ 2)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

This API is used to set the moment that data move from hardware FIFO to the buffer that uses set its size through internal interrupt for performing continuous signal check. (It does not support SIO-AI4RB).

The types of interrupt mask are as follows. The basic setting value is FIFO_HALF_FULL.

Type	Contents
SCAN_END(1)	Interrupt is generated every time ADC conversion is completed on any of configured channel, and the data will be copied to memory buffer. (Recommended for low frequency sampling rate – for low frequencey trigger signal of 100~200Hz or less)
FIFO_HALF_FULL(2)	Internal interrupt is generated when the FIFO level is over half full, and the data will be copied to the memory buffer. (Applied for high frequency sampling rate)

If the interrupt mask is set as SCAN_END for high speed sampling, overhead becomes noticeable as each ADC conversion induces the interrupt. Contrary to this, if the interrupt mask is set as FIFO_HALF_FULL for low speed sampling, the time delay for data becomes noticeable as the data copy is done as long as the data reach the half of FIFO size.

For example, when a sampling is done for a channel at 10 Hz, if FIFO_HALF_FULL is set, the collected data can be read after the time delay of about 51 seconds because 512 data must be collected for the copy to memory buffer as the FiFO size of SIO_AI16HB is 1024. However, if SCAN_END is set, 0.1 second which is time for a sinle sampling is required for the first collected datum to be read in memory buffer. Also, if SCAN_END is set for having a sampling for a channel at 100Khz frequency, at every 10 usec, the interrupt is created for making data coty to cuause a huge degree of overhead. For this case, FIFO_HALF_FULL is set, at every 5msec, a copy of 512 data is done so the load from overhead does not occur.

Example

C++

```
// Determine hardware interrupt maks of the 0th module.

AxaiSetTriggerMode(0, TIMER_MODE);
AxaiHwSetSampleFreq(0, 10);                 // 10Hz low speed sampling
AxaiInterruptSetModuleMask(0, SCAN_END);
// For low speed sampling when performing continuous signal check in either external or
// Timer Trigger Mode Use SCAN_END.
```

Visual Basic

```
' Determine hardware interrupt maks of the 0th module.

AxaiInterruptSetModuleMask 0, FIFO_HALF_FULL
```

Delphi

```
{ Determine hardware interrupt maks of the 0th module. }
begin
  AxaiInterruptSetModuleMask(0, FIFO_HALF_FULL);
end;
```

See Also

AxaiEventSetChannel, AxaiEventSetChannelEnable, AxaiEventGetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventSetChannelMask, AxaiEventGetChannelMask, AxaiEventSetMultiChannelMask, AxaiEventRead, AxaiInterruptGetModuleMask

AxaiInterruptGetModuleMask

Purpose

Verify interrupt mask of the designated module.

Format

C++

```
DWORD AxaiInterruptGetModuleMask(long lModuleNo, DWORD *upMask);
```

Visual Basic

```
Function AxaiInterruptGetModuleMask(ByVal lModuleNo As Long, ByRef
upMask As Long) As Long
```

Delphi

```
function AxaiInterruptGetModuleMask(lModuleNo : LongInt; upMask :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module No
[<<]upMask	[02h] Interrupt mask to be checked (1~2)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

This API checks hardware interrupt mask designated in module.

(It does not support SIO-AI4RB.)

Types of mask are as follow. Basic set value is FIFO_HALF_FULL

Type	Contents
SCAN_END(1)	Interrupt is generated every time ADC conversion is completed on any of configured channel, and the data will be copied to memory buffer. (Recommended for low frequency sampling rate – for low frequencey trigger signal of 100~200Hz or less.)
FIFO_HALF_FULL(2)	Internal interrupt is generated when the FIFO level is over half full, and the data will be copied to the memory buffer. (Applied for high frequency sampling rate)

Example

C++

```
// Verify the hardware interrupt mask of the 0th module.
DWORD upMask;
AxaiInterruptGetModuleMask(0, &upMask);
```

Visual Basic

```
' Verify the hardware interrupt mask of the 0th module.
Dim lMode As long
AxaiInterruptGetModuleMask 0, lMode
```

Delphi

```
{ Verify the hardware interrupt mask of the 0th module. }
var
  dwMode : DWord;

begin
  AxaiInterruptSetModuleMask(0, @dwMode);
end;
```

See Also

AxaiEventSetChannel, AxaiEventSetChannelEnable, AxaiEventGetChannelEnable, AxaiEventSetMultiChannelEnable, AxaiEventSetChannelMask, AxaiEventGetChannelMask, AxaiEventSetMultiChannelMask, AxaiEventRead, AxaiInterruptSetModuleMask

Setting and Checking Input Module Parameter

Function	Description
AxaiSetRange	Setting the range of input voltage for the designated input channel.
AxaiGetRange	Checking the range of input voltage for the designated input channel.
AxaiSetMultiRange	Setting the range of allowed input voltage for several designated input channels.
AxaiHwSetBufferOverflowMode	Setting the management method when the buffer in the designated input channel is full.
AxaiHwGetBufferOverflowMode	Checking the management method when the buffer in the designated input channel is full.
AxaiHwSetMultiBufferOverflowMode	Setting the management method when the buffers in several designated input channels are full.
AxaiHwSetLimit	Setting the upper and lower limit of the buffer of the designated input channel.
AxaiHwGetLimit	Checking the upper and lower limit of the buffer of the designated input channel.
AxaiHwSetMultiLimit	Setting the upper and lower limit of the buffers of several designated input channels.
AxaiSetTriggerMode	Setting the trigger mode of the designated input module.
AxaiGetTriggerMode	Checking the trigger mode of the designated input module.
AxaiHwSetSampleFreq	Setting the sampling interval of the designated module in the unit of frequency.
AxaiHwGetSampleFreq	Checking the sampling interval of the designated module in the unit of frequency.
AxaiHwSetSamplePeriod	Setting the sampling interval of the designated module in the unit of time.
AxaiHwGetSamplePeriod	Checking the sampling interval of the designated module in the unit of time.
AxaiSetModuleOffsetValue	Setting the Offset value of the designated input module.
AxaiGetModuleOffsetValue	Checking the Offset value of the designated input module.

AxaiSetRange

Purpose

Set the range of input voltage for the designated input channel.

Format

C++

```
DWORD AxaiSetRange(long lAdcChannelNo, double dMinVolt, double
dMaxVolt);
```

Visual Basic

```
Function AxaiSetRange(ByVal lAdcChannelNo As Long, ByVal dMinVolt As
Double, ByVal dMaxVolt As Double) As Long
```

Delphi

```
function AxaiSetRange(lAdcChannelNo : LongInt; dMinVolt : Double;
dMaxVolt : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel No
[>>]dMinVolt	Minimum input voltage
[>>]dMaxVolt	Maximum input voltage

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the range of allowed input voltage of the input channel designated by user.

Setting the right reference voltage has an impact on performance so be careful about it. Each module has its own allowed input voltage. If user inputs other voltage, the input value automatically resets to another value to which the input value is the closest. The automatically reset value can be checked by AxaiGetRange. The minimum input voltage means the lowest voltage among the allowed voltages of the analog input module. That is, the user cannot input any voltage below the minimum voltage. And the value that the user originally input is set as the closet value to one including the minimum allowed voltage that hardware supports. As SIO-AI4RB provides the minimum reference voltages as -10V, -5V, and 0V, if user inputs -8V, it is automatically reset as -10V.

The maximum voltage means the highest voltage among the allowed analog input voltages. In other words, user cannot set any voltage higher than this. The value the user inputs automatically resets to the closest value within the allowed maximum voltage that hardware provides. SIO-AI4RB provides 10V and 5V as the maximum allowed voltage. If user inputs 8V, it is automatically reset as 10V. On the

contrary, SIO-AI16HB whose resolution is 16 bit has fixed range of input voltage, AXT_RT_AIO_INVALID_USE error returns when executing the API.

Setting range of input voltage

Module	Min. Voltage	Max. Voltage	Min Measurement Unit
SIO-AI4RB 12bit Resolution	-10V	10V	4.88mV
	-5V	5V	2.44mV
	0V	5V	1.22mV
	0V	10V	2.44mV
SIO-AI16HB(PCI-AI16HR) Fixed input voltage range 16bit Resolution	-10V	10V	0.305mV

Example

C++

```
// Set the range of input voltage for input channel 0.
AxaiSetRange(0, -10, 10);
```

Visual Basic

```
' Set the range of input voltage for input channel 0.
AxaiSetRange 0, -10, 10
```

Delphi

```
{ Set the range of input voltage for input channel 0. }
AxaiSetRange(0, -10, 10);
```

See Also

AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit,AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod,AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiGetRange

Purpose

Verify the range of input voltage for the designated input channel.

Format

C++

```
DWORD AxaiGetRange(long lAdcChannelNo, double *dpMinVolt, double
*dpMaxVolt);
```

Visual Basic

```
Function AxaiGetRange(ByVal lAdcChannelNo As Long, ByRef dpMinVolt As
Double, ByRef dpMaxVolt As Double) As Long
```

Delphi

```
function AxaiGetRange(lAdcChannelNo : LongInt; dpMinVolt : PDouble;
dpMaxVolt : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel No. (0 ~ N-1)
[<<]dpMinVolt	Min. input voltage
[<<]dpMaxVolt	Max. input voltage

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the range of the allowed input voltage for the input channel designated by user.

It is used when user checks the actual set value after he sets the allowed input voltage using AxaiSetRange API. Therefore, if the user sets a value that H/W does not provide, the actual set value can be different from the initially input value.

Setting range of input voltage

Module	Min. Voltage	Max. Voltage	Min Measurement Unit
SIO-AI4RB 12bit Resolution	-10V	10V	4.88mV
	-5V	5V	2.44mV
	0V	5V	1.22mV
	0V	10V	2.44mV

SIO-AI16HB(PCI-AI16HR) Fixed input voltage range 16bit Resolution	-10V	10V	0.305mV

Example

C++

```
// Set the range of input voltage for input channel 0.
double  dMinVolt;
double  dMaxVolt;

AxaiGetRange(0, &dMinVolt, &dMaxVolt);
```

Visual Basic

```
' Set the range of input voltage for input channel 0.
Dim lMask As Long

AxaiGetRange 0, dMinVolt, dMaxVolt
```

Delphi

```
{ Set the range of input voltage for input channel 0. }
var
 dMinVolt : Double;
 dMaxVolt : Double;
 strData : String;

begin
  AxaiGetRange(0, @dMinVolt, @dMaxVolt);

  strData := 'MinVolt : ' + FloatToStr(dMinVolt) + ', MaxVolt : ' +
     loatToStr(dMaxVolt);
 Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiSetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiSetMultiRange

Purpose

Set the range of allowed input voltage for several designated input channels.

Format

C++

```
DWORD AxaiSetMultiRange(long lSize, long *lpAdcChannelNo, double
dMinVolt, double dMaxVolt);
```

Visual Basic

```
Function AxaiSetMultiRange(ByVal lSize As Long, ByRef lpAdcChannelNo
As Long, ByVal dMinVolt As Double, ByVal dMaxVolt As Double) As Long
```

Delphi

```
function AxaiSetMultiRange(lSize : LongInt; lpAdcChannelNo : PLongInt;
dMinVolt : Double; dMaxVolt : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channel for use
[>>]lpAdcChannelNo	Array of numbers of input channel for use
[>>]dMinVolt	Min. input voltage
[>>]dMaxVolt	Max. input voltage

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the range of the allowed input voltage for several input channels designated by user.

It is to set the ranges of the allowed input voltage for several analog input channels at the same time.

Setting the right reference voltage has an impact on performance so be careful about it. Each module has the allowed input voltage. If user inputs other voltages, the input value is automatically reset to another value to which the input value is the closest. The automatically reset value can be checked by AxaiGetRange. The minimum input voltage means the lowest voltage among the allowed voltages of the analog input module. That is, the user cannot input any voltage below the minimum voltage. And the value that the user originally input is set as the closet value to one including the minimum allowed voltage that hardware supports. As SIO-AI4R provides the minimum reference voltages as -10V, -5V, and 0V, if user inputs -8V, it is automatically reset as -10V.

The maximum voltage means the highest voltage among the allowed analog input voltages. That is, user cannot set any voltage higher than this. The value the user inputs is automatically reset to the

closest value within the allowed maximum voltage that hardware provides. SIO-AI4Rprovides 10V and 5V as the maximum allowed voltage. If user inputs 8V, it is automatically reset as 10V. On the contrary, SIO-AI16HB whose resolution is 16 bit has fixed range of input voltage, AXT_RT_AIO_INVALID_USE error returns when executing the API.

Setting range of input voltage

Module	Min. Voltage	Max. Voltage	Min Measurement Unit
SIO-AI4RB 12bit Resolution	-10V	10V	4.88mV
	-5V	5V	2.44mV
	0V	5V	1.22mV
	0V	10V	2.44mV
SIO-AI16HB(PCI-AI16HR) Fix the range of input voltage 16bit Resolution	-10V	10V	0.305mV

Input the number of channel for use for lSize.

*lpAdcChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

Example

C++

```
// Set the ranges of input voltages for several input channels.
long lAdcChannelNo[4] = {0, 1, 2, 3};

AxaiSetMultiRange(4, lAdcChannelNo, -10, 10);
```

Visual Basic

```
' Set the ranges of input voltages for several input channels.
Dim lAdcChannelNo(0 To 3) As Long

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiSetMultiRange 4, lAdcChannelNo(0), -10, 10
```

Delphi

```
{ Set the ranges of input voltages for several input channels. }
var
   lAdcChannelNo : array [0..3] of LongInt;

begin
   lAdcChannelNo[0] := 0;
   lAdcChannelNo[1] := 1;
```

```
  lAdcChannelNo[2] := 2;
  lAdcChannelNo[3] := 3;

  AxaiSetMultiRange(4, @lAdcChannelNo, -10, 10);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwSetBufferOverflowMode

Purpose

Set the management method when the buffer in the designated input channel is full.

Format

C++

```
DWORD AxaiHwSetBufferOverflowMode(long lAdcChannelNo, DWORD
uFullMode);
```

Visual Basic

```
Function AxaiHwSetBufferOverflowMode(ByVal lAdcChannelNo As Long,
ByVal uFullMode As Long) As Long
```

Delphi

```
function AxaiHwSetBufferOverflowMode(lAdcChannelNo : LongInt;
uFullMode : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number (0 ~ N-1)
[>>]uFullMode	Data buffer mode : AXT_AIO_FULL_MODE - [00h] Keep the new data - [01h] Keep the current data

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the management method when the buffer of input channel designated by user is full of data.

It is used when A/D conversion takes place using the internal timer of H/W.

When the allotted memory buffer is full of data, if new data come in, either way that the current data are removed to store the new data or the new data are not taken in should be deciced by using Timer Trigger.

In uFulMode, when the buffer allotted for each channel is full, input 'CURR_ DATA_KEEP' to discard the new data to keep the current data or input 'NEW_DATA_ KEEP' to remove the current data to store the new data.

Example

C++

```
// Set the management method so that it keeps the new data when the buffer for the 0th
// input channel is full of data.
AxaiHwSetBufferOverflowMode(0, NEW_DATA_KEEP);
```

Visual Basic

```
' Set the management method so that it keeps the new data when the buffer for the 0th
' input channel is full of data.
AxaiHwSetBufferOverflowMode 0, NEW_DATA_KEEP
```

Delphi

```
{ Set the management method so that it keeps the new data when the buffer for the 0th
input channel is full of data. }
AxaiHwSetBufferOverflowMode(0, NEW_DATA_KEEP);
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwGetBufferOverflowMode

Purpose

Verify the management method when the buffer in the designated input channel is full.

Format

C++

```
DWORD AxaiHwGetBufferOverflowMode(long lAdcChannelNo, DWORD
*upFullMode);
```

Visual Basic

```
Function AxaiHwGetBufferOverflowMode(ByVal lAdcChannelNo As Long,
ByRef upFullMode As Long) As Long
```

Delphi

```
function AxaiHwGetBufferOverflowMode(lAdcChannelNo : LongInt;
upFullMode : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number (0 ~ N-1)
[<<]upFullMode	Data buffer mode : AXT_AIO_FULL_MODE - [00h] Keep the new data - [01h] Keep the current data

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Verify the management method when the buffer of input channel designated by user is full of data.

The return value of 'CURR_DATA_KEEP' means to deny the new data and keep the current data. On the contrary, the return value of 'NEW_ DATA_KEEP' means to remove the current data and store the new data.

Example

C++

```
// Verify the management method when the buffer of input channel 0 is full of data.
DWORD dwFullMode;

AxaiHwGetBufferOverflowMode(0, &dwFullMode);
```

Visual Basic

```
' Verify the management method when the buffer of input channel 0 is full of data.
Dim lFullMode As Long

AxaiHwGetBufferOverflowMode 0, dwFullMode
```

Delphi

```
{ Verify the management method when the buffer of input channel 0 is full of data. }
var
  dwFullMode : DWord;

begin
  AxaiHwGetBufferOverflowMode(0, @dwFullMode);
  Application.MessageBox (PCHAR(IntToStr(dwFullMode)), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod,AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwSetMultiBufferOverflowMode

Purpose

Set the management method when the buffers in several designated input channels are full.

Format

C++

```
DWORD AxaiHwSetMultiBufferOverflowMode(long lSize, long
*lpAdcChannelNo, DWORD uFullMode);
```

Visual Basic

```
Function AxaiHwSetMultiBufferOverflowMode(ByVal lSize As Long, ByRef
lpAdcChannelNo As Long, ByVal uFullMode As Long) As Long
```

Delphi

```
function AxaiHwSetMultiBufferOverflowMode(lSize : LongInt;
lpAdcChannelNo : PLongInt; uFullMode : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of inpt channel for use
[>>]lpAdcChannelNo	Array of number of channels for use
[>>]uFullMode	Data buffer mode : AXT_AIO_FULL_MODE – [00h] Keep the new data – [01h] Keep the current data

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the management method when the buffers of several input channels designated by user are full of data.

It is used when A/D conversion takes place using the internal timer of H/W.

When the allotted memory buffer is full of data, if new data come in, either way that the current data are removed to store the new data or the new data are not taken in should be deciced by using Timer Trigger.

In uFulMode, when the buffer allotted for each channel is full, input 'CURR_ DATA_KEEP' to discard the new data to keep the current data or input 'NEW_DATA_ KEEP' to remove the current data to store the new data.

For lSize, input the number of channels for use.

*lpAdcChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

Example

C++

```
// Set the management method to keep the new data when the buffers of several input
// channels are full of data.
long lAdcChannelNo[4] = {0, 1, 2, 3};

AxaiHwSetMultiBufferOverflowMode(4, lAdcChannelNo, NEW_DATA_KEEP);
```

Visual Basic

```
' Set the management method to keep the new data when the buffers of several input
' channels are full of data.
Dim lAdcChannelNo(0 To 3) As Long

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiHwSetMultiBufferOverflowMode 4, lAdcChannelNo(0), NEW_DATA_KEEP
```

Delphi

```
{ Set the management method to keep the new data when the buffers of several input
channels are full of data. }
var
  lAdcChannelNo : array [0..3] of LongInt;

begin
  lAdcChannelNo[0] := 0;
  lAdcChannelNo[1] := 1;
  lAdcChannelNo[2] := 2;
  lAdcChannelNo[3] := 3;

  AxaiHwSetMultiBufferOverflowMode(4, @lAdcChannelNo, NEW_DATA_KEEP);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwSetLimit

Purpose

Set the upper and lower limit of the buffer of designated input channel.

Format

C++

```
DWORD AxaiHwSetLimit(long lAdcChannelNo, long lLowLimit, long
lUpLimit);
```

Visual Basic

```
Function AxaiHwSetLimit(ByVal lAdcChannelNo As Long, ByVal lLowLimit
As Long, ByVal lUpLimit As Long) As Long
```

Delphi

```
function AxaiHwSetLimit(lAdcChannelNo : LongInt; lLowLimit : LongInt;
lUpLimit : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number
[>>]lLowLimit	Lower limit of buffer
[>>]lUpLimit	Upper limit of buffer that cannot go over lBuffSize of AxaiHwStartMultiChannel

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the upper and lower limit of the buffer for the channel designated by user.

This API is used to set the upper limit to receive the buffer state event for each channel before calling up AxaiHwStartMultiChannel API or AxaiHwStartMultiFilter API or to check buffer state for the continuous signal check.

Other APIs related to this API used to check buffer state are AxaiHwIsBufferUpper and AxaiHwIsBufferLower. Data can be processed by creating event at the moment user wants when the number of data in memory buffer is over the upper limit or is below the lower limit. Refer to <Setting and Checking Input Module Interrupt/Channel Event> for this part.

lLowLimit sets the lower limit of the memory buffer allotted for each channel.

lUpLimit sets the upper limit of the the memory buffer allotted for each channel.

Example

C++

```
// Set the upper and lower limit of the buffer for input channel 0.
AxaiHwSetLimit(0, 0, 1000);
```

Visual Basic

```
' Set the upper and lower limit of the buffer for input channel 0.
AxaiHwSetLimit 0, 0, 1000
```

Delphi

```
{ Set the upper and lower limit of the buffer for input channel 0. }
AxaiHwSetLimit(0, 0, 1000);
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwGetLimit

Purpose

Verify the upper and lower limit of the buffer of designated input channel.

Format

C++

```
DWORD AxaiHwGetLimit(long lAdcChannelNo, long *lpLowLimit, long
*lpUpLimit);
```

Visual Basic

```
Function AxaiHwGetLimit(ByVal lAdcChannelNo As Long, ByRef lpLowLimit
As Long, ByRef lpUpLimit As Long) As Long
```

Delphi

```
function AxaiHwGetLimit(lAdcChannelNo : LongInt; lpLowLimit :
PLongInt; lpUpLimit : PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number
[<<]lpLowLimit	Lower limit of buffer
[<<]lpUpLimit	Upper limit of buffer

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Verify the upper and lower limit of the buffer for input channel designated by user.

* upLowLimit checks the lower limit of the memory buffer allotted for each channel.

* upUpLimit checks the upper limit of the memory buffer allotted for each channel.

Example

C++

```
// Verify the upper and lower limit of the buffer for input channel 0.
long lLowLimit = 0;
long lUpLimit = 0;

AxaiHwGetLimit(0, &lLowLimit, &lUpLimit);
```

Visual Basic

```
' Verify the upper and lower limit of the buffer for input channel 0.
Dim lLowLimit As Long
Dim lUpLimit As Long

AxaiHwGetLimit 0, lLowLimit, lUpLimit
```

Delphi

```
{ Verify the upper and lower limit of the buffer for input channel 0. }
var
   lLowLimit : LongInt;
   lUpLimit : LongInt;
   strData : String;

begin
   AxaiHwGetLimit(0, @lLowLimit, @lUpLimit);
   strData := 'LowLimit : ' + FloatToStr(lLowLimit) + ', UpLimit : ' +
      FloatToStr(lUpLimit);
   Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwSetMultiLimit

Purpose

Set the upper and lower limits of the buffers of several designated input channels.

Format

C++

```
DWORD AxaiHwSetMultiLimit(long lSize, long *lpAdcChannelNo, long lLowLimit, long lUpLimit);
```

Visual Basic

```
Function AxaiHwSetMultiLimit(ByVal lSize As Long, ByRef lpAdcChannelNo As Long, ByVal lLowLimit As Long, ByVal lUpLimit As Long) As Long
```

Delphi

```
function AxaiHwSetMultiLimit(lSize : LongInt; lpAdcChannelNo : PLongInt; lLowLimit : LongInt; lUpLimit : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channels for use
[>>]lpAdcChannelNo	Array of the number of channels for use
[>>]lLowLimit	Lower limit of buffer
[>>]lUpLimit	Upper limit of buffer

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the upper and lower limits of the buffers of several input channels designated by user.

This API is used to set the upper limit used to receive the buffer state event for each channel before calling up AxaiHwStartMultiChannel API or AxaiHwStartMultiFilter API or to check buffer state, for the continuous signal check.

It sets the upper and lower limits of the memory buffers allotted for channels at the same time.

Other APIs related to this API used to check buffer state are AxaiHwIsBufferUpper and AxaiHwIsBufferLower. Data can be processed by creating event at the moment user wants when the number of data in memory buffer is over the upper limit or is below the lower limit. Refer to <Setting and Checking Input Module Interrupt/Channel Event> for this part.

For lSize, input the number of channels for use.

*lpAdcChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

Example

C++

```
// Set the upper and lower limits of the buffers for several input channels.
long lAdcChannelNo[4] = {0, 1, 2, 3};

AxaiHwSetMultiLimit(4, lAdcChannelNo, 0, 1000);
```

Visual Basic

```
' Set the upper and lower limits of the buffers for several input channels.
Dim lAdcChannelNo(0 To 3) As Long

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiHwSetMultiLimit 4, lAdcChannelNo(0), 0, 1000
```

Delphi

```
{ Set the upper and lower limits of the buffers for several input channels. }
var
  lAdcChannelNo : array [0..3] of LongInt;

begin
  lAdcChannelNo[0] := 0;
  lAdcChannelNo[1] := 1;
  lAdcChannelNo[2] := 2;
  lAdcChannelNo[3] := 3;

  AxaiHwSetMultiLimit(4, @lAdcChannelNo, 0, 1000);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiSetTriggerMode

Purpose

Set trigger mode of the designated input module.

Format

C++

```
DWORD AxaiSetTriggerMode(long lModuleNo, DWORD uTriggerMode);
```

Visual Basic

```
Function AxaiSetTriggerMode(ByVal lModuleNo As Long, ByVal
uTriggerMode As Long) As Long
```

Delphi

```
function AxaiSetTriggerMode(lModuleNo : LongInt; uTriggerMode :
DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module number
[>>]uTriggerMode	A/D conversion mode : AXT_AIO_TRIGGER_MODE – [00h] Do not use the trigger – [01h] Software Trigger method that performs A/D conversion at the moment user wants (default value) – [02h] Trigger method that performs A/D conversion by using the internal clock of H/W – [03h] Trigger method that performs A/D conversion by using the clock of external input terminal

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Set trigger mode of the module designated by user.

An API that sets the A/D conversion method for each module must be called up. The basic setting value is Normal Trigger Mode

The table below shows Trigger Mode for each A/D conversion method.

Trigger Mode related APIs

Trigger Mode	Major Related APIs
Normal Trigger Mode	AxaiSwReadVoltage,

	AxaiSwReadMultiVoltage
Timer Trigger Mode	(API group : AxaiHw.., AxaiEvent..,AxaiInterrupt..) AxaiEventSetChannel AxaiHwSetSampleFreq, AxaiHwSetSamplePeriod AxaiHwStartMultiChannel , AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetLimit, AxaiHwReadDataLength AxaiHwReadSampleVoltage AxaiHwSetMultiAccess(ImmediateMode), AxaiHwStartMultiAccess(Immediate Mode) Etc..
External Trigger Mode	The same as Timer Trriger Mode, but Immediate Mode API and AxaiHwSetSampleFreq / AxaiHwSetSamplePeriod are not related.

uTriggerMode is a value that sets A/D conversion method. There are three A/d conversion methods: Normal Trigger method that performs sampling at the moment that user wants, Timer Trigger method that performs sampling by using the internal Timer of H/W, and External Trigger method that performs A/D conversion at constant period after a certain external signal. For Timer Trigger method, the sampling rate must be set by using AxaiHwSetSampleFreq or AxaiHwSetSamplePeriod API.

Example

C++

```
// Set Normal Trigger mode for input module 0.
AxaiSetTriggerMode(0, NORMAL_MODE);
```

Visual Basic

```
' Set Normal Trigger mode for input module 0.
AxaiSetTriggerMode 0, NORMAL_MODE
```

Delphi

```
{ Set Normal Trigger mode for input module 0. }
AxaiSetTriggerMode(0, NORMAL_MODE);
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiGetTriggerMode

Purpose

Verify the trigger mode of the designated input module.

Format

C++

```
DWORD AxaiGetTriggerMode(long lModuleNo, DWORD *upTriggerMode);
```

Visual Basic

```
Function AxaiGetTriggerMode(ByVal lModuleNo As Long, ByRef
upTriggerMode As Long) As Long
```

Delphi

```
function AxaiGetTriggerMode(lModuleNo : LongInt; upTriggerMode :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module number
[<<]upTriggerMode	A/D conversion mode : AXT_AIO_TRIGGER_MODE – [00h] Do not use the trigger – [01h] Software Trigger method that performs A/D conversion at the moment user wants (default value) – [02h] Trigger method that performs A/D conversion by using the internal clock of H/W – [03h] Trigger method that performs A/D conversion by using the clock of external input terminal

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Verify the Trigger Mode of the module that user designates.

Example

C++

```
// Verify the Trigger Mode of input module 0.
DWORD dwTriggerMode;

AxaiGetTriggerMode(0, &dwTriggerMode);
```

Visual Basic

```
' Verify the Trigger Mode of input module 0.
Dim lTriggerMode As Long

AxaiGetTriggerMode 0, lTriggerMode
```

Delphi

```
{ Verify the Trigger Mode of input module 0. }
var
   dwTriggerMode : DWord;

begin
   AxaiGetTriggerMode(0, @dwTriggerMode);
   Application.MessageBox (PCHAR(IntToStr(dwTriggerMode)), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwSetSampleFreq

Purpose

Set the sampling interval of the designated module in the unit of frequency.

Format

C++

```
DWORD AxaiHwSetSampleFreq(long lModuleNo, double dSampleFreq);
```

Visual Basic

```
Function AxaiHwSetSampleFreq(ByVal lModuleNo As Long, ByVal
dSampleFreq As Double) As Long
```

Delphi

```
function AxaiHwSetSampleFreq(lModuleNo : LongInt; dpSampleFreq :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module number
[>>]dSampleFreq	Setting value of sampling period in frequency unit (Hz)(10~100000)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

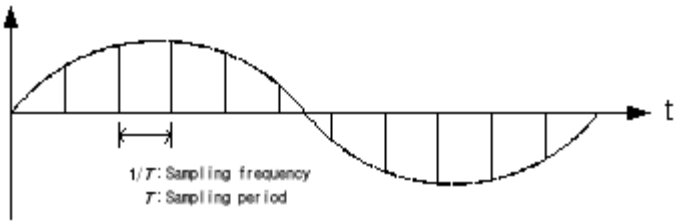
Description

Set the sampling internval of the module that user designates in frequency unit.

Sampling period must be set when the module for use takes Timer Trigger Mode as A/D conversion method.

The API sets the sampling period in frequency unit. As SIO-AI16HB has 16 internal channels that use a common ADC(Analog Digital Converter), the samplings for all 16 channels are done by one sampling frequency or period. For example, when performing sampling of one channel of SIO-AI16HB, the sampling can be done at the maximum frequency of 100Khz which is 0.01msec period. And if the sampling of 16 channels is done at the same time, the frequency that should be set by AxaiHwSetSampleFreq API cannot go over 100K/16 = 6250Hz.

However, if a value over the maxium sampling limit set by hardware is typed in, the sampling frequency is automatically figured again to be lower than the upper limit. For SIO-AI4RB whose hardware is different from SIO-AI16HB, as it uses 4 channel ADC, the selected number of channel does not impose limit on the frequency. However, the sampling frequency for SIO-AI4RB could not run over 100Khz, the ADC sampling limit.



dSampleFreq must convert analog data to digital data at constant interval. If the time is called sampling period, T, 1/T is sampling frequency. They are the frequency or period for sampling. The unit of frequency is Hz.

Example

C++

```
// Set the sampling interval for input module 0 as 1000Hz.
AxaiHwSetSampleFreq(0, 1000);
```

Visual Basic

```
' Set the sampling interval for input module 0 as 1000Hz.
AxaiHwSetSampleFreq 0, 1000
```

Delphi

```
{ Set the sampling interval for input module 0 as 1000Hz. }
AxaiHwSetSampleFreq(0, 1000);
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwGetSampleFreq

Purpose

Verify the sampling interval of the designated module in the unit of frequency.

Format

C++

```
DWORD AxaiHwGetSampleFreq(long lModuleNo, double *dpSampleFreq);
```

Visual Basic

```
Function AxaiHwGetSampleFreq(ByVal lModuleNo As Long, ByRef
dpSampleFreq As Double) As Long
```

Delphi

```
function AxaiHwGetSampleFreq(lModuleNo : LongInt; dpSampleFreq :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module number
[<<]dpSampleFreq	Setting value of sampling period in frequency unit (Hz) (10~100000)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Verify the sampling intervnal set in frequency unit of the module designated by user.

The unit of frequency is Hz.

Example

C++

```
// Verify the sampling interval set in frequency unit of input module 0.
double  dSampleFreq;

AxaiHwGetSampleFreq(0, &dSampleFreq);
```

Visual Basic

```
' Verify the sampling interval set in frequency unit of input module 0.
Dim dSampleFreq As Double

AxaiHwGetSampleFreq 0, dSampleFreq
```

Delphi

```
{ Verify the sampling interval set in frequency unit of input module 0. }
var
  dSampleFreq : Double;

begin
  AxaiHwGetSampleFreq(0, @dSampleFreq);
  Application.MessageBox (PCHAR(FloatToStr(dSampleFreq)), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwSetSamplePeriod

Purpose

Set the sampling interval of the designated module in a unit of time.

Format

C++

```
DWORD AxaiHwSetSamplePeriod(long lModuleNo, double dSamplePeriod);
```

Visual Basic

```
Function AxaiHwSetSamplePeriod(ByVal lModuleNo As Long, ByVal
dSamplePeriod As Double) As Long
```

Delphi

```
function AxaiHwSetSamplePeriod(lModuleNo : LongInt; dSamplePeriod :
Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module number
[>>]dSamplePeriod	Setting value of sampling period in time unit (uSec) (100000 ~ 1000000000)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Set the sampling interval of the module designated by user in time unit.

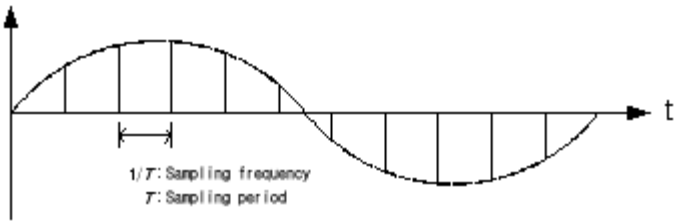
Sampling period must be set when the module for use takes Timer Trigger Mode as A/D conversion method.

This API sets the sampling period in time unit.

For SIO-AI16HB, all 16 internal channels use a common ADC(Analog Digital Converter), all 16 channels's sampling is done at one sampling frequency or period. The API sets the sampling period in frequency unit. As SIO-AI16HB has 16 internal channels that use a common ADC(Analog Digital Converter), the samplings for all 16 channels are done by one sampling frequency or period. For example, when performing sampling of one channel of SIO-AI16HB, the sampling can be done at the maximum frequency of 100Khz which is 0.01msec period. And if the sampling of 16 channels is done at the same time, the period that should be set by AxaiHwSetSamplePeriod API cannot go below (1000000 * (100K/16 = 6250Hz)) = 160uSec

However, if a value below the minimum sampling limit set by hardware is typed in, the sampling period is automatically figured again to be bigger than the lower limit. For SIO-AI4RB whose hardware is different from SIO-AI16HB, as it uses 4 channel ADC, the selected number of channel does not

impose limit on the period. However, the sampling period for SIO-AI4RB could not run below 10 uSec, the ADC sampling limit.



dSamplePeriod must convert analog data to digital data at constant interval. The time for conversion is called sampling period, T. 1/T is sampling frequency. They are the frequency or period for sampling. The unit of time is uSec.

Example

C++

```
// Set the sampling interval of input module 0 as 100mSec.
AxaiHwSetSamplePeriod(0, 100);
```

Visual Basic

```
' Set the sampling interval of input module 0 as 100mSec.
AxaiHwSetSamplePeriod 0, 100
```

Delphi

```
{ Set the sampling interval of input module 0 as 100mSec. }
AxaiHwSetSamplePeriod(0, 100);
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwGetSampleFreq, AxaiHwGetSamplePeriod,AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiHwGetSamplePeriod

Purpose

Verify the sampling interval of the designated module in the unit of time, uSec.

Format

C++

```
DWORD AxaiHwGetSamplePeriod(long lModuleNo, double *dpSamplePeriod);
```

Visual Basic

```
Function AxaiHwGetSamplePeriod(ByVal lModuleNo As Long, ByRef
dpSamplePeriod As Double) As Long
```

Delphi

```
function AxaiHwGetSamplePeriod(lModuleNo : LongInt; dpSamplePeriod :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module Number
[<<]dpSamplePeriod	The setting value of sampling period in time unit (uSec)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Verify the sampling interval of the module designated by user.

The time unit is uSec.

Example

C++

```
// Verify the sampling interval, in the unit of time, of input module 0.
double  dSamplePeriod;

AxaiHwGetSamplePeriod(0, &dSamplePeriod);
```

Visual Basic

```
' Verify the sampling interval, in the unit of time, of input module 0.
Dim dSamplePeriod As Double

AxaiHwGetSamplePeriod 0, dSamplePeriod
```

Delphi

```
{ Verify the sampling interval, in the unit of time, of input module 0. }
var
  dSamplePeriod : Double;

begin
  AxaiHwGetSamplePeriod(0, @dSamplePeriod);
  Application.MessageBox (PCHAR(FloatToStr(dSamplePeriod)), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, , AxaiSetModuleOffsetValue, AxaiGetModuleOffsetValue

AxaiSetModuleOffsetValue

Purpose

Set the Offset value of the designated input module in mVolt.

Format

C++

```
DWORD AxaiSetModuleOffsetValue(long lModuleNo, double dMiliVolt);
```

Visual Basic

```
Function AxaiSetModuleOffsetValue(ByVal lModuleNo As Long, ByVal
dMiliVolt As Double) As Long
```

Delphi

```
function AxaiSetModuleOffsetValue(lModuleNo : LongInt; dMiliVolt :
Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input module number
[>>]dMiliVolt	Offset setting value (mV) (-100~100)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[2101] AXT_RT_AIO_INVALID_MODULE_NO : Invalid AIO module
* See error code Table for more information on status error codes

Description

It is correct that 0V should be read if the input terminal receives 0V as an input. But the operational limit of hardware creates miniscule difference. This is a setting value to compensate the difference when taking in 0V from Calibrator, a constant voltage supplier used when adjusting measurement device by outputting correct constant voltage or constant current.

That is, for the case of reading correct 0V using Calibrator, if the measured input value is 0.010V, adjust Offset value as -10mV by using AxaiSetModuleOffsetValue API to compensate this value. After the moment, all measured values are applied with the newly set offset value and read.

The setting unit is mVolt. The maximum adjustable range is -100mV~100mV.

Example

C++

```
// Adjust the Offset of input module 0 as -10mV.

AxaiSetModuleOffsetValue(0, -10);
```

Visual Basic

```
' Adjust the Offset of input module 0 as -10mV.

AxaiSetModuleOffsetValue 0, -10
```

Delphi

```
{ Adjust the Offset of input module 0 as -10mV. }

begin
  AxaiSetModuleOffsetValue (0, -10);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiGetModuleOffsetValue

AxaiGetModuleOffsetValue

Purpose

Verify the Offset value of the designated input module.

Format

C++

```
DWORD AxaiGetModuleOffsetValue(long lModuleNo, double *dpMiliVolt);
```

Visual Basic

```
Function AxaiGetModuleOffsetValue(ByVal lModuleNo As Long, ByRef
dpMiliVolt As Double) As Long
```

Delphi

```
function AxaiGetModuleOffsetValue(lModuleNo : LongInt; dpMiliVolt :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Input moduel number
[<<]dpMiliVolt	Set offset value(mV) (-100~100)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[1054] AXT_RT_NOT_SUPPORT_VERSION : Hardware not supported
[2101] AXT_RT_AIO_INVALID_MODULE_NO : Invalid AIO module
* See error code Table for more information on status error codes

Description

Verify the Offset value set in input module, using AxaiSetModuleOffsetValue API. The unit is mVolt.

Example

C++

```
// Verify the Offset of input module 0.
double dMiliVolt;
AxaiGetModuleOffsetValue(0, &dMiliVolt);
```

Visual Basic

```
' Verify the Offset of input module 0.
Dim dMiliVolt As Double
AxaiGetModuleOffsetValue 0, dMiliVolt
```

Delphi

```
{ Verify the Offset of input module 0. }
var
  dMiliVolt: Double;

begin
  AxaiGetModuleOffsetValue (0, @dMiliVolt);
end;
```

See Also

AxaiSetRange, AxaiGetRange, AxaiSetMultiRange, AxaiHwSetBufferOverflowMode, AxaiHwGetBufferOverflowMode, AxaiHwSetMultiBufferOverflowMode, AxaiHwSetLimit, AxaiHwGetLimit, AxaiHwSetMultiLimit, AxaiSetTriggerMode, AxaiGetTriggerMode, AxaiHwSetSampleFreq, AxaiHwGetSampleFreq, AxaiHwSetSamplePeriod, AxaiHwGetSamplePeriod, AxaiSetModuleOffsetValue

A/D Conversion of Input Module

Function	Description
AxaiSwReadVoltage	Return analog input value to the designated input channel in voltage value.
AxaiSwReadDigit	Return analog input value to the designated input channel in digit value.
AxaiSwReadMultiVoltage	Return analog input value to several designated input channels in voltage value.
AxaiSwReadMultiDigit	Return analog input value to several designated input channels in digit value.
AxaiHwStartMultiChannel	Start A/D conversions for the designated input channels, using H/S timer.
AxaiHwStartMultiFilter	After starting A/D conversions for the designated input channels, filter out and return the wanted number of values as voltage.
AxaiHwStopMultiChannel	Stop the A/D conversion using H/W timer.
AxaiHwSetMultiAccess	Set the setting value to use Immediate mode for several designated input channels.
AxaiHwStartMultiAccess	Return data after performing A/D conversion of the designated number.
AxaiHwReadSampleVoltage	Read the designated input channel for the value after A/D conversion in voltage value, using H/W timer.
AxaiHwReadSampleDigit	Read the designated input channel for the value after A/D conversion in digit value , using H/W timer.

AxaiSwReadVoltage

Purpose

Return analog input value to the designated input channel in voltage value.

Format

C++

```
DWORD AxaiSwReadVoltage(long lAdcChannelNo, double *dpVolt);
```

Visual Basic

```
Function AxaiSwReadVoltage(ByVal lAdcChannelNo As Long, ByRef dpVolt
As Double) As Long
```

Delphi

```
function AxaiSwReadVoltage(lAdcChannelNo : LongInt; dpVolt :
PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number
[<<]dpVolt	Input voltage value (V)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Return analog input value as voltage after performing A/D conversion to the input channel designated by user.

Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Return analog input value to input channel 0 as voltage value.
double  dVolt;

AxaiSwReadVoltage(0, &dVolt);
```

Visual Basic

```
' Return analog input value to input channel 0 as voltage value.
Dim dVolt As Double

AxaiSwReadVoltage 0, dVolt
```

Delphi

```
{ Return analog input value to input channel 0 as voltage value. }
var
   dVolt : Double;

begin
   AxaiSwReadVoltage(0, @dVolt);
   Application.MessageBox (PCHAR(FloatToStr(dVolt)), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit, AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess,AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiSwReadDigit

Purpose

Return analog input value to the designated input channel in digit value.

Format

C++

```
DWORD AxaiSwReadDigit(long lAdcChannelNo, DWORD *upDigit);
```

Visual Basic

```
Function AxaiSwReadDigit(ByVal lAdcChannelNo As Long, ByRef upDigit
As Long) As Long
```

Delphi

```
function AxaiSwReadDigit(lAdcChannelNo : LongInt; upDigit : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number
[<<]upDigit	Digit value

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Return analog input value as digit value after performing A/D conversion to the input channel designated by user.

The range of the value depends on the resolution characteristics of module's H/W. SIO-AI16HB ha s 0~65535 as the range value. The meaning of the digit range is varied for each module. For SIO-AI16HB that uses fixed input value, it is Two's Complement as in the example. For SIO-AI4RB, the value of Digit 0 is the same as the minimum voltage of the set input range.

Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Return analog input value to input channel 0 as Digit value.
DWORD dwDigit;

AxaiSwReadDigit(0, &dwDigit);
short retShort = (short) dwDigit;          // Type conversion to signed 2Byte
double dVolt = (((double)retShort ) / 32768.0) * 10;
```

```
// For SIO-AI16HB, as Digit value is Two's complement, the closet voltage value would
// be given after the conversion.
```

Visual Basic

```
' Return analog input value to input channel 0 as Digit value.
Dim lDigit As Long

AxaiSwReadDigit 0, lDigit
```

Delphi

```
{ Return analog input value to input channel 0 as Digit value. }
var
  dwDigit : DWord;

begin
  AxaiSwReadDigit(0, @dwDigit);
  Application.MessageBox (PCHAR(IntToStr(dwDigit)), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiSwReadVoltage, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit, AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess, AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiSwReadMultiVoltage

Purpose

Return analog input value to several designated input channels in voltage value.

Format

C++

```
DWORD AxaiSwReadMultiVoltage(long lSize, long *lpAdcChannelNo, double *dpVolt);
```

Visual Basic

```
Function AxaiSwReadMultiVoltage(ByVal lSize As Long, ByRef lpAdcChannelNo As Long, ByRef dpVolt As Double) As Long
```

Delphi

```
function AxaiSwReadMultiVoltage(lSize : LongInt; lpAdcChannelNo : PLongInt; dpVolt : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channel for use
[>>]lpAdcChannelNo	Array of numbers of the channel for use
[<<]dpVolt	Array of voltages to be taken in as input

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Return analog input value to several channels designated by user as voltage value after performing A/D conversion.

Input the number of channel for use for lSize.

*lpAdcChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

*dpVolt is an array to store voltage value. For instance, for the case that uses 4 channels, declare is as double dVolt[4].

Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Return analog input value to several input channels as voltage value.
long lAdcChannelNo[4] = {0, 1, 2, 3};
double dVolt[4];

AxaiSwReadMultiVoltage(4, lAdcChannelNo, dVolt);
```

Visual Basic

```
' Return analog input value to several input channels as voltage value.
Dim lAdcChannelNo(0 To 3) As Long
Dim dVolt(0 To 3) As Double

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiSwReadMultiVoltage 4, lAdcChannelNo(0), dVolt(0)
```

Delphi

```
{ Return analog input value to several input channels as voltage value. }
var
  lAdcChannelNo : array [0..3] of LongInt;
  dVolt : array [0..3] of Double;

begin
  lAdcChannelNo[0] := 0;
  lAdcChannelNo[1] := 1;
  lAdcChannelNo[2] := 2;
  lAdcChannelNo[3] := 3;

  AxaiSwReadMultiVoltage(4, @lAdcChannelNo, @dVolt);
end;
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiDigit, AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess,AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiSwReadMultiDigit

Purpose

Return analog input value to several designated input channels in digit value.

Format

C++

```
DWORD AxaiSwReadMultiDigit(long lSize, long *lpAdcChannelNo, DWORD
*upDigit);
```

Visual Basic

```
Function AxaiSwReadMultiDigit(ByVal lSize As Long, ByRef
lpAdcChannelNo As Long, ByRef upDigit As Long) As Long
```

Delphi

```
function AxaiSwReadMultiDigit(lSize : LongInt; lpAdcChannelNo :
PLongInt; upDigit : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channel for use
[>>]lpAdcChannelNo	Array of numbers of the channel for use
[<<]upDigit	Array of digit values to be taken in as input

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Return analog input value to several channels designated by user as digit value after performing A/D conversion.

The range of the value depends on the resolution characteristics of module's H/W. SIO-AI16HB ha s 0~65535 as the range value. The meaning of the digit range is varied for each module. For SIO-AI16HB that uses fixed input value, it is Two's Complement as in the example. For SIO-AI4RB, the value of Digit 0 is the same as the minimum voltage of the set input range.

Input the number of channel for use for lSize.

*lpAdcChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

For nDataNum, input the number of data subject to A/D conversion for each channel

*upDigit is the array to store voltage values. For instance, if 4 channels are used, declare DWORD upDigit[4]. Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Return analog input value to several input channels as Digit value.
long lAdcChannelNo[4] = {0, 1, 2, 3};
DWORD dwDigit[4];

AxaiSwReadMultiDigit(4, lAdcChannelNo, dwDigit);
short retShort = (short) dwDigit[0];                // Type conversion to signed 2Byte
double dVoltFirst = (((double)retShort ) / 32768.0) * 10;
// For SIO-AI16HB, as Digit value is Two's complement, the closest voltage value would
// be given after the conversion.
```

Visual Basic

```
' Return analog input value to several input channels as Digit value.
Dim lAdcChannelNo(0 To 3) As Long
Dim lDigit(0 To 3) As Long

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiSwReadMultiDigit 4, lAdcChannelNo(0), dwDigit(0)
```

Delphi

```
{ Return analog input value to several input channels as Digit value. }
var
   lAdcChannelNo : array [0..3] of LongInt;
   dwDigit : array [0..3] of DWord;

begin
   lAdcChannelNo[0] := 0;
   lAdcChannelNo[1] := 1;
   lAdcChannelNo[2] := 2;
   lAdcChannelNo[3] := 3;

   AxaiSwReadMultiDigit(4, @lAdcChannelNo, @dwDigit);
end;
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess,AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiHwStartMultiChannel

Purpose

Start A/D conversions for the designated input channels, using H/S timer.

Format

C++

```
DWORD AxaiHwStartMultiChannel(long lSize, long *lpAdcChannelNo, long lBuffSize);
```

Visual Basic

```
Function AxaiHwStartMultiChannel(ByVal lSize As Long, ByRef lpAdcChannelNo As Long, ByVal lBuffSize As Long) As Long
```

Delphi

```
function AxaiHwStartMultiChannel(lSize : LongInt; lpAdcChannelNo : PLongInt; lBuffSize : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channel for use
[>>]lpAdcChannelNo	Array of numbers of the channel for use
[>>]lBuffSize	As it is the number of buffer allotted for each channel, it should run over the upper limt for the buffer of AxaiHwSetLimit

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Start A/D conversion for several input channels designated by user using H/W timer(Timer Trigger Mode/External Trigger Mode).

By using H/W timer, it inputs the analog input value of the designated channel after performing A/D conversion to the buffer in a continuous manner whenever Trigger signal is created until the moment that AxaiHwStopMultiChannel is called up.

The conversion value stored in internal buffer can be read by AxaiHwReadSampleVoltage or AxaiHwReadSampleDigit API.

Input the number of channel for use for lSize.

*lpAdcChannelNo is an array that stores channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

lBuffSize determines the number of buffer to be allotted for each channel. The buffer that internally stores A/D data is run in the form of circular buffer. Circular buffer is used to store infinite number of data in the finite buffer. It employs a method that stores data at the starting point again when the last spot of the buffer to store data is filled.

Example

C++

```
// Start A/D conversion for several input channels using H/W timer.
long lAdcChannelNo[4] = {0, 1, 2, 3};

AxaiHwStartMultiChannel(4, lAdcChannelNo, 1000);
```

Visual Basic

```
' Start A/D conversion for several input channels using H/W timer.
Dim lAdcChannelNo(0 To 3) As Long
Dim dVolt(0 To 3) As Double

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiHwStartMultiChannel 4, lAdcChannelNo(0), 1000
```

Delphi

```
{ Start A/D conversion for several input channels using H/W timer. }
var
   lAdcChannelNo : array [0..3] of LongInt;

begin
   lAdcChannelNo[0] := 0;
   lAdcChannelNo[1] := 1;
   lAdcChannelNo[2] := 2;
   lAdcChannelNo[3] := 3;

   AxaiHwStartMultiChannel(4, @lAdcChannelNo, 1000);
end;
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess,AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiHwStartMultiFilter

Purpose

After starting A/D conversions for several designated input channel, filter out and return the wanted number of values as voltage.

Format

C++

```
DWORD AxaiHwStartMultiFilter(long lSize, long *lpAdcChannelNo, long lFilterCount, long lBuffSize);
```

Visual Basic

```
Function AxaiHwStartMultiFilter(ByVal lSize As Long, ByRef lpAdcChannelNo As Long, ByVal lFilterCount As Long, ByVal lBuffSize As Long) As Long
```

Delphi

```
function AxaiHwStartMultiFilter(lSize : LongInt; lpAdcChannelNo : PLongInt; lFilterCount : LongInt; lBuffSize : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channel for use
[>>]lpAdcChannelNo	Array of numbers of the channel for use
[>>]lFilterCount	Number of data subject to filtering
[>>]lBuffSize	As it is the number of buffer allotted for each channel, it should run over the upper limt for the buffer of AxaiHwSetLimit

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

After starting A/D conversions for several input channels designated by user, filter out and return the wanted number of values as voltage.

Input the number of channel for use for lSize.

*lpAdcChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, long lpAdcChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

lFilterCount determines the number of data subject to filtering when using AxaiHwStartMultiFilter API. Filtering is applied to the set number of data. That is, the value stored in buffer at every sampling is the final average value of actual data up to lFilterCount amount.

lBuffSize determines the number of buffer to be allotted for each channel. The buffer that internally stores A/D data is run in the form of circular buffer. Circular buffer is used to store infinite number of data in the finite buffer. It employs a method that stores data at the starting point again when the last spot of the buffer to store data is filled.

Example

C++

```
// After starting A/D conversions for several input channels, filter out and return the
// set number of data in voltage.
long lAdcChannelNo[4] = {0, 1, 2, 3};

AxaiHwStartMultiFilter(4, lAdcChannelNo, 500, 512);
```

Visual Basic

```
' After starting A/D conversions for several input channels, filter out and return the
' set number of data in voltage.
Dim lAdcChannelNo(0 To 3) As Long

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

AxaiHwStartMultiFilter 4, lAdcChannelNo(0), 500, 512
```

Delphi

```
{ After starting A/D conversions for several input channels, filter out and return
the set number of data in voltage. }
var
  lAdcChannelNo : array [0..3] of LongInt;

begin
  lAdcChannelNo[0] := 0;
  lAdcChannelNo[1] := 1;
  lAdcChannelNo[2] := 2;
  lAdcChannelNo[3] := 3;

  AxaiHwStartMultiFilter(4, @lAdcChannelNo, 500, 512);
end;
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit, AxaiHwStartMultiChannel, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess, AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiHwStopMultiChannel

Purpose

Stop continuous signal A/D conversion using H/W timer.

Format

C++

```
DWORD AxaiHwStopMultiChannel(long lModuleNo);
```

Visual Basic

```
Function AxaiHwStopMultiChannel(ByVal lModuleNo As Long) As Long
```

Delphi

```
function AxaiHwStopMultiChannel(lModuleNo : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module number which A/D conversion would be stopped

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Stop A/D conversion using H/W timer. When lModuleNo is -1, the modules performing continuous signal check among all the modules mounted on the system by using AxaiHwStartMultiChannel or AxaiHwStartMultiFilter will be stopped.

Example

C++

```
// Stop A/D conversion using H/W timer.
AxaiHwStopMultiChannel(-1);
```

Visual Basic

```
' Stop A/D conversion using H/W timer.
AxaiHwStopMultiChannel -1
```

Delphi

```
{ Stop A/D conversion using H/W timer. }
AxaiHwStopMultiChannel(-1);
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit,

AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwSetMultiAccess,
AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiHwSetMultiAccess

Purpose

Set the setting value to use Immediate mode for several designated input channels.

Format

C++

```
DWORD AxaiHwSetMultiAccess(long lSize, long *lpAdcChannelNo, long
*lpWordSize);
```

Visual Basic

```
Function AxaiHwSetMultiAccess(ByVal lSize As Long, ByRef
lpAdcChannelNo As Long, ByRef lpWordSize As Long) As Long
```

Delphi

```
function AxaiHwSetMultiAccess(lSize : LongInt; lpAdcChannelNo :
PLongInt; lpWordSize : PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of input channel for use
[>>]lpAdcChannelNo	Array of numbers of the channel for use
[>>]lpWordSize	Number of data for each channel

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Set the setting value to use Immediate mode for several input channels designated by user.

Set the setting value to use AxaiHwStartMultiAccess API.

This mode takes a method that reads what have been stored in FIFO in H/W after performing A/D conversion.

*lpWordSize is the number of data for each channel. SIO-AI16HB can store 1024Word data internally. (For SIO-AI4RB, it is 4096Word) Therefore, if user makes use of 1 channel, 1024 data can be designated. For the case using a few channels, the number of data, of course, would take 1024 / the number of channel as the maximum setting value. If user sets any value bigger than this, the value is automatically limited for using.

The maximum number for each channel= 1024/number of channel for use

After running the API, open *lpWordSize to check the value automatically limited.

Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Set the setting value to use Immediate mode for several input channels.
long lAdcChannelNo[4] = {0, 1, 2, 3};
long lWordSize[4] = {1024, 1024, 1024, 1024};

AxaiHwSetMultiAccess(4, lAdcChannelNo, lWordSize);
```

Visual Basic

```
' Set the setting value to use Immediate mode for several input channels.
Dim lAdcChannelNo(0 To 3) As Long
Dim lWordSize(0 To 3) As Long

lAdcChannelNo(0) = 0
lAdcChannelNo(1) = 1
lAdcChannelNo(2) = 2
lAdcChannelNo(3) = 3

lWordSize(0) = 1024
lWordSize(1) = 1024
lWordSize(2) = 1024
lWordSize(3) = 1024

AxaiHwSetMultiAccess 4, lAdcChannelNo(0), lWordSize(0)
```

Delphi

```
{ Set the setting value to use Immediate mode for several input channels. }
var
   lAdcChannelNo : array [0..3] of LongInt;
   lWordSize : array [0..3] of LongInt;

begin
   lAdcChannelNo[0] := 0;
   lAdcChannelNo[1] := 1;
   lAdcChannelNo[2] := 2;
   lAdcChannelNo[3] := 3;

   lWordSize[0] := 1024;
   lWordSize[1] := 1024;
   lWordSize[2] := 1024;
   lWordSize[3] := 1024;

   AxaiHwSetMultiAccess(4, @lAdcChannelNo, @lWordSize);
end;
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit, AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel,

AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiHwStartMultiAccess

Purpose

Return voltage values after performing A/D conversion of the designated number.

Format

C++

```
DWORD AxaiHwStartMultiAccess(double *dpBuffer[]);
```

Visual Basic

```
Function AxaiHwStartMultiAccess(ByRef dpBuffer() As Double) As Long
```

Delphi

```
function AxaiHwStartMultiAccess(dpBuffer : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]dpBuffer[]	2 dimensinal array of voltages after A/D conversion

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
* See error code Table for more information on status error codes

Description

It takes a method that reads data that have been stored in FIFO in H/S after performing A/D conversion.

SIO-AI16HB can store 1024Word data internally. (For SIO-AI4RB, it is 4096Word) Therefore, if user makes use of 1 channel, 1024 data can be designated. For the case using a few channels, the number of data, of course, would take 1024 / the number of channel as the maximum setting value. If user sets any value bigger than this, the value is automatically limited for using. The API gives the number of data internally set for each channel. After the end of running the API, the value after A/D conversion is automatically stored in the array.

*dpBuffer[] is the array, which stores the data value after A/D conversion, that stores pointer for 2-dimensional array. In the first place, determine the channel for use and the number of data subject to A/D conversion for each channel. For instance, for the case using 4 channels and having 10 data subject to A/D conversion for each channel, declare double dBuffer[4][10] and double *pdBuffer[4]. And allot the declared pointer of dBuffer for pdBuffer.

Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Return the set amount of data after A/D conversion.
double  pdCBuffer[4][1024];
double  *pdBuffer[4];
```

```
long lCount;

for(lCount = 0; lCount < 4; lCount++)
{
  // Buffer alloting
  pdBuffer[lCount]  = pdCBuffer[lCount];
}

AxaiHwStartMultiAccess(pdBuffer);
```

Visual Basic

```
' Return the set amount of data after A/D conversion.
Dim dpCBuffer(0 To 3, 0 To 1023) As Double
Dim dpBuffer(0 To 3) As Double

dpBuffer(0) = dpCBuffer(0, 0)
dpBuffer(1) = dpCBuffer(1, 0)
dpBuffer(2) = dpCBuffer(2, 0)
dpBuffer(3) = dpCBuffer(3, 0)

AxaiHwStartMultiAccess(dpBuffer)
```

Delphi

```
{ Return the set amount of data after A/D conversion. }
var
  dpCBuffer : array [0..3, 0..1023] of Double;
  dpBuffer : array[0..3] of Pointer;

begin
  dpBuffer[0] := @dpCBuffer[0,0];
  dpBuffer[1] := @dpCBuffer[1,0];
  dpBuffer[2] := @dpCBuffer[2,0];
  dpBuffer[3] := @dpCBuffer[3,0];

  AxaiHwStartMultiAccess(@dpBuffer);
end;
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit, AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess, AxaiHwReadSampleVoltage, AxaiHwReadSampleDigit

AxaiHwReadSampleVoltage

Purpose

Read the designated input channel for the value after A/D conversion in voltage value using H/W timer.

Format

C++

```
DWORD AxaiHwReadSampleVoltage(long lAdcChannelNo, long *lpCount,
double *dpVolt);
```

Visual Basic

```
Function AxaiHwReadSampleVoltage(ByVal lAdcChannelNo As Long, ByRef
lpCount As Long, ByRef dpVolt As Double) As Long
```

Delphi

```
function AxaiHwReadSampleVoltage(lAdcChannelNo : LongInt; lpCount :
PLongInt; dpVolt : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number (0 ~ N-1)
[<>]lpCount	In : Desingnate the size of dpVolt array and the number of voltages to get Out : Number of effective voltages taken
[<<]dpVolt	Array of voltage values taken

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

By using H/W timer, read the set amount data among the data stored in buffer after A/D conversion through AxaiHwStartMultiChannel and AxaiHwStartMultiFilter API.

Before using the API, Timer Trigger Mode / External Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Read input channel 0 as voltage value for the values after A/D conversion using H/W
// timer.
double  dpVolt[1024];
long lCount = 0;
AxaiHwReadDataLength(0, &lCount); // Take the collected data in the internal buffer
if(lCount > 1024)
```

```
  lCount = 1024;
AxaiHwReadSampleVoltage(0, &lCount, dpVolt);
// The count actually read is returned to lpCount
```

Visual Basic

```
' Read input channel 0 as voltage value for the values after A/D conversion using H/W
' timer.
Dim dVolt(0 To 1024) As Double
Dim lCount As Long
lCount = 1024
AxaiHwReadSampleVoltage 0, lCount, dVolt(0)
' The count actually read is returned to lpCount
```

Delphi

```
{ Read input channel 0 as voltage value for the values after A/D conversion using H/W
timer. }
var
  lCount : LongInt;
  dVolt : array [0..1024] of Double;

begin
  lCount = 1024;
  AxaiHwReadSampleVoltage(0, @lCount, @dVolt);
end;
{ The count actually read is returned to lpCount }
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit, AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess,AxaiHwStartMultiAccess, AxaiHwReadSampleDigit

AxaiHwReadSampleDigit

Purpose

Read the designated input channel for the value after A/D conversion in digit value using H/W timer.

Format

C++

```
DWORD AxaiHwReadSampleDigit(long lAdcChannelNo, long *lpCount, DWORD *upDigit);
```

Visual Basic

```
Function AxaiHwReadSampleDigit(ByVal lAdcChannelNo As Long, ByRef lpCount As Long, ByRef upDigit As Long) As Long
```

Delphi

```
function AxaiHwReadSampleDigit(lAdcChannelNo : LongInt; lpCount : PLongInt; upDigit : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number (0 ~ N-1)
[<>]lpCount	In : Designate the size of dpVolt array and the number of voltages to get Out : Number of effective voltages taken
[<<]upDigit	Array of Digit values taken

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

By using H/W timer, read the set amount data among the data stored in buffer after A/D conversion through AxaiHwStartMultiChannel and AxaiHwStartMultiFilter API.

The range of Digit value depends on the resolution characteristics of module's H/W. SIO-AI16HB ha s 0~65535 as the range value. The meaning of the digit range is varied for each module. For SIO-AI16HB that uses fixed input value, it is Two's Complement as in the example. For SIO-AI4RB, the value of Digit 0 is the same as the minimum voltage of the set input range.

Before using this API, Timer Trigger Mode / External Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Read input channel 0 for the Digit values after A/D conversion using H/W timer.
DWORD upDigit[1024];
long lCount = 0;
AxaiHwReadDataLength(0, &lCount);
if(lCount > 1024)
  lCount = 1024;
AxaiHwReadSampleDigit(0, &lCount, upDigit);
short retShort = (short) upDigit[0];          // Type conversion to signed 2Byte
double dVoltFirst = (((double)retShort ) / 32768.0) * 10;
// For SIO-AI16HB, as Digit value is Two's complement, the closest voltage value would
// be given after the conversion.
```

Visual Basic

```
' Read input channel 0 for the Digit values after A/D conversion using H/W timer.
Dim lDigit(0 To 1024) As Long
Dim lCount As Long
lCount = 1024

AxaiHwReadSampleDigit 0, lCount, lDigit
```

Delphi

```
{ Read input channel 0 for the Digit values after A/D conversion using H/W timer. }
Var
  lCount : LongInt;
  dwDigit : array [0..1024] of DWord;

begin
  lCount = 1024;
  AxaiHwReadSampleDigit(0, @lCount, @dwDigit);
end;
```

See Also

AxaiSwReadVoltage, AxaiSwReadDigit, AxaiSwReadMultiVoltage, AxaiSwReadMultiDigit, AxaiHwStartMultiChannel, AxaiHwStartMultiFilter, AxaiHwStopMultiChannel, AxaiHwSetMultiAccess,AxaiHwStartMultiAccess, AxaiHwReadSampleVoltage

Checking the Buffer Status of Input Module

Function	Description
AxaiHwIsBufferEmpty	Check whether or not there is data in the memory buffer of the designated input channel.
AxaiHwIsBufferUpper	Check whether or not the amount of data of the memory buffer of the designated input channel is over the set upper limit.
AxaiHwIsBufferLower	Check whether or not the amount of data of the memory buffer of the designated input channel is below the set lower limit.
AxaiHwReadDataLength	Check the numer of data stored in the memory buffer of the designated input channel.

AxaiHwIsBufferEmpty

Purpose

Check whether or not there is data in the memory buffer of the designated input channel.

Format

C++

```
DWORD AxaiHwIsBufferEmpty(long lAdcChannelNo, DWORD *upEmpty);
```

Visual Basic

```
Function AxaiHwIsBufferEmpty(ByVal lAdcChannelNo As Long, ByRef
upEmpty As Long) As Long
```

Delphi

```
function AxaiHwIsBufferEmpty(lAdcChannelNo : LongInt; upEmpty :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number
[<<]upEmpty	Memory buffer status – [00h] When there is data in memory buffer – [01h] When there is no data in memory buffer

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Check whether or not there is data in the memory buffer allotted for each channel. If there is no data, return 'TRUE', and if there is even a single datum, return 'FALSE'.

Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Check if there is data in the memory buffer of input channel 0.
DWORD dwEmpty;
AxaiHwIsBufferEmpty(0, &dwEmpty);

if (dwEmpty == TRUE)
  AfxMessageBox("There is no data in the memory buffer of input channel 0.");
else
  AfxMessageBox("There is data in the memory buffer of input channel 0.");
```

Visual Basic

```
' Check if there is data in the memory buffer of input channel 0.
Dim lEmpty As Long

AxaiHwIsBufferEmpty 0, dwEmpty

If dwEmpty = TRUE Then
  MsgBox "There is no data in the memory buffer of input channel 0."
Else
  MsgBox "There is data in the memory buffer of input channel 0."
End If
```

Delphi

```
{ Check if there is data in the memory buffer of input channel 0. }
var
  dwEmpty : DWord;

begin
  AxaiHwIsBufferEmpty(0, @dwEmpty);

  if (dwEmpty = 1) then
     Application.MessageBox('There is no data in the memory buffer of input channel
        0.', 'Ajinextek', MB_OK);
  else
     Application.MessageBox('There is data in the memory buffer of input channel 0.',
        'Ajinextek', MB_OK);
end;
```

See Also

AxaiHwIsBufferUpper, AxaiHwIsBufferLower, AxaiHwReadDataLength

AxaiHwIsBufferUpper

Purpose

Check whether or not the amount of data of the memory buffer of the designated input channel is over the set upper limit.

Format

C++

```
DWORD AxaiHwIsBufferUpper(long lAdcChannelNo, DWORD *upUpper);
```

Visual Basic

```
Function AxaiHwIsBufferUpper(ByVal lAdcChannelNo As Long, ByRef
upUpper As Long) As Long
```

Delphi

```
function AxaiHwIsBufferUpper(lAdcChannelNo : LongInt; upUpper:
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number
[<<]upUpper	Memory buffer status – [00h] When data in memory buffer is below the lower limit – [01h] When data in memory buffer is above the upper limit

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Check if the amount of data in the memory buffer allotted for each channel is over or below the upper limit set by AxaiHwSetLimit. If it is over the limit, return 'TRUE', and if it is below the upper limit, return 'FALSE'.

Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Check if the amount of data in the memory buffer of input channel 0 is over the
// set upper limit.
DWORD dwUpper;

AxaiHwIsBufferUpper(0, &dwUpper);
```

```
if (dwUpper == TRUE)
  AfxMessageBox("The amount of data in the memory buffer of input channel 0 is over the
     upper limit.");
else
  AfxMessageBox("The amount of data in the memory buffer of input channel 0 is below
     the upper limit.");
```

Visual Basic

```
' Check if the amount of data in the memory buffer of input channel 0 is over the
' set upper limit.
Dim lUpper As Long

AxaiHwIsBufferUpper 0, lUpper

If lUpper = TRUE Then
  MsgBox "The amount of data in the memory buffer of input channel 0 is over the upper
     limit."
Else
  MsgBox "The amount of data in the memory buffer of input channel 0 is below the upper
     limit."
End If
```

Delphi

```
{ Check if the amount of data in the memory buffer of input channel 0 is over the
set upper limit. }
var
  dwUpper : DWord;

begin
  AxaiHwIsBufferUpper(0, @dwUpper);

  if (dwUpper = 1) then
     Application.MessageBox('The amount of data in the memory buffer of input channel 0
        is over the upper limit.', 'Ajinextek', MB_OK);
  else
     Application.MessageBox('The amount of data in the memory buffer of input channel 0
        is below the upper limit.', 'Ajinextek', MB_OK);
end;
```

See Also

AxaiHwIsBufferEmpty, AxaiHwIsBufferLower, AxaiHwReadDataLength

AxaiHwIsBufferLower

Purpose

Check whether or not the amount of data of the memory buffer of the designated input channel is below the set lower limit.

Format

C++

```
DWORD AxaiHwIsBufferLower(long lAdcChannelNo, DWORD *upLower);
```

Visual Basic

```
Function AxaiHwIsBufferLower(ByVal lAdcChannelNo As Long, ByRef
upLower As Long) As Long
```

Delphi

```
function AxaiHwIsBufferLower(lAdcChannelNo : LongInt; upLower :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number
[<<]upLower	Memory buffer status – [00h] When data in memory buffer is above the upper limit – [01h] When data in memory buffer is below the lower limit

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Check if the amount of data in the memory buffer allotted for each channel is over or below the lower limit set by AxaiHwSetLimit. If it is below the limit, return 'TRUE', and if it is above the lower limit, return 'FALSE'.

Before using this API, Normal Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Check if the amount of data in the memory buffer of input channel 0 is below the set
// lower limit.
DWORD dwLower;

AxaiHwIsBufferLower(0, &dwUpper);
```

```
if (dwUpper == TRUE)
  AfxMessageBox("The amount of data in the memory buffer of input channel 0 is below
     the lower limit.");
else
  AfxMessageBox("The amount of data in the memory buffer of input channel 0 is above
     the lower limit.");
```

Visual Basic

```
' Check if the amount of data in the memory buffer of input channel 0 is below the set
' lower limit.
Dim lLower As Long

AxaiHwIsBufferLower 0, lLower

If lLower = TRUE Then
  MsgBox "The amount of data in the memory buffer of input channel 0 is below the lower
     limit."
Else
  MsgBox "The amount of data in the memory buffer of input channel 0 is above the lower
     limit."
End If
```

Delphi

```
{ Check if the amount of data in the memory buffer of input channel 0 is below the set
lower limit. }
var
  dwLower : DWord;

begin
  AxaiHwIsBufferLower(0, @dwLower);

  if (dwLower = 1) then
     Application.MessageBox('The amount of data in the memory buffer of input channel 0
        is below the lower limit.', 'Ajinextek', MB_OK);
end;
```

See Also

AxaiHwIsBufferEmpty, AxaiHwIsBufferUpper, AxaiHwReadDataLength

AxaiHwReadDataLength

Purpose

Check the number of data stored in the memory buffer of the designated input channel.

Format

C++

```
DWORD AxaiHwReadDataLength(long lAdcChannelNo, long *lpDataLength);
```

Visual Basic

```
Function AxaiHwReadDataLength(ByVal lAdcChannelNo As Long, ByRef
lpDataLength As Long) As Long
```

Delphi

```
function AxaiHwReadDataLength(lAdcChannelNo : LongInt; lpDataLength :
PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lAdcChannelNo	Input channel number
[<<]lpDataLength	Number of data stored in memory buffer

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Return the number of data in the memory buffer allotted for each channel.

This API is only for continuous signal check such as AxaiHwStartMultiChannel or AxaiHwStartMultiFilter.

Before using this API, Timer Trigger Mode or External Trigger Mode must be set by using AxaiSetTriggerMode API.

Example

C++

```
// Check the number of data stored in the memory buffer of input channel 0.
long lDataLength;

AxaiHwReadDataLength(0, &lDataLength);
```

Visual Basic

```
' Check the number of data stored in the memory buffer of input channel 0.
Dim lDataLength As Long
AxaiHwReadDataLength 0, lDataLength
```

Delphi

```
{ Check the number of data stored in the memory buffer of input channel 0. }
var
  lDataLength : LongInt;

begin
  AxaiHwReadDataLength(0, @lDataLength);
  Application.MessageBox (PCHAR(IntToStr(lDataLength)), 'Ajinextek', MB_OK);
end;
```

See Also

AxaiHwIsBufferEmpty, AxaiHwIsBufferUpper, AxaiHwIsBufferLower

Output Module Information Search

Function	Description
AxaoInfoGetModuleNoOfChannelNo	Check module number as the designated output channel number.
AxaoInfoGetChannelCount	Check the total channel number of analog output module.

AxaoInfoGetModuleNoOfChannelNo

Purpose

Check module number as the designated output channel number.

Format

C++

```
DWORD AxaoInfoGetModuleNoOfChannelNo(long lDacChannelNo, long
*lpModuleNo);
```

Visual Basic

```
Function AxaoInfoGetModuleNoOfChannelNo(ByVal lDacChannelNo As Long,
ByRef lpModuleNo As Long) As Long
```

Delphi

```
function AxaoInfoGetModuleNoOfChannelNo(lDacChannelNo : LongInt;
lpModuleNo : PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lDacChannelNo	Output channel number (0 ~ N-1)
[<<]lpModuleNo	Module number

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Check module number as the output channel number designated by user.

Example

C++

```
// Check the module number of output channel 0.
long lModuleNo;
CString strData;

AxaoInfoGetModuleNoOfChannelNo(0, &lModuleNo);
strData.Format("The module number of output channel 0 is %d .", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the module number of output channel 0.
Dim lModuleNo As Long
Dim strData As String

AxaoInfoGetModuleNoOfChannelNo 0, lModuleNo
strData = "The module number of output channel 0 is" + "CStr(lModuleNo)."

MsgBox strData
```

Delphi

```
{ Check the module number of output channel 0. }
var
  lModuleNo : LongInt;
  strData : String;

begin
  AxaoInfoGetModuleNoOfChannelNo(0, @lModuleNo);
  strData  := 'The module number of output channel 0 is' + 'IntToStr(lModuleNo).';

  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaoInfoGetChannelCount

AxaoInfoGetChannelCount

Purpose

Check the total channel number of analog output module.

Format

C++

```
DWORD AxaoInfoGetChannelCount(long *lpDacChannelCount);
```

Visual Basic

```
Function AxaoInfoGetChannelCount(ByRef lpDacChannelCount As Long) As
Long
```

Delphi

```
function AxaoInfoGetChannelCount(lpDacChannelCount : PLongInt) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]lpDacChannelCount	Total channel number of output module

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2101] AXT_RT_AIO_NOT_MODULE : Invalid AIO module
* See error code Table for more information on status error codes

Description

Check the total channel number of analog output module registered in the system.

Example

C++

```
// Check the total channel number of analog output module.
long lCount;
CString strData;

AxaoInfoGetChannelCount(&lCount);
strData.Format("The total channel number of analog output module is %d . ", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the total channel number of analog output module.
Dim lCount As Long
Dim strData As String
```

```
AxaoInfoGetChannelCount lCount
strData = "The total channel number of analog output module is" + "CStr(lCount)."

MsgBox strData
```

Delphi

```
{ Check the total channel number of analog output module. }
var
   lCount : LongInt;
   strData : String;

begin
   AxaoInfoGetChannelCount(@lCount);
   strData  := 'The total channel number of analog output module is' +
      'IntToStr(lCount).';

   Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaoInfoGetModuleNoOfChannelNo

Setting and Checking Output Module

Function	Description
AxaoSetRange	Set the range of output voltage for the designated output channel.
AxaoGetRange	Check the range of output voltage for the designated output channel.
AxaoSetMultiRange	Set the range of output voltage for several designated output channels.
AxaoWriteVoltage	The designated output channel outputs the input voltage.
AxaoWriteMultiVoltage	Several designated output channels outputs the input voltage.
AxaoReadVoltage	Check the output voltage value of the designated output channel.
AxaoReadMultiVoltage	Check the output voltage values of several designated output channels.

AxaoSetRange

Purpose

Set the range of output voltage for the designated output channel.

Format

C++

```
DWORD AxaoSetRange(long lDacChannelNo, double dMinVolt, double
dMaxVolt);
```

Visual Basic

```
Function AxaoSetRange(ByVal lDacChannelNo As Long, ByVal dMinVolt As
Double, ByVal dMaxVolt As Double) As Long
```

Delphi

```
function AxaoSetRange(lDacChannelNo : LongInt; dMinVolt : Double;
dMaxVolt : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lDacChannelNo	Output channel number
[>>]dMinVolt	Min. output voltage
[>>]dMaxVolt	Max. output voltage

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the output voltage range of the output channel designated by user.

As the accuracy of output voltage varies with the setting, it must be set properly. For example, when using the 12 bit analog output module, suppose that the output that ranges 0V~5V is required. If the reference voltage is set as 0V~5V, the user can control the system with a unit of 0.244mV=5V/4096 in theory. But if the reference voltage is set as -10V ~ 10V, the user can control the system with a unit of 4.8mV=20V/4096. Therefore the reference voltage must be determined adequately to control the system accurately.

The minimum output voltage means the lowest voltage among the reference voltages among analog output. That is, user cannot measure any voltage smaller than this. The value that user sets is determined as a value closest to one of the given reference values, including the lowest reference voltage that hardware provides. As SIO-AO4R provides -10V, -5V, and 0V as the lowest reference voltage, if user inputs -8V, -10V is automatically set.

Maximum output voltage means the highest voltage among reference voltages of analog output. The value that user sets is determined as a value closest to one of the given reference values, including the highest reference voltages that hardware provides. As SIO-AO4R provides 10V and 5V as the maximum reference voltage, if user inputs 8V, it is automatically set as 10V.

On the contrary, SIO-AO2HB, Output 2 channel module, of PCI-AI16HR board whose resolution is 16bit has the fixed output voltage range. Thus, when running the API, AXT_RT_AIO_INVALID_USE error is returned.

Setting range of ouput voltage

Module	Min. Voltage	Max. Voltage
SIO-AO4R	-10V	10V
	-5V	5V
	0V	5V
	0V	10V
SIO-AO2HB(PCI-AI16HR) Fix the range of output volage	-10V	10V

Example

C++

```
// Set the output voltage range of output channel 0.
AxaoSetRange(0, -10, 10);
```

Visual Basic

```
' Set the output voltage range of output channel 0.
AxaoSetRange 0, -10, 10
```

Delphi

```
{ Set the output voltage range of output channel 0. }
AxaoSetRange(0, -10, 10);
```

See Also

AxaoGetRange, AxaoSetMultiRange, AxaoWriteVoltage, AxaoWriteMultiVoltage, AxaoReadVoltage, AxaoReadMultiVoltage

AxaoGetRange

Purpose

Check the range of output voltage for the designated output channel.

Format

C++

```
DWORD AxaoGetRange(long lDacChannelNo, double *dpMinVolt, double
*dpMaxVolt);
```

Visual Basic

```
Function AxaoGetRange(ByVal lDacChannelNo As Long, ByRef dpMinVolt As
Double, ByRef dpMaxVolt As Double) As Long
```

Delphi

```
function AxaoGetRange(lDacChannelNo : LongInt; dpMinVolt : PDouble;
dpMaxVolt : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lDacChannelNo	Ouput channel number (0 ~ N-1)
[<<]dpMinVolt	Min. output voltage
[<<]dpMaxVolt	Max. output voltage

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Check the output voltage range of the output channel designated by user.

It is used when user checks the actual set value after setting the reference voltage using AxaiSetRange API. Therefore, when user sets any reference voltage that H/W does not provide, the actual set value can be different from the initial value set by the user.

Setting range of output voltage

Module	Min. Voltage	Max. Voltage
SIO-AO4R	-10V	10V
	-5V	5V
	0V	5V
	0V	10V

SIO-AO2HB(PCI-AI16HR) Fix the range of output voltage	-10V	10V

Example

C++

```
// Check the output voltage range of output channel 0.
double  dMinVolt;
double  dMaxVolt;

AxaoGetRange(0, &dMinVolt, &dMaxVolt);
```

Visual Basic

```
' Check the output voltage range of output channel 0.
Dim lMask As Long

AxaoGetRange 0, dMinVolt, dMaxVolt
```

Delphi

```
{ Check the output voltage range of output channel 0.}
var
 dMinVolt : Double;
 dMaxVolt : Double;
 strData : String;

begin
  AxaoGetRange(0, @dMinVolt, @dMaxVolt);

  strData := 'MinVolt : ' + FloatToStr(dMinVolt) + ', MaxVolt : ' +
     FloatToStr(dMaxVolt);
  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxaoSetRange, AxaoSetMultiRange, AxaoWriteVoltage, AxaoWriteMultiVoltage, AxaoReadVoltage, AxaoReadMultiVoltage

AxaoSetMultiRange

Purpose

Set the range of output voltage for several designated output channels.

Format

C++

```
DWORD AxaoSetMultiRange(long lSize, long *lpDacChannelNo, double
dMinVolt, double dMaxVolt);
```

Visual Basic

```
Function AxaoSetMultiRange(ByVal lSize As Long, ByRef lpDacChannelNo
As Long, ByVal dMinVolt As Double, ByVal dMaxVolt As Double) As Long
```

Delphi

```
function AxaoSetMultiRange(lSize : LongInt; lpDacChannelNo : PLongInt;
dMinVolt : Double; dMaxVolt : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of output channel for use
[>>]lpDacChannelNo	Array of numbers of the channel for use
[>>]dMinVolt	Min. output voltage
[>>]dMaxVolt	Max. output voltage

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Set the output voltage range of the output channel designated by user.

As the accuracy of output voltage varies with the setting, it must be set properly. For example, when using the 12 bit analog output module, let's suppose that the output that ranges 0V~5V is required. If the reference voltage is set as 0V~5V, the user can control the system with a unit of 0.244mV=5V/4096 in theory. But if the reference voltage is set as -10V ~ 10V, the user can control the system with a unit of 4.8mV=20V/4096. Therefore the reference voltage must be determined adequately to control the system accurately.

The minimum output voltage means the lowest voltage among the reference voltages among analog output. That is, user cannot measure any voltage smaller than this. The value that user sets is determined as a value closest to one of the given reference values, including the lowest reference voltage that hardware provides. As SIO-AO4R provides -10V, -5V, and 0V as the lowest reference voltage, if user inputs -8V, -10V is automatically set.

Maximum output voltage means the highest voltage among reference voltages of analog output. The value that user sets is determined as a value closest to one of the given reference values, including the highest reference voltages that hardware provides. As SIO-AO4R provides 10V and 5V as the maximum reference voltage, if user inputs 8V, it is automatically set as 10V.

On the contrary, SIO-AO2HB, Output 2 channel module, of PCI-AI16HR single unit board whose resolution is 16bit has the fixed output voltage range. Thus, when running the API, AXT_RT_AIO_INVALID_USE error is returned.

Setting range of output voltage

Module	Min. Voltage	Max. Voltage
SIO-AO4R	-10V	10V
	-5V	5V
	0V	5V
	0V	10V
SIO-AO2HB(PCI-AI16HR) Fix the output voltage range	-10V	10V

Example

C++

```
// Set the output voltage range for several output channels.
long lDacChannelNo[4] = {0, 1, 2, 3};

AxaoSetMultiRange(4, lDacChannelNo, -10, 10);
```

Visual Basic

```
' Set the output voltage range for several output channels.
Dim lDacChannelNo(0 To 3) As Long

lDacChannelNo(0) = 0
lDacChannelNo(1) = 1
lDacChannelNo(2) = 2
lDacChannelNo(3) = 3

AxaoSetMultiRange 4, lDacChannelNo(0), -10, 10
```

Delphi

```
{ Set the output voltage range for several output channels. }
var
   lDacChannelNo : array [0..3] of LongInt;

begin
   lDacChannelNo[0] := 0;
   lDacChannelNo[1] := 1;
   lDacChannelNo[2] := 2;
   lDacChannelNo[3] := 3;
```

```
  AxaoSetMultiRange(4, @lDacChannelNo, -10, 10);
end;
```

See Also

AxaoSetRange, AxaoGetRange, AxaoWriteVoltage, AxaoWriteMultiVoltage, AxaoReadVoltage, AxaoReadMultiVoltage

AxaoWriteVoltage

Purpose

The designated output channel outputs the input voltage.

Format

C++

```
DWORD AxaoWriteVoltage(long lDacChannelNo, double dVolt);
```

Visual Basic

```
Function AxaoWriteVoltage(ByVal lDacChannelNo As Long, ByVal dVolt As Double) As Long
```

Delphi

```
function AxaoWriteVoltage(lDacChannelNo : LongInt; dVolt : Double) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lDacChannelNo	Output channel number
[>>]dVolt	Output voltage value (V)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

The output channel designated by user outputs the input voltage.

If user inputs a voltage, the output voltage is the voltage inputted by the user and keeps the value until a new value is set.

This output voltage cannot run over the range set by AxaoSetRange.

Example

C++

```
// Output channel 0 outputs 5.7V.
AxaoWriteVoltage(0, 5.7);
```

Visual Basic

```
' Output channel 0 outputs 5.7V
AxaoWriteVoltage 0, 5.7
```

Delphi

```
{ Output channel 0 outputs 5.7V. }
AxaoWriteVoltage(0, 5.7);
```

See Also

AxaoSetRange, AxaoGetRange, AxaoSetMultiRange, AxaoWriteMultiVoltage, AxaoReadVoltage, AxaoReadMultiVoltage

AxaoWriteMultiVoltage

Purpose

Several designated output channels output the input voltage.

Format

C++

```
DWORD AxaoWriteMultiVoltage(long lSize, long *lpDacChannelNo, double
*dpVolt);
```

Visual Basic

```
Function AxaoWriteMultiVoltage(ByVal lSize As Long, ByRef
lpDacChannelNo As Long, ByRef dpVolt As Double) As Long
```

Delphi

```
function AxaoWriteMultiVoltage(lSize : LongInt; lpDacChannelNo :
PLongInt; dpVolt : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of channel for use
[>>]lpDacChannelNo	Array of numbers of the channel for use
[>>]dpVolt	Array of output voltages

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Several output channels designated buy user output the input voltage.

Input the number of channel for use for lSize.

*lpDacChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, lpDacChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

*dpVolt is an array to store voltage value. For instance, for the case that uses 4 channels, declare it as double dVolt[4] and fill the array value with the voltage values subject to output.

If user inputs a voltage, the output voltage is the voltage inputted by the user and keeps the value until a new value is set.

The output voltage cannot run over the voltage range set by AxaoSetRange.

Example

C++

```
// Several output channels output different voltages.
long lDacChannelNo[4] = {0, 1, 2, 3};
double dVolt[4] = {3.2, 5,3, 6,2, 8.7};

AxaoWriteMultiVoltage(4, lDacChannelNo, dVolt);
```

Visual Basic

```
' Several output channels output different voltages
Dim lDacChannelNo(0 To 3) As Long
Dim dVolt(0 To 3) As Double

lDacChannelNo(0) = 0
lDacChannelNo(1) = 1
lDacChannelNo(2) = 2
lDacChannelNo(3) = 3

dVolt(0) = 3.2
dVolt(1) = 5.3
dVolt(2) = 6.2
dVolt(3) = 8.7

AxaoWriteMultiVoltage 4, lDacChannelNo(0), dVolt(0)
```

Delphi

```
{ Several output channels output different voltages. }
var
  DacChannelNo : array [0..3] of LongInt;
  Volt : array [0..3] of Double;

begin
  DacChannelNo[0] := 0;
  DacChannelNo[1] := 1;
  DacChannelNo[2] := 2;
  DacChannelNo[3] := 3;
  Volt[0] := 3.2;
  Volt[1] := 5.3;
  Volt[2] := 6.2;
  Volt[3] := 8.7;

  AxaoWriteMultiVoltage(4, @lDacChannelNo, @dVolt);
end;
```

See Also

AxaoSetRange, AxaoGetRange, AxaoSetMultiRange, AxaoWriteVoltage, AxaoReadVoltage, AxaoReadMultiVoltage

AxaoReadVoltage

Purpose

Check the output voltage value of the designated output channel.

Format

C++

```
DWORD AxaoReadVoltage(long lDacChannelNo, double *dpVolt);
```

Visual Basic

```
Function AxaoReadVoltage(ByVal lDacChannelNo As Long, ByRef dpVolt As Double) As Long
```

Delphi

```
function AxaoReadVoltage(lDacChannelNo : LongInt; dpVolt : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lDacChannelNo	Output channel number
[<<]dpVolt	Output voltage value (V)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Check the output voltage value of the output channel designated by user.

This output voltage cannot run over the voltage range set by AxaoSetRange.

Example

C++

```
// Return analog output voltage to output channel 0 as voltage value.
double  dVolt;

AxaoReadVoltage(0, &dVolt);
```

Visual Basic

```
' Return analog output voltage to output channel 0 as voltage value
Dim dVolt As Double

AxaoReadVoltage 0, dVolt
```

Delphi

```
{ Return analog output voltage to output channel 0 as voltage value. }
var
  dVolt : Double;

begin
  AxaoReadVoltage(0, @dVolt);
  Application.MessageBox (PCHAR(FloatToStr(dVolt)), 'Ajinextek', MB_OK);
end;
```

See Also

AxaoSetRange, AxaoGetRange, AxaoSetMultiRange, AxaoWriteVoltage, AxaoWriteMultiVoltage, AxaoReadMultiVoltage

AxaoReadMultiVoltage

Purpose

Check the output voltage values of several designated output channels.

Format

C++

```
DWORD AxaoReadMultiVoltage(long lSize, long *lpDacChannelNo, double
*dpVolt);
```

Visual Basic

```
Function AxaoReadMultiVoltage(ByVal lSize As Long, ByRef
lpDacChannelNo As Long, ByRef dpVolt As Double) As Long
```

Delphi

```
function AxaoReadMultiVoltage(lSize : LongInt; lpDacChannelNo :
PLongInt; dpVolt : PDouble) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lSize	Number of output channel for use
[>>]lpDacChannelNo	Array of numbers of the channel for use
[<<]dpVolt	Array of output voltages

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO : Invalid AIO channel number
* See error code Table for more information on status error codes

Description

Check the output voltage values of several designated output channels.

Input the number of channel for use for lSize.

*lpDacChannelNo is an array that would store channel information. *lpAdcChannelNo can be filled with the channel numbers for use. For example, lpDacChannelNo[4] = {0, 2, 3, 4} means that 0, 2, 3, and 4 channels are set.

*dpVolt is an array to store voltage value. For instance, for the case that uses 4 channels, declare it as double dVolt[4].

This output voltage cannot run over the voltage range set by AxaoSetRange.

Example

C++

```
// Check the output voltage for each output channel.
long lDacChannelNo[4] = {0, 1, 2, 3};
double dVolt[4]

AxaoReadMultiVoltage(4, lDacChannelNo, dVolt);
```

Visual Basic

```
' Check the output voltage for each output channel.
Dim lDacChannelNo(0 To 3) As Long
Dim dVolt(0 To 3) As Double

lDacChannelNo(0) = 0
lDacChannelNo(1) = 1
lDacChannelNo(2) = 2
lDacChannelNo(3) = 3

AxaoReadMultiVoltage 4, lDacChannelNo(0), dVolt(0)
```

Delphi

```
{ Check the output voltage for each output channel. }
var
  lDacChannelNo : array [0..3] of LongInt;
  dVolt : array [0..3] of Double;

begin
  lDacChannelNo[0] := 0;
  lDacChannelNo[1] := 1;
  lDacChannelNo[2] := 2;
  lDacChannelNo[3] := 3;

  AxaoReadMultiVoltage(4, @lDacChannelNo, @dVolt);
end;
```

See Also

AxaoSetRange, AxaoGetRange, AxaoSetMultiRange, AxaoWriteVoltage, AxaoWriteMultiVoltage, AxaoReadVoltage

Error Code Table Check

Error coded returned from the library functions can be used to check the errors generated during the execution of library functions and to process the result accordingly. Error code returned from each of the library functions indicates reasons of failure of corresponding function, and you can check the error code to understand the cause of failure. In general, it is good idea to check the code when the function returns the code other than AXT_RT_SUCCESS for the purpose of easier debugging.

Error Code	Description
[0000] AXT_RT_SUCCESS	API function executed successfully
[1001] AXT_RT_OPEN_ERROR	Library is not open
[1002] AXT_RT_OPEN_ALREADY	Library is already open
[1053] AXT_RT_NOT_OPEN	Failed to initialize the library
[1054] AXT_RT_NOT_SUPPORT_VERSION	Hardware not supported
[1101] AXT_RT_INVALID_BOARD_NO	Invalid board number
[1102] AXT_RT_INVALID_MODULE_POS	Invalid module position
[1103] AXT_RT_INVALID_LEVEL	Invalid level
[1151] AXT_RT_ERROR_VERSION_READ	Failed to read library version
[1152] AXT_RT_NETWORK_ERROR	In the network connection, slave node is not connected or has error
[1153] AXT_RT_NETWORK_LOCK_MISMATCH	Stored connection information and the actual connection information is incorrect. Lock of the board in EzConfig information does not match with the actual connection status. Lock information for reconstruction of a slave existing EzConfig should give clear in the unclock
[1160] AXT_RT_1ST_BELOW_MIN_VALUE	First parameter is below the minimum value
[1161] AXT_RT_1ST_ABOVE_MAX_VALUE	First parameter is above the maximum value
[1170] AXT_RT_2ND_BELOW_MIN_VALUE	Second parameter is below the minimum value
[1171] AXT_RT_2ND_ABOVE_MAX_VALUE	Second parameter is above the maximum value
[1180] AXT_RT_3RD_BELOW_MIN_VALUE	Third parameter is below the minimum value
[1181] AXT_RT_3RD_ABOVE_MAX_VALUE	Third parameter is above the maximum value
[1190] AXT_RT_4TH_BELOW_MIN_VALUE	Fourth parameter is below the minimum value
[1191] AXT_RT_4TH_ABOVE_MAX_VALUE	Fourth parameter is above the maximum value
[1200] AXT_RT_5TH_BELOW_MIN_VALUE	Fifth parameter is below the minimum value
[1201] AXT_RT_5TH_ABOVE_MAX_VALUE	Fifth parameter is above the maximum value
[1210] AXT_RT_6TH_BELOW_MIN_VALUE	Sixth parameter is below the minimum value
[1211] AXT_RT_6TH_ABOVE_MAX_VALUE	Sixth parameter is above the maximum value
[1220] AXT_RT_7TH_BELOW_MIN_VALUE	Seventh parameter is below the minimum value
[1221] AXT_RT_7TH_ABOVE_MAX_VALUE	Seventh parameter is above the maximum value
[1230] AXT_RT_8TH_BELOW_MIN_VALUE	Eighth parameter is below the minimum value

[1231] AXT_RT_8TH_ABOVE_MAX_VALUE	Eighth parameter is above the maximum value
[1240] AXT_RT_9TH_BELOW_MIN_VALUE	Nineth parameter is below the minimum value
[1241] AXT_RT_9TH_ABOVE_MAX_VALUE	Nineth parameter is above the maximum value
[1250] AXT_RT_10TH_BELOW_MIN_VALUE	Tenth parameter is below the minimum value
[1251] AXT_RT_10TH_ABOVE_MAX_VALUE	Tenth parameter is above the maximum value
[2001] AXT_RT_AIO_OPEN_ERROR	Failed to open AIO module
[2051] AXT_RT_AIO_NOT_MODULE	No AIO module
[2052] AXT_RT_AIO_NOT_EVENT	Failed to read AIO event
[2101] AXT_RT_AIO_INVALID_MODULE_NO	Invalid AIO module
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO	Invalid AIO channel number
[2106] AXT_RT_AIO_INVALID_USE	Can not use AIO
[2107] AXT_RT_AIO_INVALID_TRIGGER_MODE	Invalid trigger mode
[3001] AXT_RT_DIO_OPEN_ERROR	Failed to open DIO module
[3051] AXT_RT_DIO_NOT_MODULE	No DIO module
[3052] AXT_RT_DIO_NOT_INTERRUPT	DIO Interrupt not set
[3101] AXT_RT_DIO_INVALID_MODULE_NO	Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO	Invalid DIO OFFSET number
[3103] AXT_RT_DIO_INVALID_LEVEL	Invalid DIO level
[3104] AXT_RT_DIO_INVALID_MODE	Invalid DIO mode
[3105] AXT_RT_DIO_INVALID_VALUE	Invalid setting value
[3106] AXT_RT_DIO_INVALID_USE	Can not use DIO API function
[4001] AXT_RT_MOTION_OPEN_ERROR	Failed to open motion library
[4051] AXT_RT_MOTION_NOT_MODULE	No motion module is installed in the system
[4052] AXT_RT_MOTION_NOT_INTERRUPT	Failed to read the resul of interrupt
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO	Failed to initialize the motion of corresponding axis
[4054] AXT_RT_MOTION_NOT_IN_CONT_INTERPOL	Command to stop continuous interpolation while not in continuous interpolation motion
[4055] AXT_RT_MOTION_NOT_PARA_READ	Failed to load parameters for home return drive
[4101] AXT_RT_MOTION_INVALID_AXIS_NO	Corresponding axis does not exist
[4102] AXT_RT_MOTION_INVALID_METHOD	Invalid setting for corresponding axis drive
[4103] AXT_RT_MOTION_INVALID_USE	Invalid setting for 'uUse' parameter
[4104] AXT_RT_MOTION_INVALID_LEVEL	Invalid setting for 'uLevel' parameter
[4105] AXT_RT_MOTION_INVALID_BIT_NO	Invalid setting for general purpose input/output bit
[4106] AXT_RT_MOTION_INVALID_STOP_MODE	Invalid setting values for motion stop mode
[4107] AXT_RT_MOTION_INVALID_TRIGGER_MODE	Invalid setting for trigger setting mode
[4108] AXT_RT_MOTION_INVALID_TRIGGER_LEVEL	Invalid setting for trigger output level

[4109] AXT_RT_MOTION_INVALID_SELECTION	'uSelection' parameter is set to a value other than COMMAND or ACTUAL
[4110] AXT_RT_MOTION_INVALID_TIME	Invalid setting for Trigger output time value
[4111] AXT_RT_MOTION_INVALID_FILE_LOAD	Failed to load the file containing motion setting values
[4112] AXT_RT_MOTION_INVALID_FILE_SAVE	Failed to save the file containing motion setting values
[4113] AXT_RT_MOTION_INVALID_VELOCITY	Motion error occurred since the motion drive velocity value is set as zero
[4114] AXT_RT_MOTION_INVALID_ACCELTIME	Motion error occurred since motion drive acceleration time value is set as zero
[4115] AXT_RT_MOTION_INVALID_PULSE_VALUE	Input pulse value is set to a value less than zero when setting motion unit
[4116] AXT_RT_MOTION_INVALID_NODE_NUMBER	Called position or velocity override function while not in motion
[4117] AXT_RT_MOTION_INVALID_TARGET	Flag indicating the cause of multi axis motion stop
[4151] AXT_RT_MOTION_ERROR_IN_NONMOTION	Not in motion drive when it should be
[4152] AXT_RT_MOTION_ERROR_IN_MOTION	Called another motion drive fuction before the completetion of current motion
[4153] AXT_RT_MOTION_ERROR	Error occurred during the execution of multi axis motion stop function
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE	Gantry-enable is activated again while in motion with gantry enabled
[4155] AXT_RT_MOTION_ERROR_GANTRY_AXIS	Invalid input of gantry axis master channel (axis) number (starting from zero)
[4156] AXT_RT_MOTION_ERROR_MASTER_SERVOON	Servo ON is not enabled for the master axis
[4157] AXT_RT_MOTION_ERROR_SLAVE_SERVOON	Servo ON is not enabled for the slavve axis
[4158] AXT_RT_MOTION_INVALID_POSITION	Not in a valid position
[4159] AXT_RT_ERROR_NOT_SAME_MODULE	Not in the same module
[4160] AXT_RT_ERROR_NOT_SAME_BOARD	Not in the same board
[4161] AXT_RT_ERROR_NOT_SAME_PRODUCT	When products are different each other
[4162] AXT_RT_NOT_CAPTURED	Failed to capture the position
[4163] AXT_RT_ERROR_NOT_SAME_IC	Not in the same chip
[4164] AXT_RT_ERROR_NOT_GEARMODE	Failed to change to the gear mode
[4165] AXT_ERROR_CONTI_INVALID_AXIS_NO	Invalid axis during continuous interpolation axix mapping
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO	Invalid mapping number during continuous interpolation mapping
[4167] AXT_ERROR_CONTI_EMPTY_MAP_NO	Continuous interpolation mapping number is empty
[4168] AXT_RT_MOTION_ERROR_CACULATION	Error in calculation
[4169] AXT_RT_ERROR_MOVE_SENSOR_CHECK	Before continuous interpolation, sensor error (Alarm, EMG, Limit, etc.) is detected
[4170] AXT_ERROR_HELICAL_INVALID_AXIS_NO	Invalid axis for helical axis mapping
[4171] AXT_ERROR_HELICAL_INVALID_MAP_NO	Invalid mapping number for helical mapping
[4172] AXT_ERROR_HELICAL_EMPTY_MAP_NO	Helical mapping number is empty
[4180] AXT_ERROR_SPLINE_INVALID_AXIS_NO	Invalid axis for spline axis mapping

[4181] AXT_ERROR_SPLINE_INVALID_MAP_NO	Invalid mapping number for spline mapping
[4182] AXT_ERROR_SPLINE_EMPTY_MAP_NO	Spline Mapping number is empty
[4183] AXT_ERROR_SPLINE_NUM_ERROR	Spline point value is inapplicable
[4184] AXT_RT_MOTION_INTERPOL_VALUE	Invalid input value for interpolation
[4185] AXT_RT_ERROR_NOT_CONTIBEGIN	CONTIBEGIN function is not called in continuous interpolation
[4186] AXT_RT_ERROR_NOT_CONTIEND	CONTIEND function is not called in continuous interpolation
[4201] AXT_RT_MOTION_HOME_SEARCHING	Other function is called while in home search motion
[4202] AXT_RT_MOTION_HOME_ERROR_SEARCHING	Home search motion is fored to stop by the user or something from external
[4203] AXT_RT_MOTION_HOME_ERROR_START	Failed to start home search drive due to intialization problem
[4204] AXT_RT_MOTION_HOME_ERROR_GANTRY	Failed to execute Gantry enable during home search motion
[4251] AXT_RT_MOTION_POS_OUTOFBOUND	Configured position is either above the maximum value or below the minimum value
[4252] AXT_RT_MOTION_PROFILE_INVALID	Invalid setting for velocity profile
[4253] AXT_RT_MOTION_VELOCITY_OUTOFBOUND	Configured velocity is either above the maximum value or below the minimum value
[4254] AXT_RT_MOTION_MOVE_UNIT_IS_ZERO	Unit of the motion values is set as zero
[4255] AXT_RT_MOTION_SETTING_ERROR	Invalid setting for Velocity, Acceleration, Jerk, or Profile
[4256] AXT_RT_MOTION_IN_CONT_INTERPOL	Execuation of motion start or restart function before the completion of current continous interpolated motion
[4257] AXT_RT_MOTION_DISABLE_TRIGGER	Trigger output is disabled
[4258] AXT_RT_MOTION_INVALID_CONT_INDEX	Invalid setting for continuous interpolation index value
[4259] AXT_RT_MOTION_CONT_QUEUE_FULL	Continuous Interpolation queue of motion chip is full
[4260] AXT_RT_PROTECTED_DURING_SERVOON	Can not be used in servo-on status
[4261] AXT_RT_HW_ACCESS_ERROR	Board hardware access error has occurred. Check the status of the board or battery operation to check the status of power input

The contents of this manual can be changed without notice. The name of company, institution, production, people and event used in the examples are not real data. None of actual company, institution, product, people, or events are intended to be connected, and it shall not be interpreted that way. It is user's responsibility to comply with the corresponding copyright law. Separately from the right in the copyright, any part of this manual shall not be copied in any form or means (electronic, mechanical, paper copy, disk copy, or other method), shall not be coped for other purpose, and shall not be stored, introduced, or transmitted without explicit written approval of Ajinextek Co., Ltd.

Ajinextek Co., Ltd reserves all rights including patents, trademark right, copyright, or other intellectual property associated with the contents of this manual. Except the right explicitly given to you from Ajinextek Co., Ltd. by the written end user license agreement, providing this manual does not allow you to use any of patents, copyright, or other intellectual properties.